

HBI FPD

API Programming Reference

28/05/2025



DT Imaging Technology (Shanghai) Co., Ltd

Revision

Version	Date	Description	Comment
Ver 4.0.0.7	2021.05.14	Draft	
Ver 4.0.0.13	2021.10.11	First edition	
Ver 4.0.0.14	2021.11.07	Multi flat panel detector	
Ver 4.0.0.15	2021.11.12	Add new interface	
Ver 4.0.1.0	2021.12.12	Encryption	
Ver 4.0.1.1	2021.12.16	Exception, optimization	
Ver 4.0.1.2	2021.12.29	Exception, optimization	
Ver 4.0.1.3	2022.02.14	Exception, optimization	
Ver 4.0.1.4	2022.03.11	Add new interface, Optimization, Preview	
Ver 4.0.1.5	2022.07.02	Update	
Ver 4.0.1.6	2022.07.18	Update	
Ver 4.0.1.7	2022.07.02	Update	
Ver 4.0.1.8	2022.08.19	Update	
Ver 4.0.1.9	2022.09.06~2022.10.08	Update	
Ver 4.0.1.10	2022.10.08~2022.11.04	Update	
Ver 4.0.1.11	2022.11.17	Update	
Ver 4.0.1.12	2022.12.18	Update	
Ver 4.0.1.13	2022.12.15	Update	
Ver 4.0.1.14	2023.02.13~2023.02.28	Update	
Ver 4.0.2.4	2023.02.28~2023.06.30	Update	MH.YANG
Ver 4.0.2.5	2023.06.30~2024.01.08	Update	MH.YANG
Ver 4.0.2.9	2024.01.08~2025.05.28	Add new interface	Haojia.Ling

Contents

Contents	2
1. Introduction	8
1.1 Purpose	8
1.2 About SDK	8
1.3 Environment	9
1.4 Installation	9
1.4.1 Ethernet Gigabit Network Card (UDP).....	9
1.4.2 Ethernet Gigabit Network Card (UDP JUMBO)	10
1.4.3 Optical Fiber communication (PCIe card).....	12
1.4.4 Wireless Network Card (WLAN UDP)	16
2. summary	16
2.1 Data flow chart	16
2.2 Call Flowchart	17
2.3 About "instance handles and panel device ID"	20
2.4 About "Callback Functions"	21
2.5 About "Trigger Mode"	22
2.6 About "Image Calibration"	23
2.7 Command.....	24
2.7.1 Request Command.....	24
2.7.2 Response Command	25
2.8 Data Structure	26
2.8.1 Connection parameters	26
2.8.2 Image Properties	26
2.8.3 Trigger Mode Classification	27
2.8.4 Continuous collection and classification	27
2.8.5 Collection Mode Types	27
2.8.6 Collecting Image Parameters	28
2.8.7 Correction enable setting parameters.....	28
2.8.8 Download template file parameters	29
2.8.9 Mode parameter	31
2.8.10 Multiple Mode List	31
2.8.11 Other Parameters.....	32
2.9 Flat Panel Status	32
2.9.1 Static flat panel detector status of Ethernet port	32
2.9.2 Dynamic flat panel detector status of network port	32
2.9.3 Dynamic flat panel status of optical port	33
2.9.4 Wireless static tablet status	34
3. API description	35
3.1 Function Types.....	35
3.2 Function classification	35
3.2.1 Initialize and Release Resources.....	35
3.2.2 Registering Callback Functions	36

3.2.3 Connecting and disconnecting panel	36
3.2.4 Trigger mode and calibration enable settings	36
3.2.5 Single frame acquisition and continuous acquisition.....	36
3.2.6 Set frame rate, PGA gear, and Binning type switching.....	37
3.2.7 Single frame acquisition mode classification	37
3.2.8 Classification of continuous acquisition modes.....	37
3.2.9 Quick template generation and template download.....	39
3.2.10 Obtain image attributes, software and firmware versions.....	39
3.2.11 Obtain tablet parameters and restore factory settings	39
3.2.12 Zoom (ROI) Area Settings.....	39
3.2.13 Calibration template path.....	39
3.2.14 Image Display and Save	40
3.2.15 Set packet interval.....	40
3.2.17 Image Rotation/Mirroring.....	40
3.2.18 Firmware Template Management Method.....	40
3.2.19 Multiple Mode Other Interface	41
3.3 Interface Functions	42
3.3.1 HBI_Init.....	42
3.3.2 HBI_Destroy.....	42
3.3.3 HBI_ConnectDetector	42
3.3.4 HBI_ConnectDetectorUdp.....	43
3.3.5 HBI_ConnectDetectorJumbo.....	43
3.3.6 HBI_ConnectDetectorPcie.....	44
3.3.7 HBI_Connect DetectorEx	44
3.3.8 HBI_DisConnectDetector.....	45
3.3.9 HBI_RegEventCallBackFun.....	46
3.3.10 HBI_ResetDetector.....	46
3.3.11 HBI_ResetFirmware	46
3.3.12 HBI_GetFPDSerialNumber	47
3.3.13 HBI_GetSDKVersion.....	47
3.3.14 HBI_GetFirmwareVerion	48
3.3.15 HBI_GetError	48
3.3.16 HBI_GetFpdCfgIfo.....	48
3.3.17 HBI_GetImageProperty	49
3.3.18 HBI_SinglePrepare	49
3.3.19 HBI_SingleAcquisition.....	49
3.3.20 HBI_LiveAcquisition.....	50
3.3.21 HBI_StopAcquisition.....	51
3.3.22 HBI_SetSinglePrepareTime.....	51
3.3.23 HBI_GetSinglePrepareTime	52
3.3.24 HBI_SetLiveAcquisitionTime	52
3.3.25 HBI_GetLiveAcquisitionTime.....	53
3.3.26 HBI_TriggerAndCorrectDisplay	53
3.3.27 HBI_UpdateTriggerMode.....	54

3.3.28 HBI_UpdateCorrectEnable.....	54
3.3.29 HBI_GetCorrectEnable.....	54
3.3.30 HBI_SetSoftwareCorrect.....	55
3.3.31 HBI_GetSoftwareCorrect.....	55
3.3.32 HBI_SetAedIntegrateTm.....	56
3.3.33 HBI_GetAedIntegrateTm.....	56
3.3.34 HBI_SetAedThreshold.....	56
3.3.35 HBI_GetAedThreshold.....	57
3.3.36 HBI_SetsaturationValue.....	57
3.3.37 HBI_GetsaturationValue.....	57
3.3.38 HBI_SetClippingValue.....	58
3.3.39 HBI_GetClippingValue.....	58
3.3.40 HBI_SetLPFLevel.....	58
3.3.41 HBI_GetLPFLevel.....	59
3.3.42 HBI_SetPGAlevel.....	59
3.3.43 HBI_GetPGAlevel.....	60
3.3.44 HBI_SetBinning.....	60
3.3.45 HBI_SetBinning.....	61
3.3.46 HBI_SetPreviewMode.....	61
3.3.47 HBI_GetPreviewMode.....	62
3.3.48 HBI_SAecModeApply.....	62
3.3.49 HBI_SAecAcq.....	63
3.3.50 HBI_RegProgressCallBack.....	63
3.3.51 HBI_GenerateTemplate.....	64
3.3.52 HBI_DownloadTemplate.....	65
3.3.53 HBI_StopDownloadTemplate.....	65
3.3.54 HBI_InitDual.....	65
3.3.55 HBI_InitEx.....	66
3.3.56 HBI_DestroyEx.....	66
3.3.57 HBI_GetDetectorHandle.....	66
3.3.58 HBI_GetDetectorId.....	67
3.3.59 HBI_ActivateDetector.....	67
3.3.60 HBI_GetPreviewImageProperty.....	67
3.3.61 HBI_PgaBinningAcqTime.....	68
3.3.62 HBI_TriggerPgaBinningAcqTime.....	68
3.3.63 HBI_TriggerBinningAcqTime.....	69
3.3.64 HBI_PgaAcqTime.....	70
3.3.65 HBI_SetSelfDumpingTime.....	70
3.3.66 HBI_GetSelfDumpingTime.....	70
3.3.67 HBI_TriggerPgaBinningAcqTimeCorrect.....	71
3.3.69 HBI_OpenTemplateWizard.....	72
3.3.70 HBI_SetRoiMode.....	72
3.3.71 HBI_SetZoomModeRect.....	73
3.3.72 HBI_SetCorrectFileDirectory.....	73

3.3.73 HBI_GetCorrectFileDirectory	74
3.3.74 HBI_UpdateSaveOption	74
3.3.75 HBI_SetPacketIntervalTime	74
3.3.76 HBI_AutoWindowWidthWindowLevel.....	75
3.3.77 HBI_SetImageRotateAndFlip.....	75
3.3.78 HBI_Rotate90AndFlip.....	76
3.3.79 HBI_ResetCorrectTemplate	76
3.3.80 HBI_GetCurZoomRect.....	77
3.3.81 HBI_IsExistsOfCurCorrectTemplate	77
3.3.82 HBI_GetCurrentCorrectFileName.....	77
3.3.83 HBI_GetCurTempFileFullDirectory.....	78
3.3.84 HBI_GetCurrentCorrectFilePath	78
3.3.85 HBI_Prepare	78
3.3.86 HBI_DownloadTemplateByType.....	79
3.3.87 HBI_GetCurModeList	80
3.3.88 HBI_UpdateModeList	80
3.3.89 HBI_GetCurModeData	80
3.3.90 HBI_UpdateCurModeData	81
3.3.91 HBI_UpdateModeData	81
3.3.92 HBI_GetCurModeID	81
3.3.93 HBI_SwitchUserMode.....	82
3.3.94 HBI_ModeCorrectTempalteEnable	82
3.3.95 HBI_AllModeCorrectTempalteEnable	82
3.3.96 HBI_SaveModeList2File.....	83
3.3.97 HBI_DownloadModeTemplate.....	83
3.3.98 HBI_DownloadModeTemplateByType	83
3.3.99 HBI_DownloadModeTemplateByIdAndType	84
3.3.100 HBI_GetCurModeTemplateStatus	84
3.3.101 HBI_FirmwareManagementTempalteType.....	84
3.3.102 HBI_SetValidModeItemNumber	85
3.3.103 HBI_IsMutilMode	85
3.3.104 HBI_MUpdateTriggerMode	86
3.3.105 HBI_MSetBinning.....	86
3.3.106 HBI_MSetPGAlevel.....	87
3.3.107 HBI_MSetRoiMode.....	87
3.3.108 HBI_MSetZoomModeRect.....	88
3.3.109 HBI_MSetSelfDumpingTime	88
3.3.110 HBI_MSetLiveAcquisitionTime.....	89
3.3.111 HBI_MSetSinglePrepareTime.....	89
3.3.112 HBI_MSetFpsOfTime.....	90
3.3.113 HBI_MPgaAcqTime	90
3.3.114 HBI_PgaBinningAcqTime	90
3.3.115 HBI_MTriggerAndCorrectDisplay	91
3.3.116 HBI_MTriggerBinningAcqTime.....	92

3.3.117 HBI_MTriggerPgaBinningAcqTime.....	92
3.3.118 HBI_MTriggerPgaBinningAcqTimeCorrect.....	93
3.3.119 HBI_GetModeData	94
3.3.120 HBI_GetModeListSize	94
3.3.121 HBI_ModeExists	95
3.3.122 HBI_CheckCurtMode.....	95
3.3.123 HBI_InitDetectorParams	95
3.3.124 HBI_SetWindowMode	96
3.3.125 HBI_GetWindowMode.....	96
3.3.126 HBI_SetBinningDivisor.....	97
3.3.127 HBI_GetBinningDivisor.....	97
3.4 Callback Event.....	97
3.5 Return Code.....	104
4. Detector calibration	108
4.1 About Calibration.....	108
4.2. Quick template generation.....	110
4.2.1 Offset Template.....	111
4.2.2 Gain Template.....	111
4.2.3 Defect Template.....	112
4.2.4 Template Download.....	112
4.2.5 Calibration Enable	113
4.3 Template Wizard Resources.....	113
4.3.1 Open Template Wizard	113
4.3.2 Offset Template Generation Page	114
4.3.3 Gain Template Generation Page	114
4.3.4 Defect template generation page.....	115
4.3.5 Configuration Page	115
4.4 Template Management.....	116
4.4.1 Local Template Management.....	116
4.4.2 Firmware Template Management	117
5. Secondary development.....	117
5.1 Interface Description	118
5.2 Software deployment	118
5.3 Binning Integration.....	119
5.3.1 Steps.....	119
5.3.2 Offset Template Generation and Download.....	123
5.3.3 Gain template generation and download.....	124
5.3.4 Defect template generation and download.....	126
5.3.5 Progress Status Callback Function.....	128
5.3.6 Update calibration enable	130
5.4 Integration of multiple modes.....	131
5.4.1 Steps.....	132
5.4.2 Multi Mode Management	136
5.4.3 Offset Template Generation and Download.....	145



DT Imaging Technology (Shanghai) Co., Ltd	
5.4.4 Gain template generation and download.....	147
5.4.5 Defect template generation and download.....	149
5.4.6 Progress Status Callback Function.....	152
5.4.7 Status of calibration template	154
5.4.8 Update calibration enable	155
5.5 Problems and Suggestions	156
6. Image overlay	157
6.1 Software Overlay	157
6.2 Firmware Overlay	157
7. Rotate and Mirror	158
7.1 Software Rotation and Mirroring.....	159
7.2 Firmware Mirroring	159
8. Preview mode	159
9. SAEC mode	160
10. Modify Detetcor's IP	162
11. Spot and perspective	164
12. "Dual panels" solution	164
13. Develop Demo	165
Appendix I: Table of panel types and resolutions.....	166
Appendix II: Configuration Files.....	167
Appendix III: Callback Function Parameters and States	167
Appendix IV: Switching from old version to the latest version	168
Appendix V: Window width and window level algorithm.....	172

1. Introduction

1.1 Purpose

The main purpose is to facilitate users to quickly complete secondary development, and document support is provided. Additionally, users can refer to the SDK Demo source code project and its instruction manual.

The target audience is users who are conducting secondary development based on DT imaging flat panel detectors.

1.2 About SDK

For the convenience of user integration development, we provide a standard dynamic link library (DLL) interface. The user's upper computer can call the interface program encapsulated in the dynamic link library by importing it to achieve control and image acquisition of the tablet detector. At present, this version is highly integrated and supports communication methods including **wired, wireless, and optical communication**. In addition, it supports a **"dual (multi) board"** solution.

Main content: The functions, parameters, calling process, correction template generation/download, and development support of the SDK function library interface. The following mainly introduces "single board" integration, and we will separately explain the "dual board" solution in Chapter 12.

The overall structure of the software system and the main functions of the interface layer include image acquisition and calibration, detector calibration, and system management, as shown in Figure 1.

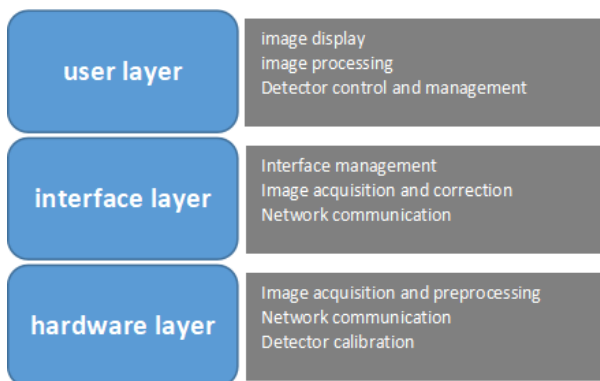


Figure 1

This interface software is implemented using **C++**. The return type is a pointer type, indicating that a handle or string address or custom data needs to be returned.

This software can run on **32-bit** or **64 bit** Windows operating systems.

Supports Ethernet gigabit network cards and fiber optic communication.

This interface is based on the C/C++ development language, pay attention to dependency libraries.

Including Header Files:

HbiFpd.h // Interface function file

HbiType.h // parameter, structure data type file



DT Imaging Technology (Shanghai) Co., Ltd

HbiError.h // Error List File

Linking the Library with dynamic library files:

Generate Library: HBISDKApi.lib // Static Library
HBISDKApi.DLL // Dynamic Library
Dependency library: opencv_World341.dll // OpenCV library

1.3 Environment

Operating system: Windows 7 or above, both 32-bit and 64-bit are supported, Linux version will be released later

Memory: >= 4GB

Hard disk: >= 10G

Gigabit network card or support for jumbo

PCIe card: fiber optic communication

1.4 Installation

At present, this interface supports four communication methods: Ethernet Gigabit Network Card UDP, Ethernet Gigabit Network Card UDP (JUMBO), Fiber Optic PCIE Card, and Wireless Network Card UDP.

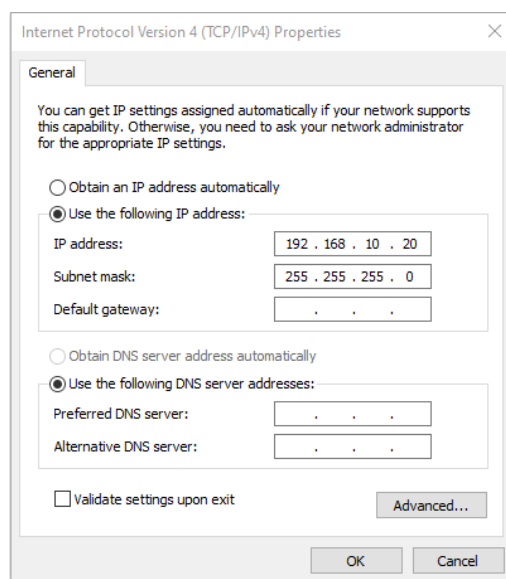
You can modify the tablet IP through the XDiscovery tool.

1.4.1 Ethernet Gigabit Network Card (UDP)

For detailed Ethernet settings, please refer to the “DT Imaging XDiscovery User Manual V *. *. *. * _xxx.pdf”.

1> Environment configuration:

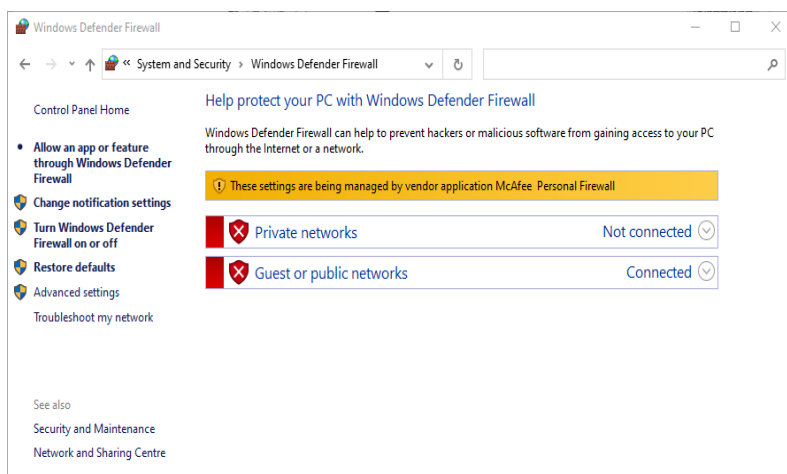
- Local IP address: 192.168.10.20
- Remote Device IP Address: 192.168.10.40



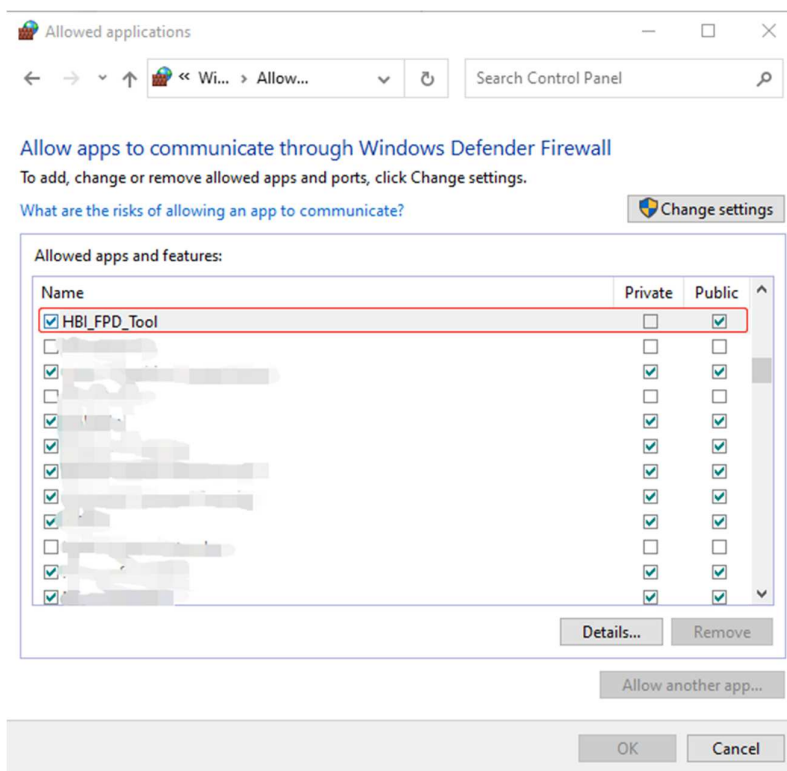
Tips: When opening the tool software for the first time and prompted to disable or allow when connecting, select

allow. Otherwise, there may be a successful connection failed or a missing image. If there is such a situation, pay attention to checking and configuring the firewall. There are two methods:

Method 1: Directly close the firewall;



Method 2: Firewall exception setting - "Allow applications to pass through Windows Firewall";



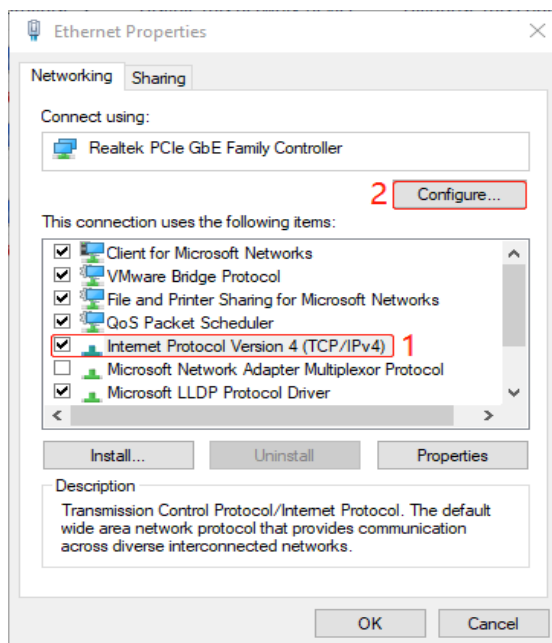
1.4.2 Ethernet Gigabit Network Card (UDP JUMBO)

The configuration method for Ethernet Gigabit Network Card UDP is the same as '1.4.1', only the default parameters of the network card need to be modified (provided that the network card supports JUMBO settings), as follows:

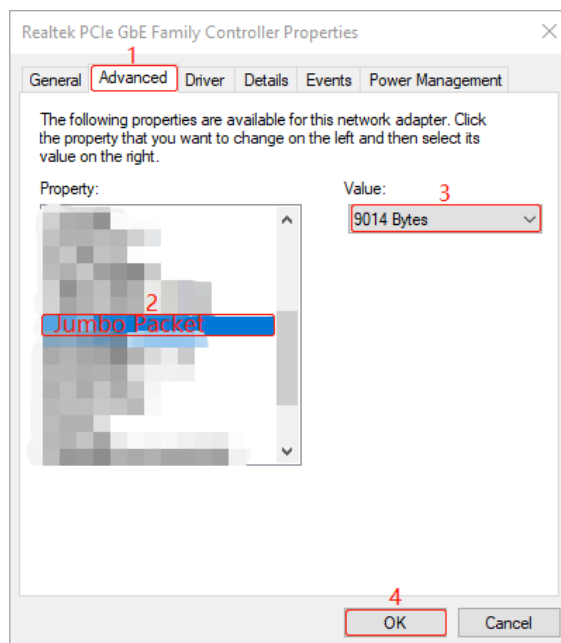
1> Jumbo packet setting

Note: Jumbo packet require network cards, and some older versions of network cards do not support Jumbo packet.

1》 Control Panel ->Select "Network Card" ->Right click "Properties" ->Open the "Ethernet Properties" page, as follows:



2》 "Advanced" - "Jumbo Packet" - MTU Value "9014 Bytes" -" OK ".



3》 "Advanced" - "Receive Buffers" - "Value" 2048 -" OK ".



Reference value:

Receive buffer: 2048

Transit buffer: 2048

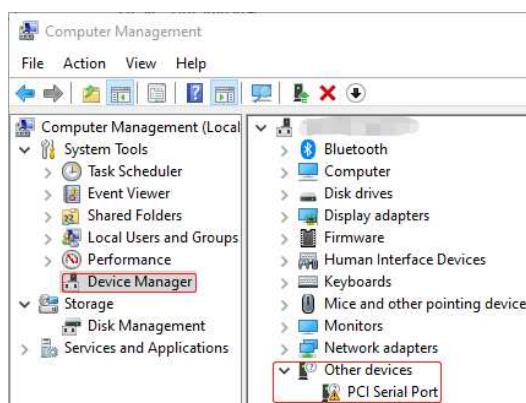
Jumbo packet: supports 9KB

1.4.3 Optical Fiber communication (PCIe card)

1> Environment configuration:

After installing the PCIe device card, connect the fiber optic cable to the tablet and PC, and connecting the power supply.

Open "Device Manager" ->"Other Devices" ->"PCI Serial Port", as shown in the figure below, a yellow exclamation mark appears!.



Note: Now the company has adopted Microsoft digital signature, you can skip the step of disabling driver signature and installing certificate, and directly update the driver of the PCIe card.

2> Prohibit driver signature

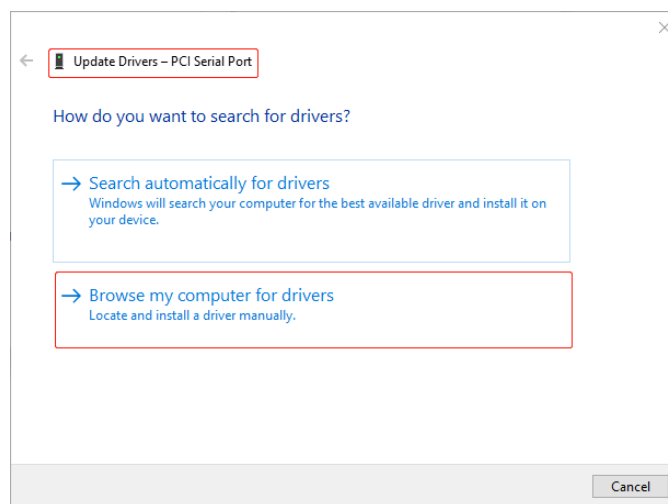
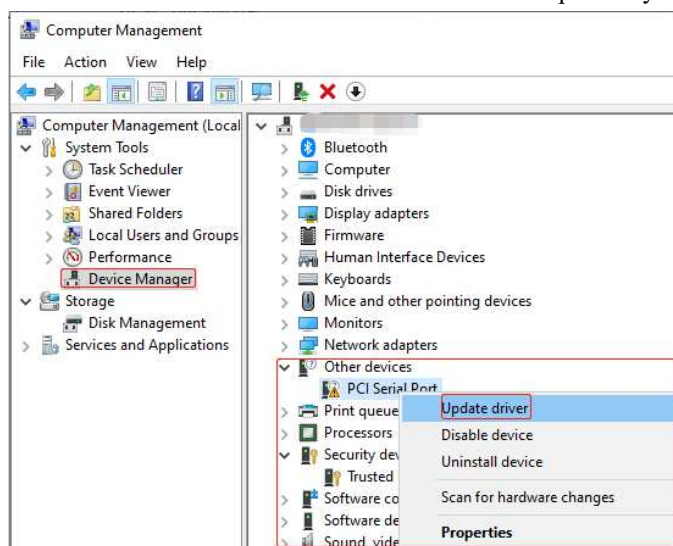
- 1》 Open the DOS window in administrator mode;
- 2》 Enter the "bcdedit.exe/set testing on" - "Enter" key;
- 3》 Restart the system, and the words "Test Mode" appear in the bottom right corner of the desktop.

4》 Double click on “XDMA.cer”, other actions are the default.

XDMA_Driver	2021/5/14 12:48
XDMA.cer	2021/5/14 10:54
XDMA.inf	2021/5/14 10:54
XDMA.pdb	2021/5/14 10:54
XDMA.sys	2021/5/14 10:54

3> Installing Drivers

Right click on "PCI Serial Port" ->Select "Update Driver " ->"Brows my computer for drivers" ->Select "XDA.sys" ->"Always Install This Driver Software" ->The installation is completed by default.



Win7_Release > XDMA_Driver	
名称	修改日期
WdfCoinstaller01011.dll	2018/8/9 14:43
xdma.cat	2021/5/14 10:54
XDMA.inf	2021/5/14 10:54
xdma.lib	2021/5/14 10:54
XDMA.sys	2021/5/14 10:54

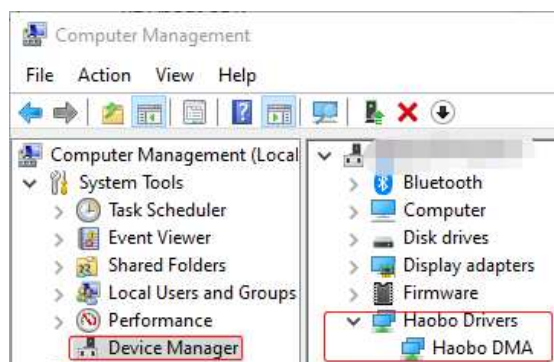
When a Windows Security Alert appears, choose to always install this driver software.

← Update Drivers – Haobo DMA

Windows has successfully updated your drivers

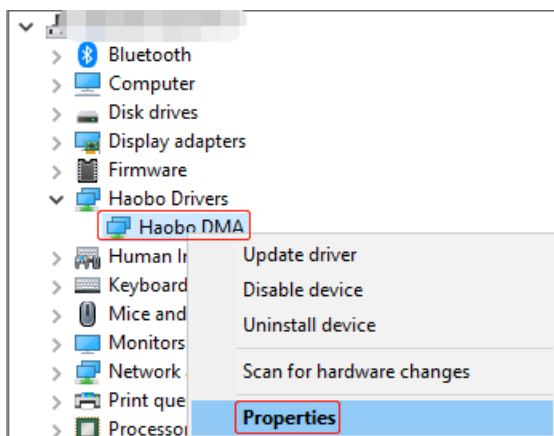
Windows has finished installing the drivers for this device:

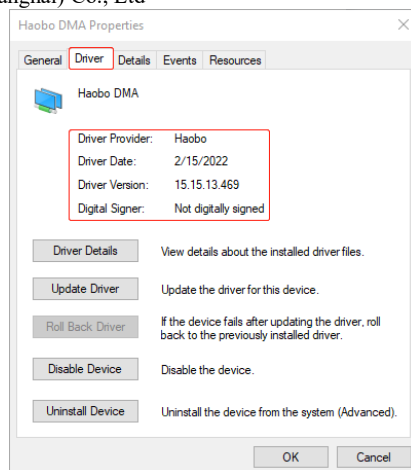
Haobo DMA



At this point, the PCIe driver has been successfully installed.

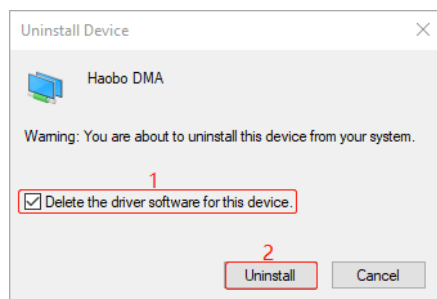
4> Restart and verify





5> Update and uninstall

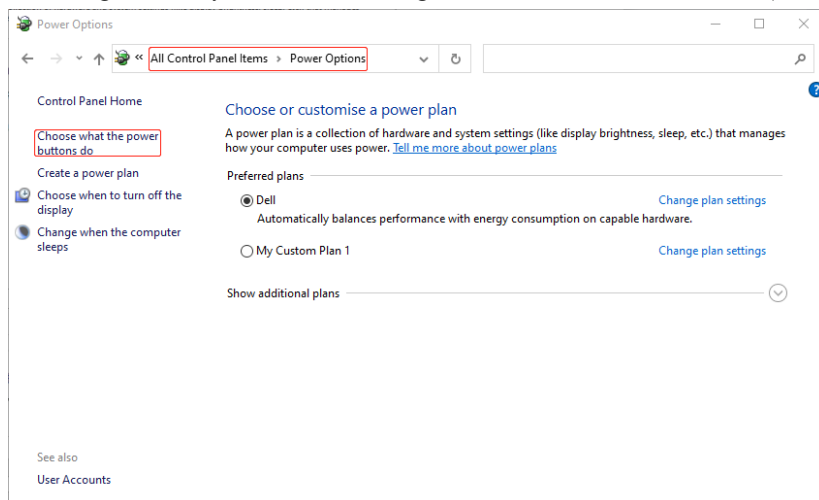
To update the PCIe device driver, first uninstall the driver and then reinstall it after completion.

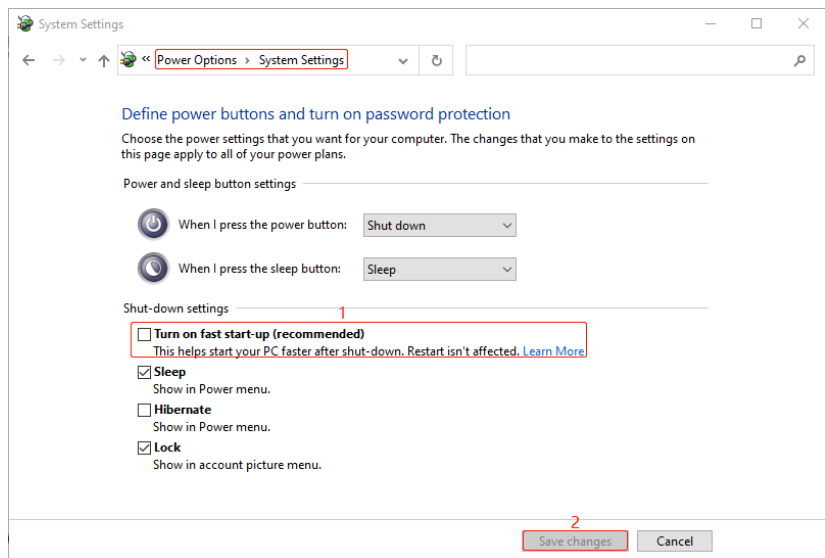
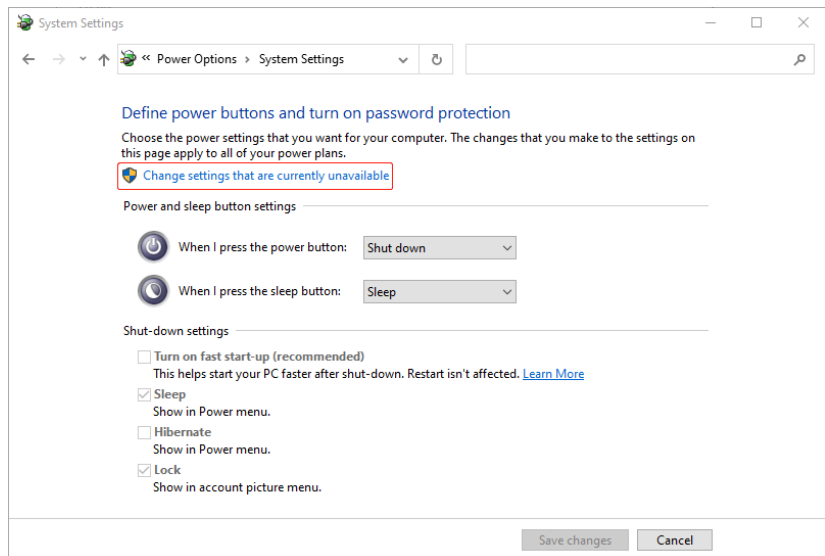


6> Set startup options

Mainly targeting the Win10 system, the purpose is to avoid failed to load PCIe device drivers during startup.

Steps: "Control Panel" ->"All Control Panel Items" ->"Power Options" ->"Select the function of the power button" ->"Change currently unavailable settings" ->"Cancel" Start Quick Start (recommended) "->" Save Changes ".





1.4.4 Wireless Network Card (WLAN UDP)

Due to some content being verified and the product not being released, the detailed configuration is temporarily omitted.

2. summary

2.1 Data flow chart

The data between this software and detector devices can be divided into two types: command data and image data. The command data, connection, collection, setting or acquisition of parameters, and image data bright field diagram need to be linked with the high-voltage generator. The data flow is as follows, as shown in Figure 2-1 and

Figure 2-2.

Command Data Flowchart

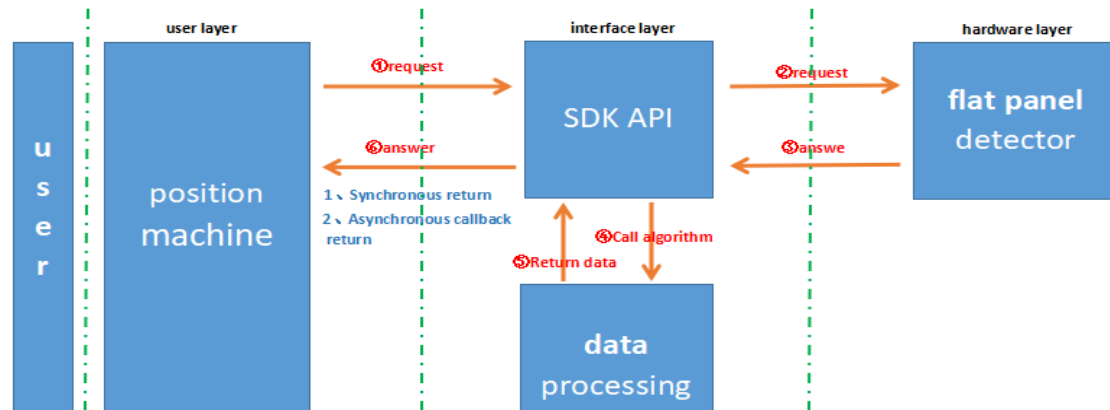


Figure 2-1

Image data flowchart

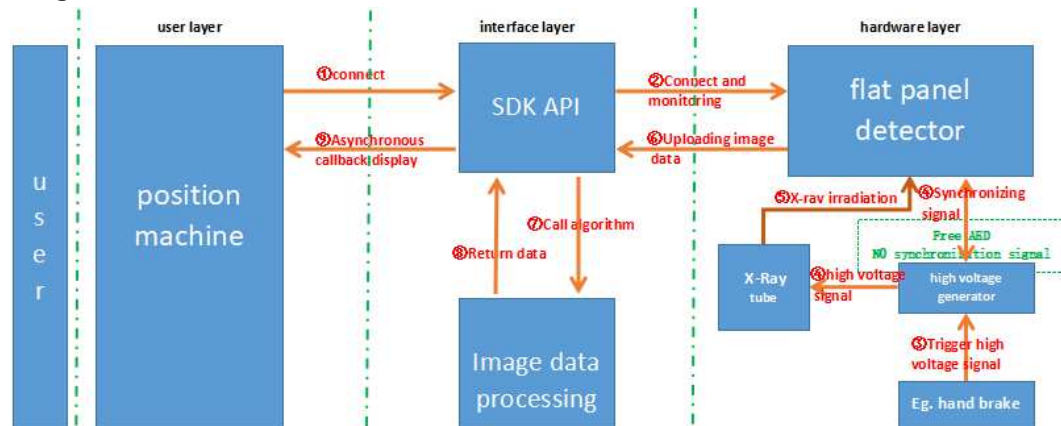


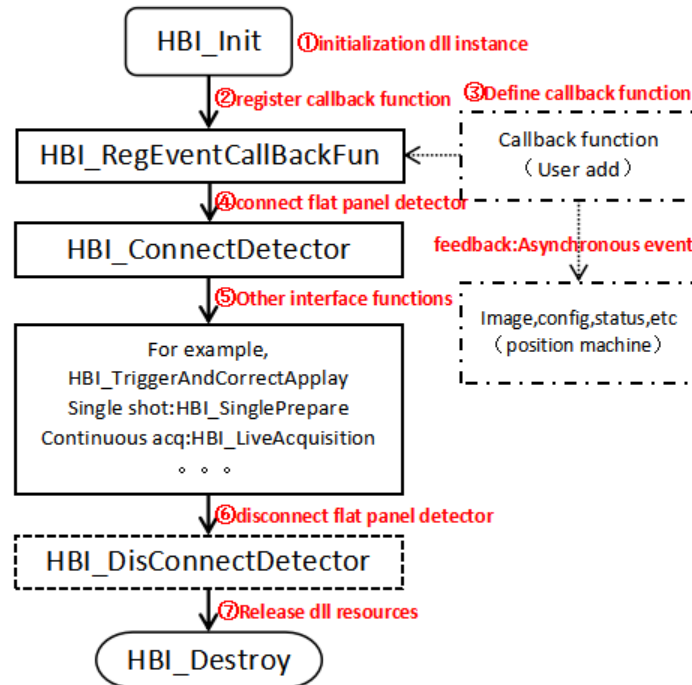
Figure 2-2

In addition, the image data differs in triggering modes such as soft trigger, high voltage trigger, "Free AED" trigger, and "Dynamic: FpdContinue" trigger, depending on whether synchronization signals are sent to the panel simultaneously with different types of tablets.

So soft triggering, high voltage triggering, and Free AED triggering are applied to static tablets, while "Dynamic: FpdContinue" is applied to dynamic high frame rate tablets.

2.2 Call Flowchart

Main process flowchart:



Main process:

Start ->Initialize 'HBI_Init' ->Register callback function 'HBI_RegEventCallBackFun' -> Connect to detector 'HBI_ConnectDetector' -> Set Trigger Mode and Enabling Relationship 'HBI_TriggerAndCorrectApply' ->Send acquisition command (single frame acquisition 'HBI_SinglePrepare' or continuous acquisition 'HBI_liveAcquisition') ->Disconnect 'HBI_DisConnectDetector' -> Release DLL resource 'HBI_Destroy' -> ends.

Attention: HBI_Destroy consists of two parts (disconnecting and releasing resources).

Users obtain tablet status, images, configurations, and other data through callback functions.

Other operational flowchart:

Parameter settings:

HBI_Set * ->Update Parameters ->HBI_Get * ->Get the latest configuration

Single frame acquisition:

Method 1: Prepare with a delay of 0, HBI_Single Prepare -> HBI_SingleAcquisition ->Notification receiving thread ->Receive data single frame processing

Method 2: Prepare delay>0, HBI_SinglePrepare ->Notify receiving thread ->Receive data single frame processing
The prepare delay time can be achieved through HBI_SetSinglePrepareTime modification.

Multi frame acquisition:

HBI_LiveAcquisition ->Notify receiving thread, pass in parameters ->Receive data for multi frame processing

The frame rate (also known as continuous acquisition time interval) can be achieved through dynamic Jumbo packet and optical port 'HBI_SetSelfDumpingTime' modification, while the standard static or wireless version is modified through 'HBI_SetLiveAcquisitionTime' modification.

The parameter of frame number for continuous collection is 0, and needs to be sent_stops continuous collection by



DT Imaging Technology (Shanghai) Co., Ltd

‘HBI_StoppAcquisition’ function. Otherwise, if the parameter of frame number is greater than 0, the specified collection frame rate will automatically stop after completion;

Offset template generation:

Generate template ‘HBI_GenerateTemplate’-> Update Calibration Enable ‘HBI_UpdateCorrectEnable’.

Gain template generation:

Generate template ‘HBI_GenerateTemplate’-> - Download Template ‘HBI_Download Template’->Update Calibration Enable ‘HBI_UpdateCorrectEnable’.

Defect template generation:

Generate template ‘HBI_GenerateTemplate’-> Download Template HBI_Download Template’ ->Update Calibration Enable ‘HBI_UpdateCorrectEnable’.

Notice:

1》 Before the panel connection is successful, the set parameters will not take effect because the parameters cannot be synchronized with the firmware. For example, a user calls The ‘HBI_SetenableCalibrate’ interface before connection will not take effect;

2》 Continuously setting parameters, note that the interface is asynchronous. After successfully issuing the setting command, the firmware needs time to write to the storage unit and provide feedback to the SDK for synchronization. There is a time difference between them. Therefore, it is best to have a delay when setting parameters continuously, otherwise the parameters will be overwritten by old parameters, indicating that the parameters have not been modified;

3》 If the parameters do not require real-time updates, it is recommended to fix them in the firmware and use them directly (some are indeed unnecessary). Generally, only ‘HBI_TriggerAndCorrectDisplay’ is needed to update the trigger mode calibration enable relationship;

4》 Similarities and differences in setting parameter feedback information

At present, after successfully setting parameters for some interfaces, the SDK synchronizes and provides feedback through callback functions, with some feedback on firmware parameters, as shown in the ECALLBACK_TYPE_ROM_UPLOAD event, some feedback confirmation information, see event ECALLBACK_TYPE_SET_CFG_OK.

5》 Communication methods, triggering modes, image calibration, and application differences

The current image calibration is divided into offset calibration, gain calibration, and defect calibration.

Type	Communication	Trigger Mode	Offset Calibration	Gain Calibration	Defect Calibration	Application
Static	00: UDP	01-static: software trigger mode; 03- static: hvg trigger mode; 04- static: AED trigger mode; 08- static: SAEC trigger mode	01: software pre-offset; 02: firmware post-offset;	01: software gain;	01: software defect	Medical, Veterinary
	03: UDP(Wireless)	01-static: software trigger mode; 03- static: hvg trigger mode; 04- static: AED trigger mode; 08- static: SAEC trigger mode;	01: software pre-offset; 02: firmware post-offset;	01: software gain;	01: software defect	Medical, Veterinary
Dynamic	01: UDP(Jumbo)	05-dynamic: hvg pulse sync in mode; 06- dynamic: fpd pulse sync out mode;	03: firmware pre-offset;	02: firmware gain;	02: firmware defect;	Medical, Industry

		07- dynamic: free run mode;				
	02: Optical Fiber (PCIe)	05-dynamic: hvg pulse sync in mode; 06- dynamic: fpd pulse sync out mode; 07- dynamic: free run mode;	03: firmware pre-offset;	02: firmware gain;	02: firmware defect;	Medical, Industry

a、Firmware calibration requires downloading the template generated by the template to the firmware and enabling calibration before it can take effect;

b、Calibration template generation. Currently, we support two ways to generate templates: through DT tool software or by user calling template interface functions. Please refer to "5. Secondary development" for details.

c、The dynamic panel firmware pre-offset calibration template can be generated by the firmware through commands (note to close the ball tube), as follows

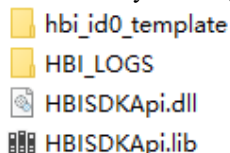
(1) Flat panel connection function, set the last parameter of 'HBI_ConnectDetector' to 1, and automatically generate a pre offset template during connection;

(2) Generate a template with the type of 'OFFSET_TEMPLATE_TYPE' through the interface 'HBI_GenerateTemplate' function;

(3) Continuous acquisition command 'HBI_LiveAcquisition' function, set acquisition parameters, eAqccmd=LIVE_ACQ_DEFAULT_TYPE, eLivetype=ONLY_PREOFFSET (only generate pre offset template) or eLivetype=PREOFFSET_IMAGE (continuously capture images after generating pre offset template), all other parameters are default. Static detectors are generally calibrated using software, while dynamic detectors are calibrated using firmware;

d、Pay attention to the network card settings when using Ethernet port jumbo packet types (supports Jumbo packet);
6》Template file and log file directory

The current template file and log file are automatically generated based on the dynamic library Dll path. Therefore, they are located in the same directory as the dynamic library Dll file, as shown in the following figure:



The root directory of the template for "panel A" is "hbi_id0_template", and vice versa, "panel B" is "hbi_id1template".

Generate calibration templates and download them in the "DT SDK Development DEMO Engineering Description Ver *. *. *. *. doc".

The firmware upgrade interface is not currently available.

2.3 About "instance handles and panel device ID"

A handle is an identifier used to identify an object or project, and can be used to describe forms, files, etc. It is worth noting that handles cannot be constant.

The reason why Windows needs to set up handles is fundamentally due to issues with memory management mechanisms, namely virtual addresses. In short, the address of data needs to be changed, and after the change, someone needs to record and manage the change. Therefore, the system uses handles to record the change of data address. In programming, a handle is a special



DT Imaging Technology (Shanghai) Co., Ltd

type of smart pointer that is used when an application needs to reference memory blocks or objects managed by other systems (such as databases, operating systems).

Handle is a term often mentioned in C++ programming design. It is not a specific, fixed and unchanging data type or entity, but represents a broad concept in programming design. A handle generally refers to a method of obtaining another object - a generalized pointer, which may take the form of an integer, an object, or a real pointer, and its purpose is to establish a unique connection with the object being accessed.

In C++, to access an object, it is usually possible to establish a pointer to the object. However, in many specific applications, using pointers directly to represent objects is not a good solution.

A handle is the identifier of an object or instance in a Windows system, which includes modules, application instances, windows, controls, bitmaps, GDI objects, resources, files, etc.

From a data type perspective, it is just a 16 bit unsigned integer. Applications always obtain a handle by calling the Windows API, which can then be used by other Windows functions to reference and manipulate the corresponding kernel object. A handle can be left empty like a pointer, making it meaningless and not representing any kernel object.

The above content is sourced from Baidu Baike

"Instance handle and panel device ID" are unsigned pointers and unsigned integer variables abstracted by R&D personnel to facilitate the management of SDK objects and support applications such as "dual panels". They serve as important parameters for interface functions, such as the default single flat panel, handles handle 1, ID 0, and dual flat panels, A panel handles handle 1, ID 0, and B panel handles handle 2, ID 1.

2.4 About "Callback Functions"

A callback function is a function called through a [function pointer](#). If you [pass the pointer](#) (address) of a function as a [parameter](#) to another function, and this pointer is used to call the function it points to, we call it a callback function. The callback function is not directly called by the implementer of the function, but by another party when a specific event or condition occurs, used to respond to that event or condition.

A callback is any method that is called by another method with a method as its first parameter. Many times, a callback is a method that is called when certain events occur.

The above content is sourced from Baidu Baike.

In order to prevent UI blocking, some interfaces adopt asynchronous interfaces and use registration callback functions. There are generally two types of registration functions: global functions and static member functions:

```
Static int handleCommandEvent (void * _context, int nfpdId, unsigned char command, void * pvParam1, int nParam2, int nParam3, int nParam4);
```

```
Class CXXX{
```

```
.....
```

```
Public:
```

```
Static int handleCommandEvent (void * _context, int nfpdId, unsigned char command, void * pvParam1, int nParam2, int param3, int param4);
```

```
.....
```

```
};
```

**Parameter Description:**

```
// @pContext:      parameter 1, object pointer.
// @ ufpdId:       parameter 2, for example, FPD ID.
// @byteEventId:   parameter 3, event ID, refer to enum eCallbackEventCommType of "HbiType.h".
// @pvParam1:      parameter 4, configuration structure pointer or image structure pointer
// @nParam2:       parameter 5, for example, data size or status
// @ nParam3:      parameter 6, for example, frame id
// @ nParam4:      parameter 7, for example, frame rate or status
// callback function return value type: the return value is 1 or 0, 1: success, 0: failed
```

If there is no return value or the return value is 0 or defaults to 0, the SDK will be considered as a feedback failed, and the SDK will be recorded in the log. Therefore, if the upper computer receives any callback message from the SDK, please return 1.

For example:

```
int handleCommandEvent (void * _context, unsigned char command, void * pvParam1, int len, int nParam2, int param3, int param4)
{
    ...
}
```

//At the end of the function

Return 1// Convention: Upon receiving any callback message (including success or failed), return 1 upon completion, indicating that the upper computer has received it!

```
}
}
```

2.5 About "Trigger Mode"

Trigger mode is a concept abstracted from the way the panel collects images, and currently supports trigger mode including:

Type	Name	Remarks
0x01: STATIC_SOFTWARE_TRIGGER_MODE	Static Collection - Soft Trigger Mode	All operations on the panel are controlled by software commands, and there is no need to connect a high-voltage trigger line between the flat detector and the high-voltage generator, allowing for single frame image acquisition
0x02: STATIC_PREP_TRIGGER_MODE	Static collection - Clear mode	Not yet enabled, not supported
0x03: STATIC_HVG_TRIGGER_MODE	Static Collection - High Voltage Trigger Mode	Between the detector and the high-voltage generator, an external high-voltage trigger line is required for flat panel acquisition operation, which is not controlled by software instructions, but is triggered and controlled by the high-voltage output trigger signal to capture a single frame image
0x04: STATIC_AED_TRIGGER_MODE	Static Collection - AED Trigger Mode	Between the detector and the high-voltage generator, there is no need to connect a high-voltage trigger line or send instructions from software. The high-voltage is directly exposed, and the detector will automatically sense the exposure internally, triggering a single acquisition operation to capture a single frame image

0x05: DYNAMIC_HVG_TRIGGER_MODE	Dynamic acquisition - High voltage synchronization mode	Between the detector and the high-voltage generator, an external high-voltage trigger line is required for flat panel acquisition operation, which is not controlled by software instructions, but is triggered by the high-voltage output trigger signal to continuously capture images
0x06: DYNAMIC_FPD_TRIGGER_MODE	Dynamic collection - FPD synchronization mode	Between the detector and the high-voltage generator, a high-voltage trigger line needs to be connected, and software needs to send instructions. The detector internally exposes the high-voltage signal and continuously collects images
0x07: DYNAMIC_FPD_Continue_MODE	Dynamic collection - FPD Continue mode	All operations on the tablet are controlled by software commands, and there is no need to connect a high-voltage trigger line between the tablet detector and the high-voltage generator, allowing for continuous image acquisition
0x08: DYNAMIC_FPD_SAEC_MODE	Static acquisition - SAEC Mode mode	For some tablets: In this mode, first capture a thumbnail (Preview) image, and then capture a normal image

2.6 About "Image Calibration"

Calibration type currently supports three calibration methods: offset calibration, gain calibration, and defect calibration, which belong to image preprocessing. In the following content, offset refers to offset calibration, gain refers to gain calibration, and defect refers to bad point calibration.

Calibration type	Remarks
Offset Calibration - Offset	Eliminating physical differences between image sensors and hardware
Gain calibration - Main	Calibrate the conical beam of the spherical tube and convert the imaging features of the conical beam into a flat image
Bad point calibration - Defect	Calibrate physical defects on image sensors

Calibration template, currently image calibration is achieved by saving calibration coefficients through the calibration template and reading them into memory. The calibration algorithm calibrates the image based on the calibration coefficients.

Calibration type	Template (hbi_id0_template, where id0 represents panel A)
Offset Calibration - Offset	Offset_panel size_Binning type_Frame rate parameter. raw, such as offset_02_1_00250.raw
Gain calibration - Main	Gaina.raw and gainb.raw (under the directory composed of tablet model, binning type, and PGA gear)
Bad point calibration - Defect	Defect.raw (composed of tablet model, binning type, and PGA gear in the directory)

Software calibration and Firmware calibration. "Software calibration" refers to the automatic completion of image calibration (completed by the computer) by the SDK after setting the software calibration enable. It is suitable for low frame rate or single frame acquisition. "Firmware calibration" refers to setting up firmware calibration enable, scanning and reading sensor data through firmware, and performing image algorithm calibration. The SDK will receive the calibrated image (completed by the detector end), which is highly efficient and suitable for high frame rate acquisition.

Template generation, the Offset template is a template generated from a set of dark images, the Gain template requires at least one set of bright image generated templates, and the Defect template requires at least three sets of bright image generated templates with different doses. The bright image must be an empty field and cannot be generated by occlusions, etc. Users can use the DT tool software to use template wizards for distributed generation or integrate template generation interface functions



DT Imaging Technology (Shanghai) Co., Ltd

with high-pressure generation. For detailed information, please refer to "2. Second development". In the "firmware calibration" section, the Offset calibration template can be used to generate firmware through commands, as it is a dark image. The speed is relatively fast and the effect is good.

Template download is aimed at dynamic high frame rate detector. In order to improve the frame rate of the panel and reduce time loss caused by SDK calibration of images, "firmware calibration" is used here. The prerequisite is to first download the appropriate template to the firmware, and then use firmware calibration to enable it. The panel is uploaded to the SDK and upper computer software to obtain the calibrated image. This method is highly efficient. Therefore, the necessary step here is to download the template. In the "firmware calibration" section, the offset calibration template does not need to be downloaded because it is generated by the firmware and already exists in the firmware. However, it is best to update the offset calibration template through the command during the first startup or power outage.

Calibration enable refers to enabling the correct calibration to enable when the calibration template already exists. The current image will be automatically calibrated and fed back to the upper computer. If the enable is cancelled, the original image will be fed back. Calibration has a sequential order, with the default being to first perform offset calibration, then perform Gain calibration, and finally perform Defect calibration. Offset calibration cannot be skipped for Gain and Defect calibration. Similarly, completing offset and Gain calibration before performing Defect calibration is necessary, but only offset calibration or both can be performed.

The following is the calibration enable status:

Calibration type	Calibration enable status
Offset calibration - Offset	0- Do not perform offset calibration; 1-software pre offset calibration; 2-firmware post offset calibration; 3-firmware pre offset calibration;
Gain calibration	0- Do not perform gain calibration; 1-software gain calibration; 2-firmware gain calibration;
Bad Point Correction - Defect	0- Do not perform defect calibration; 1-software defect calibration; 2-firmware defect calibration;

2.7 Command

2.7.1 Request Command

//Notice: Request Command

Enum eRequestCommType

```
{
    EDL_COMMON_TYPE_INVALVE      = 0x00,
    EDL_COMMON_TYPE_GLOBAL_RESET = 0x01,
    EDL_COMMON_TYPE_PREPARE      = 0x02,
    EDL_COMMON_TYPE_SINGLE_SHORT = 0x03,
    EDL_COMMON_TYPE_LIVE_ACQ      = 0x04,
    EDL_COMMON_TYPE_STOP_ACQ      = 0x05,
    EDL_COMMON_TYPE_PACKET_MISS   = 0x06,
    EDL_COMMON_TYPE_FRAME_MISS    = 0x07,
    EDL_COMMON_TYPE_DUMMPING      = 0x08,
    EDL_COMMON_TYPE_END_RESPONSE  = 0x0A, // End response packet
    EDL_COMMON_TYPE_CONNECT_FPD   = 0x0B, // connect fpd
    EDL_COMMON_TYPE_DOWNLOAD_IMAGE = 0x0C, // wireless download image
    EDL_COMMON_TYPE_SLEEP_STATE   = 0x0D, // wireless set sleep state
}
```

```

EDL_COMMON_TYPE_WAKE_UP          = 0x0E, // wireless wake up
EDL_COMMON_TYPE_DISCONNECT_FPD    = 0x0F, // disconnect fpd
EDL_COMMON_TYPE_SET_RAM_PARAM_CFG = 0x10,
EDL_COMMON_TYPE_GET_RAM_PARAM_CFG = 0x11,
EDL_COMMON_TYPE_SET_ROM_PARAM_CFG = 0x12,
EDL_COMMON_TYPE_GET_ROM_PARAM_CFG = 0x13,
EDL_COMMON_TYPE_SET_FACTORY_PARAM_CFG = 0x14,
EDL_COMMON_TYPE_GET_FACTORY_PARAM_CFG = 0x15,
EDL_COMMON_TYPE_RESET_FIRM_PARAM_CFG = 0x16,
EDL_COMMON_UPLOAD_OFFSET_TEMPLATE = 0x2E, //add by MH.YANG 30/12
EDL_COMMON_UPLOAD_GAIN_TEMPLATE   = 0x2F, // Upload gain template
EDL_COMMON_UPLOAD_DEFECT_TEMPLATE = 0x30, // Upload defect template
EDL_COMMON_DEFECT_AUTHOR           = 0x31, //
EDL_COMMON_ERASE_FIRMWARE          = 0x4F, // Erase old firmware package request
EDL_COMMON_UPDATE_FIRMWARE         = 0x50, // Update new firmware package request
EDL_COMMON_TYPE_SET_AQC_MODE       = 0xFF
};

```

2.7.2 Response Command

//Notice: Response Command

Enum eCallbackEventCommType

```

{
    ECALLBACK_TYPE_INVALID           = 0X00,
    ECALLBACK_TYPE_COMM_RIGHT        = 0X01,
    ECALLBACK_TYPE_COMM_WRONG        = 0X02,
    ECALLBACK_TYPE_DUMMPLING         = 0X03,
    ECALLBACK_TYPE_ACQ_END            = 0X04,
    ECALLBACK_TYPE_UPDATE_FIRMWARE    = 0X06, // Update old firmware package answer
    ECALLBACK_TYPE_ERASE_FIRMWARE     = 0X07, // Update new firmware package answer
    ECALLBACK_TYPE_FPD_STATUS         = 0X09, // Status package
    ECALLBACK_TYPE_ROM_UPLOAD         = 0X10,
    ECALLBACK_TYPE_RAM_UPLOAD         = 0X11,
    ECALLBACK_TYPE_FACTORY_UPLOAD     = 0X12,
    ECALLBACK_TYPE_SINGLE_IMAGE       = 0X51,
    ECALLBACK_TYPE_MULTIPLE_IMAGE     = 0X52,
    ECALLBACK_TYPE_PREVIEW_IMAGE      = 0X53,
    ECALLBACK_TYPE_PACKET_MISS        = 0X5B,
    ECALLBACK_TYPE_OFFSET_TMP         = 0X5C,
    ECALLBACK_OVERLAY_NUMBER          = 0X5D,
    ECALLBACK_OVERLAY_16BIT_IMAGE     = 0X5E,
    ECALLBACK_OVERLAY_32BIT_IMAGE     = 0X5F,
    ECALLBACK_TYPE_LIVE_ACQ_OK        = 0XA0,
    ECALLBACK_TYPE_PACKET_MISS_MSG    = 0XA4,
    ECALLBACK_TYPE_THREAD_EVENT       = 0XA5,
}

```

```

ECALLBACK_TYPE_ACQ_DISCARDED = 0XA6,
ECALLBACK_TYPE_OFFSET_ERR_MSG = 0XA7,
ECALLBACK_TYPE_GAIN_ERR_MSG = 0XA8,
ECALLBACK_TYPE_DEFECT_ERR_MSG = 0XA9,
ECALLBACK_TYPE_FPD_STATUS = 0XAA,
ECALLBACK_TYPE_SET_CFG_OK = 0XAB, //Setting parameters successful
ECALLBACK_TYPE_SAVE_SUCCESS = 0XAC, //Image saved successfully
ECALLBACK_TYPE_GENERATE_TEMPLATE = 0XAD, //Generate template
ECALLBACK_TYPE_FILE_NOTEXIST = 0XAE, //The file does not exist
ECALLBACK_TYPE_WORK_STATUS = 0XAF, //Working status (production mode or test mode)

```

```
};
```

Refer to <HbiType.h>.

2.8 Data Structure

2.8.1 Connection parameters

Common connection function interfaces.

Typedef struct CommCfgSt

```

{
    CommCfgSt()
    {
        _Type = FPD_COMM_TYPE: PCIE_COMM_TYPE;
        _Remoteip [0] = '\0';
        _Localip [0] = '\0';
        _LoacalPort = 32896;
        _RemotePort = 32897;
        _pcieId = 7024; // default id
    }

```

FPD_COMM_TYPE_Type;

// The following parts should only be set with ethernet interface, and PCIe interface just needs to set the type and the device ID of PCIe.

```

    char _Remoteip [MAX_IP_LEN];
    char _Localip [MAX_IP_LEN];
    unsigned short _LoacalPort;
    unsigned short _RemotePort;
    unsigned short _pcieId; // PCIe device ID
} COMM_CFG;

```

2.8.2 Image Properties

Typedef struct ImageProperty

```

{
    ImageProperty() {
        Nwidth = IMAGE_WIDTH;
    }

```

```

Nheight = IMAGE_HEIGHT;
Datatype = 0;
Ndatabit = 0;
Nendian = 0;
}
int nwidth;    // Image width.
int nheight;   // Image height.
int datatype;  // Data type: 0-unsigned char, 1-char, 2-unsigned short, 3-float, 4-double.
int ndatabit;  // Data bit: 0-16bit, 1-14bit, 2-12bit, 3-8bit.
int nendian;   // Endian: 0-little endian, 1-big endian.
}IMAGE_PROPERTY;

```

2.8.3 Trigger Mode Classification

Commonly used in setting trigger mode interfaces.

Typedef enum

```

{
    INVALID_TRIGGER_MODE          = 0x00, // Reserved.
    STATIC_SOFTWARE_TRIGGER_MODE  = 0x01, // Static: Software trigger mode.
    STATIC_PREP_TRIGGER_MODE      = 0x02, // Static: Reserved.
    STATIC_HVG_TRIGGER_MODE       = 0x03, // Static: High voltage generator trigger mode.
    STATIC_AED_TRIGGER_MODE       = 0x04, // Static: AED trigger mode.
    DYNAMIC_HVG_TRIGGER_MODE      = 0x05, // Dynamic: High voltage generator pulse sync in mode.
    DYNAMIC_FPD_TRIGGER_MODE      = 0x06, // Dynamic: Flat panel detector pulse sync out mode.
    DYNAMIC_FPD_CONTINUE_MODE     = 0x07, // Dynamic: Free run mode (Software trigger mode).
    DYNAMIC_FPD_SAEC_MODE         = 0x08, // Static:SAEC Mode
}EnumTRIGGER_MODE;

```

2.8.4 Continuous collection and classification

//Notice: Live Acquisition 'properties

Typedef enum

```

{
    PREOFFSET_IMAGE      = 0x01, // Preoffset template and image.
    ONLY_IMAGE           = 0x02, // Only image.
    ONLY_PREOFFSET       = 0x03, // Only Preoffset template.
    OVERLAY_16BIT_IMG    = 0x04, // Overlay result is 16bit image.
    OVERLAY_32BIT_IMG    = 0x05, // Overlay result is 32bit image.
}EnumLIVE_ACQUISITION;

```

2.8.5 Collection Mode Types

//Notice: acq mode: static and dynamic

Typedef enum

```

{
    SINGLE_ACQ_DEFAULT_TYPE = 0x00, // Single acquisition.
    LIVE_ACQ_DEFAULT_TYPE,      // Live acquisition.
}

```

```
// Multi step generation method
LIVE_ACQ_OFFSET_IMAGE, // Generate offset template with dynamic live acquisition.
SINGLE_ACQ_GAIN_IMAGE, // Generate gain template with static single acquisition.
LIVE_ACQ_GAIN_IMAGE, // Generate gain template with dynamic live acquisition.
SINGLE_ACQ_DEFECT_IMAGE, // Generate defect template with static single acquisition.
LIVE_ACQ_DEFECT_IMAGE, // Generate defect template with dynamic live acquisition.
// One-key quick generation mode
OFFSET_TEMPLATE_TYPE, // Quick generation process of offset template.
GAIN_TEMPLATE_TYPE, // Quick generation process of gain template- group1
DEFECT_TEMPLATE_GROUP1, // Quick generation process of defect template - group1.
DEFECT_TEMPLATE_GROUP2, // Quick generation process of defect template - group2.
DEFECT_TEMPLATE_GROUP3, // Quick generation process of defect template - group3.
SOFTWARE_OFFSET_TEMPLATE, // Quickly generate template collection type, continuously collect a set of dark field images
- SDK generates offset template
.....
}EnumIMAGE_ACQ_CMD;
```

2.8.6 Collecting Image Parameters

Including single frame acquisition and continuous acquisition, commonly found in single frame or continuous acquisition interfaces.

//Notice: fpd aqc mode

```
typedef struct fpd_aqc_mode_st {
    fpd_aqc_mode_st() {
        eAqctype = SINGLE_ACQ_DEFAULT_TYPE; // Default single image acquisition.
        eLivetype = ONLY_IMAGE; // Only image.
        ngrouppo = 0;
        nAcqnumber = 0;
        ndiscard = 0;
        nframeid = 0;
    };
    Enum IMAGE_ACQ_CMD eAqccmd; // Acquisition command.
    Enum LIVE_ACQUISITION eLivetype; // Live acquisition only,
    // detail refer to LIVE_ACQ_DEFAULT_TYPE.
    // 1-generate the firmware Preoffset template and
// then continue to acquire image.
    // 2 - image acquisition only.
    // 3 - generate firmware Preoffset template only.
    int ngrouppo; // Group number.
    int nAcqnumber; // Reserved image number.
    int ndiscard; // Discard image number.
    int nframeid; // Counter.
}FPD_AQC_MODE;
```

2.8.7 Correction enable setting parameters

Commonly used in setting correction enable state functions.



//Notice: fpd software calibrate enable info

```
typedef struct image_correct_enable_st {
    image_correct_enable_st(){
        bFeedbackCfg      = false;
        ucOffsetCorrection  = 0;
        ucGainCorrection    = 0;
        ucDefectCorrection  = 0;
        ucDummyCorrection   = 0;
    };
    bool bFeedbackCfg;    // true - ECALLBACK_TYPE_ROM_UPLOAD Event.
                        // false - ECALLBACK_TYPE_SET_CFG_OK Event.
    unsigned char ucOffsetCorrection;    // 00 - Reserved.
                                        // 01 - Software Preoffset Correction.
                                        // 02 - Firmware Postoffset Correction.
                                        // 03 - Firmware Preoffset Correction.
    unsigned char ucGainCorrection;    // 00 - Reserved.
                                        // 01 - Software Gain Correction.
                                        // 02 - Hardware Gain Correction.
    unsigned char ucDefectCorrection;    // 00 - Reserved.
                                        // 01 - Software Defect Correction.
                                        // 02 - Software Defect Correction.
    unsigned char ucDummyCorrection;    // 00 - Reserved.
                                        // 01 - Software Dummy Correction.
                                        // 02 - Software Dummy Correction.
}IMAGE_CORRECT_ENABLE;
```

2.8.8 Download template file parameters

Commonly seen in template download interface functions.

// Template file type

```
typedef enum
{
    OFFSET_TMP    = 0x00,
    GAIN_TMP      = 0x01,
    DEFECT_TMP    = 0x02
}emUPLOAD_FILE_TYPE;
```

// Template file attributes

```
typedef struct hbi_file_attr_st
{
    hbi_file_attr_st()
    {
        nModeID      = 0;
        emfiletype    = GAIN_TMP;
        nBinningtype  = -1;
        filepath[0]   = '\0';
    }
};
```

```
};  
int nModeID;  
emUPLOAD_FILE_TYPE emfiletype;  
int nBinningtype;  
char filepath[MAX_PATH];  
}DOWNLOAD_FILE;
```

Note: Compared to previous versions, a new parameter nModeID has been added (only effective in multi-mode mode) to indicate the location of firmware template storage.

In order to better compatibility with multi mode download templates, some interfaces and data types have been added as follows:

Template type:

//Notice: template file type

```
typedef enum  
{  
    OFFSET_TEMPLATE=0x00,  
    GAIN_TEMPLATE    =0x01,  
    DEFECT_TEMPLATE=0x02  
}EmpTEMPLATE_TYPE;
```

Download template dependency parameters:

//Template file attributes

```
typedef struct download_mode_file_st  
{  
    download_mode_file_st()  
    {  
        m_byModeId    = 0;  
        m_byBinningType = 1;  
        m_byPgaLevel   = 6;  
        m_unFpsTime    = 0;  
        m_usKvValue    = 0;  
        m_sZoomRectLeft = 0;  
        m_sZoomRectTop  = 0;  
        m_sZoomRectWidth = 0;  
        m_sZoomRectHeight = 0;  
        emtemplatetype = GAIN_TEMPLATE;  
    };  
    unsigned char m_byModeId;    // Mode ID  
    unsigned char m_byBinningType;// Binning type  
    unsigned char m_byPgaLevel;  
    unsigned int  m_unFpsTime;  
    unsigned short m_usKvValue;  
    unsigned short m_sZoomRectLeft;  
    unsigned short m_sZoomRectTop;  
    unsigned short m_sZoomRectWidth;
```



DT Imaging Technology (Shanghai) Co., Ltd

```
unsigned short    m_sZoomRectHeight;
emTEMPLATE_TYPE    emtemplatetype;
}DOWNLOAD_MODE_FILE;
```

2.8.9 Mode parameter

Each Mode item contains parameters including binning type, frame rate time, PGA level, high pressure dose, ZOOM area parameters, trigger mode, prepare delay, and template status. Among them, binning type, frame rate time, PGA level, high pressure dose, ZOOM area parameters are closely related to template generation and template download.

//Notice: fpd mode data information: 22

```
typedef struct fpd_mode_data_st
{
    // Register_1
    unsigned char    m_byBinningType;//01//Binning Type,1-binning1x1,2-binning2x2,3-binning3x3,binning4-4x4,
    // Register_2~5
    unsigned int      m_unFpsTime;    //04//Fps Time,unit: ms
    // Register_6
    unsigned char    m_byPgaLevel;    //01 //Pga Level
    // Register_7~Register_8
    unsigned short    m_usKvValue;    //02 //Dose Kv Value
    // Register_9~Register_10
    unsigned short    m_sZoomRectLeft; //02 //Zoom Rect Left
    // Register_11~Register_12
    unsigned short    m_sZoomRectTop; //02 //Zoom Rect Top
    // Register_13~Register_14
    unsigned short    m_sZoomRectWidth; //02    //Zoom Rect Width
    // Register_15~Register_16
    unsigned short    m_sZoomRectHeight;//02    //Zoom Rect Height
    // Register_17
    unsigned char      m_byFpdTrigger; //01    //Fpd Trigger Mode
    // Register_18~21
    unsigned int      m_unPrepareTime;    //04//Prepare Time,unit: ms
    // Register_22
    unsigned char    m_byTempStatus;    //01//0-Both dark and bright field templates are invalid, 1-Only dark field templates are
    valid
    // 2-Only the bright field template is valid, 3-Both dark and bright field templates are valid, others-Both dark and bright field
    templates are invalid
}FPD_MODE_DATA;
```

2.8.10 Multiple Mode List

The Multi Mode list contains the current Mode ID and an array of Mode items, as follows:

//User Acquisition Mode Parameter Configuration: 221

```
typedef struct fpd_mode_list_st
{
    // Register_1
```



DT Imaging Technology (Shanghai) Co., Ltd

```
unsigned char m_byCurModeId;           //01 //Current Mode Id,[0~10]
// Register_2~Register_221
FPD_MODE_DATA m_stModeArray[MODE_LIST_TOTAL]; //220 //Current Mode DATA Array
}FPD_MODE_LIST;
```

2.8.11 Other Parameters

Please refer to header file 'HbiType.h'.

2.9 Flat Panel Status

According to the tablet communication method, it can be divided into network port static, network port dynamic, optical port dynamic, and wireless static, as shown in the callback function event `ECALLBACK_TYPE_FPD_STATUS`.

Callback function: `handleCommandEventA (void * pvContext, int nDevId, unsigned char byteEvnetid, void * pvParam1, int nParam2, int nParam3, int nParam4)`

2.9.1 Static flat panel detector status of Ethernet port

	State type	Remarks
static flat panel detector	0: FPD_DISCONN_STATUS	In the initial state, if there is an abnormal disconnection message, the following situations are considered abnormal and the connection is disconnected. NParam2=0: Network not connected NParam2=-1: parameter exception NParam2=-2: Failed to return the number of prepared descriptors NParam2=-3: Receive timeout NParam2=-4: Receive failed NParam2=-5: Port unreadable NParam2=-6: Abnormal monitoring network card NParam2=-7: Monitoring network card is normal (tested, actual feedback is not provided) NParam2=-8: Firmware update completed, disconnected NParam2=-9: Optical port communication, fiber optic disconnection NParam2=-10: Optical port communication, failed to read DDR address, restarting the PCIe driver can solve the problem. NParam2=-11: The tablet is a giant frame, which means non giant frame communication is used for communication, and the receiving area overflows.
	100: FPD_CONN_STATUS	Connection successful
	1: FPD_PREPARE_STATUS	Prepare status, clear
	2: FPD_READY_STATUS	Ready state, tablet in idle waiting state
	4: FPD_EXPOSE_STATUS	Busy status, currently collecting images
	9: FPD_UPDATE_FIRMARE	Updating firmware
	10: FPD_RETRANS_MISS	Feedback on packet loss information, including the total number of packets
	11: FPD_STATUS_AED	AED status, in the AED acquisition chart

2.9.2 Dynamic flat panel detector status of network port

	State type	Remarks
dynamic flat panel detector	0: FPD_DISCONN_STATUS	In the initial state, if there is an abnormal disconnection message, the following situations are considered abnormal and the connection is disconnected. NParam2=0: Network not connected NParam2=-1: parameter exception NParam2=-2: Failed to return the number of prepared descriptors

		NParam2=-3: Receive timeout NParam2=-4: Receive failed NParam2=-5: Port unreadable NParam2=-6: Abnormal monitoring network card NParam2=-7: Monitoring network card is normal (tested, actual feedback is not provided) NParam2=-8: Firmware update completed, disconnected NParam2=-9: Optical port communication, fiber optic disconnection NParam2=-10: Optical port communication, failed to read DDR address, restarting the PCIe driver can solve the problem. NParam2=-11: The tablet is a giant frame, which means non giant frame communication is used for communication, and the receiving area overflows.
	100: FPD STATUS	Connection successful
	1: FPD Prepare STATUS	Prepare status, clear
	2: FPD_READY_STATUS	Single frame acquisition completed, ready, tablet in idle waiting state
	3:FPD_DOOFFSET_TEMPLA TE	In the firmware generation pre offset template
	4: FPD_EXPOSE STATUS	Busy status, currently collecting images
	5: FPD_CONTINUTE_READY	Continuous collection completed, ready, tablet in idle waiting state
	6: FPD_DWONLOAD_GAIN	Download gain template
	7:FPD_DWONLOAD_DEFEC T	Download defect template
	8:FPD_DWONLOAD_OFFSE T	Download offset template
	9: FPD_UPDATE_FIRMARE	Updating firmware
	10: FPD_RETRANS_MISS	Feedback on packet loss information, including the total number of packets
	11: FPD_STATUS_AED	AED status, in the AED acquisition chart

2.9.3 Dynamic flat panel status of optical port

	State type	Remarks
Optical dynamic flat detector	0: FPD_DISCONN_STATUS	In the initial state, if there is an abnormal disconnection message, the following situations are considered abnormal and the connection is disconnected. NParam2=0: Network not connected NParam2=-1: parameter exception NParam2=-2: Failed to return the number of prepared descriptors NParam2=-3: Receive timeout NParam2=-4: Receive failed NParam2=-5: Port unreadable NParam2=-6: Abnormal monitoring network card NParam2=-7: Monitoring network card is normal (tested, actual feedback is not provided) NParam2=-8: Firmware update completed, disconnected NParam2=-9: Optical port communication, fiber optic disconnection NParam2=-10: Optical port communication, failed to read DDR address, restarting the PCIe driver can solve the problem. NParam2=-11: The tablet is a giant frame, which means non giant frame communication is used for communication, and the receiving area overflows.
	100: FPD_CONN STATUS	Connection successful
	1: FPD_PREPARE STATUS	Prepare status, clear
	2: FPD_READY_STATUS	Single frame acquisition completed, ready, tablet in idle waiting state
	3:FPD_DOOFFSET_TEMPLA TE	In the firmware generation pre offset template

	4: FPD_EXPOSE_STATUS	Busy status, currently collecting images
	5: FPD_CONTINUTE_READY	Continuous collection completed, ready, tablet in idle waiting state
	6: FPD_DWONLOAD_GAIN	Download gain template
	7: FPD_DWONLOAD_DEFECT	Download defect template
	8: FPD_DWONLOAD_OFFSET	Download offset template
	9: FPD_UPDATE_FIRMWARE	Updating firmware
	10: FPD_RETRANS_MISS	Feedback on packet loss information, including the total number of packets
	11: FPD_STATUS_AED	AED status, in the AED acquisition chart

2.9.4 Wireless static tablet status

Wireless static flat detector	State type	Remarks
	0: FPD_DISCONN_STATUS	In the initial state, if there is an abnormal disconnection message, the following situations are considered abnormal and the connection is disconnected. NParam2=0: Network not connected NParam2=-1: parameter exception NParam2=-2: Failed to return the number of prepared descriptors NParam2=-3: Receive timeout NParam2=-4: Receive failed NParam2=-5: Port unreadable NParam2=-6: Abnormal monitoring network card NParam2=-7: Monitoring network card is normal (tested, actual feedback is not provided) NParam2=-8: Firmware update completed, disconnected NParam2=-9: Optical port communication, fiber optic disconnection NParam2=-10: Optical port communication, failed to read DDR address, restarting the PCIe driver can solve the problem. NParam2=-11: The tablet is a giant frame, which means non giant frame communication is used for communication, and the receiving area overflows.
	100: FPD_CONN_STATUS	Connection successful
	1: FPD_PREPARE_STATUS	Prepare status, clear
	2: FPD_READY_STATUS	Ready state, tablet in idle waiting state
	4: FPD_EXPOSE_STATUS	Busy status, currently collecting images
	9: FPD_UPDATE_FIRMWARE	Updating firmware
	10: FPD_RETRANS_MISS	Feedback on packet loss information, including the total number of packets
	11: FPD_STATUS_AED	AED status, in the AED acquisition chart
	12: FPD_STATUS_SLEEP	Wireless: tablet sleep state
	13: FPD_STATUS_WAKEUP	Wireless: tablet wake-up status
	14: FPD_DOWNLOAD_NO_IMAGE	Download EMMC storage image: no image
	15: FPD_DOWNLOAD_TAIL_IMAGE	Download the last frame of the EMMC image
	16: FPD_EMMC_MAX_NUMBER	Emmc maximum storage frame rate
	18: FPD_WLAN_BATTERY_STATUS	Wireless battery feedback information

3. API description

3.1 Function Types

The basic data types used in the API are as follows:

void	No parameters	nothing
void*	unsigned pointer	4 bytes
int	signed integer	4 bytes
short	signed short integer	2 bytes
WORD	unsigned short	2 bytes
char	character type	1 byte
BOOL	signed integer (int)	4 bytes
float	single precision floating-point type	4 bytes
unsigned int	unsigned int	4 bytes
unsigned short	unsigned short	2 bytes
int *	Integer pointer	4 bytes
char*	character pointer	4 bytes
unsigned char	unsigned character type	1 byte
CodeStringTable	Custom Type: Error Information Table	12 bytes
RegCfgInfo	Custom Type: Firmware Parameter Object	1024 bytes
ImgCaliCfg	Custom type: tablet image calibration parameters	20 bytes
SysCfgInfo	Custom Type: Tablet System Parameters	128 bytes
USER_CALLBACK_HANDLE_PROCESS	Custom Type: Callback Function Pointer Load State	4 bytes
USER_CALLBACK_HANDLE_ENVENT	Custom type: callback function pointer	4 bytes
SOFTWARE_CALIBRATE_ENABLE	Custom Type: Software Calibration Enable	4 bytes
IMAGE_PROPERTY	Custom Type: Image Properties	32 bytes
IMAGE_CORRECT_ENABLE	Custom Type: Image Calibration Enable Parameters	5 bytes
FPD_AQC_MODE	Custom Type: Collection Mode Parameters	36 bytes
DOWNLOAD_T_FILE	Custom Type: Download Template Parameters	264 bytes
CALIBRATE_INPUT_PARAM	Custom Type: Calibration Generate Model Parameters	36 bytes

3.2 Function classification

Mainly aimed at fast integration users, this article focuses on introducing several functional interfaces and categorizing them according to their needs (detailed in section 3.3 Functions), as follows:

3.2.1 Initialize and Release Resources

After initialization, the current instance handle can be obtained, which will allocate some resources to the interface. The resource release command will destroy the allocated resources. If not destroyed, it will cause memory leakage and some exceptions.

Single panel interface

No001	HBI_Init	Initialize dynamic library
No002	HBI_Destroy	Release dynamic library resources

"Dual panel" interface

No054	HBI_InitDual	Dual panels: initializing dynamic library instances (new)
No055	HBI_InitEx	Dual panels: Initialize dual board or multi board dynamic library instances (new)
No056	HBI_DestroyEx	Dual panels: All DLL resources (new)

3.2.2 Registering Callback Functions

At present, the SDK interface includes two callback functions: the main **callback function and the download progress, result, and status callback function**. The **main callback function** must be registered, and after initialization, it must be bound to the handle for registration. The connection function, parameter setting, and graph collection command must be used after registering the callback function, otherwise it cannot succeed;

No009	HBI_RegEventCallBackFun	Registering callback functions - image, configuration, status
No050	HBI_RegProgressCallBack	Register callback function, download template or firmware, provide progress and results feedback

3.2.3 Connecting and disconnecting panel

No003	HBI_Connect Detector	Establish a connection (supports Ethernet and optical ports)
No004	HBI_Connect DetectorUdp	New: Ethernet UDP connection
No005	HBI_Connect DetectorJumbo	New: Ethernet UDP Giant Frame Connection
No006	HBI_Connect DetectorPcie	New: Optical port connection (PCIe card)
No007	HBI_Connect DetectorEx	New: Connection interface, do not include parameters, require configuration file. See Appendix 2 "hbi_fpd_init.ini"
No008	HBI_DisConnectDetector	Disconnect

3.2.4 Trigger mode and calibration enable settings

Triggering has been explained in detail in the above chapter. Here, only the interfaces are listed. Please refer to the interface function description below for instructions on how to use them. There are some combination function interfaces below that can set multiple parameters for users to choose from.

No026	HBI_TriggerAndCorrectDisplay	Quick initialization parameters (trigger mode and image correction enable)
No027	HBI_UpdateTriggerMode	Set trigger mode
No028	HBI_UpdateCorrectEnable	Set firmware image calibration enable
No029	HBI_GetCorrectEnable	Obtain firmware image correction enable
No030	HBI_SetSoftwareCorrect	
No031	HBI_GetSoftwareCorrect	Obtain software correction enable
No067	HBI_TriggerPgaBinningCorrect	Set trigger mode, PGA gear, binning mode, and correction enable
No068	HBI_TriggerPgaBinningAcqTimeCorrect	Set trigger mode, PGA gear, binning mode, frame rate (image acquisition time interval), correction enable

3.2.5 Single frame acquisition and continuous acquisition

Single frame acquisition and continuous acquisition have been explained in detail in the above chapters. Here, only the interfaces are listed. Please refer to the interface function instructions below for how to use them. These interfaces are "soft trigger" commands.

Single frame acquisition command (default): SingLE_ACQ_DEFAULT_TYPE

Continuous acquisition command (default): LIVE_ACQ_DEFAULT_TYPE.

No018	HBI_SinglePrepare	Single frame acquisition preparation instruction
No019	HBI_SingleAcquisition	Single frame acquisition
No020	HBI_LiveAcquisition	Continuous collection
No021	HBI_StopAcquisition	Stop continuous collection
No022	HBI_SetSinglePrepareTime	Set the time interval between soft trigger single frame acquisition clearing and acquisition
No023	HBI_GetSinglePrepareTime	Obtain the time interval between soft trigger single frame acquisition clearing and acquisition
No024	HBI_SetLiveAcquisitionTime	Set continuous collection time interval (static or wireless tablet)
No025	HBI_GetLiveAcquisitionTime	Obtain continuous acquisition time interval (static or wireless

No065	HBI_GetSelfDumpingTime	tablet) Set the acquisition time interval (large frame or optical dynamic tablet)
No066	HBI_SetSelfDumpingTime	Obtain acquisition time interval (large frame or optical dynamic tablet)
No085	HBI_Prepare (new)	Single frame acquisition preparation instruction, for HBI_SinglePrepare supplementation and optimization

3.2.6 Set frame rate, PGA gear, and Binning type switching

Frame rate: Currently, it is achieved by setting continuous acquisition time intervals, and static wireless tablets use HBI_SetLiveAcquisitionTime, dynamic, optical tablet using HBI_SetSelfDumpingTime completed, parameter units in milliseconds.

PGA gear: Increase sensitivity, divided into 1-7 gears, COMS tablet is 8-9 gears, the lower the gear, the higher the sensitivity, and vice versa.

Binning type: supports binning1x1, binning2x2, binning3x3, and binning4x4.

No024	HBI_SetLiveAcquisitionTime	Set continuous collection time interval (static or wireless tablet)
No066	HBI_SetSelfDumpingTime	Obtain acquisition time interval (large frame or optical dynamic tablet)
No042	HBI_SetPGAlevel	Set gain mode
No043	HBI_GetPGAlevel	Obtain gain mode
D044	HBI_SetBinning	Set binding method
No045	HBI_GetBinning	Obtain binding method
No067	HBI_TriggerPgaBinningCorrect	Set trigger mode, PGA gear, binning mode, and correction enable
No068	HBI_TriggerPgaBinningAcqTimeCorrect	Set trigger mode, PGA gear, binning mode, frame rate (image acquisition time interval), correction enable

3.2.7 Single frame acquisition mode classification

Single frame acquisition can be divided into soft trigger, high voltage trigger, AED trigger, etc. according to the triggering method.

The soft trigger mode is divided into Prepare+delay and Prepare+Single Shot modes.

Prepare+Delay: If the Prepare delay time is greater than 0, call HBI_Collect one frame of image after the SinglePrepare interface;

Prepare+Single Shot: When the prepare delay time is equal to 0, call HBI_Wait for a period of time after SinglePrepare (can wait a few hundred milliseconds) to call HBI_SingleAcquisition captures one frame of image.

Prepare delay refers to the time interval between soft trigger single frame acquisition clearing and acquisition. When applying single frame acquisition, it can be achieved through HBI_SetSinglePrepareTime update, which users can choose to use in different scenarios.

HBI_The command for SingleAcquisition acquisition mode parameters is as follows:

Single frame normal acquisition command:

Single_ACQ_DEFAULT_TYPE=0x00, //default single frame acquisition

Distribution Generation Template - Single Frame Acquisition Command

Single_ACQ_GAIN_IMAGE, //Create Gain template - Single frame capture of bright field image

Single_ACQ_DEFECT_IMAGE, //Create Defect template - Single frame capture of bright field image

3.2.8 Classification of continuous acquisition modes

The commonly used interface function is HBI_LiveAcquisition is used for normal image acquisition, firmware pre offset template generation, overlay image acquisition, and template data image acquisition (distributed generation template).



DT Imaging Technology (Shanghai) Co., Ltd

Continuous acquisition command (default): LIVE_ACQ_DEFAULT_TYPE

Typedef struct FPD_AQC_MODE_St

```
{
    EnumIMAGE_ACQ_CMD eAqcCmd// Continuous acquisition command (default): LIVE_ACQ_DEFAULT_TYPE
    EnumLIVE_ACQUISITION eLivetype// Two images above; 3- Firmware only creates offset templates; 4- Overlay: Upload a 16
    bit average image; 5- Overlay: Upload 32-bit average image;
    int ngroupro// Group number
    int nAcqnumber// Actual collection frame rate
    int ndiscard// Ignored frames
    int nframeid// counter
}FPD_AQC_MODE;
```

Here are several common applications:

1》 Firmware generation of Pre offset templates and overlay images

FPD_AQC_MODE.eLivetype=PREOFFSET_IMAGE	Generate Pre offset template for firmware and continue continuous collection (trigger mode is 7-Dynamic: Continue Mode)
FPD_AQC_MODE.eLivetype=ONLY_IMAGE	Normal continuous collection and sampling of images
FPD_AQC_MODE.eLivetype=ONLY_PREOFFSET	Generate Pre offset template only for firmware (trigger mode is 7-Dynamic: Continue Mode)
FPD_AQC_MODE.eLivetype=Overlay_16BIT_IMG	Overlay 16 bit image, frame rate needs to be set, frame rate greater than 0
FPD_AQC_MODE.eLivetype=Overlay_32BIT_IMG	Overlay 32-bit images, frame rate needs to be set, frame rate greater than 0

2》 Specify the frame rate for collection and discard frames

Generally, by setting the continuous collection control attribute, the frame rate of collection can be controlled. When the frame rate is 0, the collection command is triggered until the stop continuous collection command is sent to stop collection. However, if the frame rate is less than 0, the specified frame rate can be collected. Of course, in some cases, if the data quality of the first few frames is not good, it can be discarded through parameter control.

FPD_AQC_MODE.nAcqnumber// Capture image frame rate

FPD_AQC_MODE.ndiscard// The number of frames discarded during image collection

3》 Distributed Generation Template - Continuous Collection Command

// 分布生成矫正模板，用于验证模板

LIVE_ACQ_OFFSET_IMAGE//Create Offset Template - Continuously Collect Dark Field

LIVE_ACQ_GAIN_IMAGE//Create Gain template - continuously collect bright field images

LIVE_ACQ_DEFECT_IMAGE//Create Defect template - continuously collect bright field images

The collection mode is as above, and template data files can be collected separately and saved to the default path.

4》 The following command cannot be used as HBI_Parameters of LiveAcquisition

// 快速生成矫正模板，用于系统集成

OFFSET_TEMPLATE_TYPE//Quickly generate template collection types, continuously collect a set of dark field images and generate offset templates, firmware generation templates

GAIN_TEMPLATE_TYPE//Quickly generate template collection types, continuously collect a set of bright field images and generate gain templates

DEFECT_TEMPLATE_GROUP1//Quickly generate template collection types, continuously collect a set of bright field images -

defect group1

DEFECT_TEMPLATE_GROUP2, //Quickly generate template collection types, continuously collect a set of bright field images -

defect group2

DEFECT_TEMPLATE_GROUP3, //Quickly generate template collection types, continuously collect a set of bright field images -

defect group3 and generate template

Software_OFFSET_TEMPLATE/Quick template generation collection type, continuously collecting a set of dark field images - SDK generates offset template

Note: These command types can usually only be used as interface functions 'HBI_GenerateTemplate' parameter, cannot be used as the interface function 'HBI_LiveAcquisition' parameter. HBI_GenerateTemplate is used to quickly generate templates, and the interface includes the process of sending drawing commands and generating templates.

3.2.9 Quick template generation and template download

Generate templates and template downloads. Here, only the interfaces are listed, and there is a dedicated chapter below that explains how to use them.

No050	HBI_RegProgressCallBack	Register callback function, download template or firmware, provide progress and results feedback
No051	HBI_GenerateTemplate	Quickly generate templates, including offset, gain, and defect
No052	HBI_DownloadTemplate	To download the correction template, you need to pass in the file type, directory, and other parameters
No053	HBI_StopDownloadTemplate	Stop downloading correction templates
No086	HBI_DownloadTemplateByType	Download fixed correction template files by default according to type (parameter automatic matching), it is recommended to replace HBI_DownloadTemplate.

3.2.10 Obtain image attributes, software and firmware versions

No012	HBI_GetFPDSerialNumber	Obtain detector serial number
No013	HBI_GetSDKVersion	Get SDK DLL version number
No014	HBI_GetFirmwareVersion	Obtain firmware version number
No015	HBI_GetError	Obtain error information through error codes
No016	HBI_GetFpdCfgInfo	Obtain firmware ROM parameters
No017	HBI_GetImageProperty	Get Image Properties

3.2.11 Obtain tablet parameters and restore factory settings

No016	HBI_GetFpdCfgInfo	Obtain firmware ROM parameters
No011	HBI_ResetFirmware	Reset firmware factory settings

3.2.12 Zoom (ROI) Area Settings

No070	HBI_SetRoyMode	The ROI region setting function for amorphous silicon currently only supports flat plate types including 1613-125um, 3030-140um, and 4343-100um.
No071	HBI_SetZoomModeRect	The CMOS ZOOM Mode region setting function currently only supports tablet types including CMOS-1412-50um and CMOS-0606-50um.
No080	HBI_GetCurZoomRect	Retrieve the current ZOOM region size data, where the left, top, width, and height in Rect are 0, indicating the entire image.

3.2.13 Calibration template path

No072	HBI_SetCorrectFileDirectory	The user sets the main directory for calibration files, usually using the default path, and individual users can set a custom directory
No073	HBI_GetCorrectFileDirectory	Obtain the user calibration file home directory, usually using the default path, and individual users can set a custom

No081	HBI_IsExistsOfCurCorrectTemplate	directory Determine whether the current template file exists based on the template type (new)
No082	HBI_GetCurrentCorrectFileName	Get the current template file name (new)
	HBI_GetCurTempFileFullDirectory	Obtain the full directory of the current template file based on the template type (new)
No084	HBI_GetCurrentCorrectFilePath	Obtain the absolute path of the current template file based on the template type (new)

3.2.14 Image Display and Save

No074	HBI_UpdateSaveOption	Update Save Image Options
-------	----------------------	---------------------------

3.2.15 Set packet interval

No075	HBI_SetPacketIntervalTime	Set packet interval time (100Mbps network)
No076	HBI_AutoWindowWidthWindowLevel	The automatic window width and window level algorithm converts 16 bit data into 8-bit unsigned data through window width and window level processing, allowing users to directly display 8-bit data.

3.2.17 Image Rotation/Mirroring

No077	HBI_SetImageRotateAndFlip	Set rotation 90 ° or mirror enable parameters
No078	HBI_Rotate90AndFlip	Image rotation and mirroring processing (only applicable to 16 bit unsigned image data, default to first rotate and then mirror) (new)
No079	HBI_ResetCorrectTemplate	Reset calibration template file - rotate, mirror reverse processing and save, usually completed before downloading to firmware (new)

3.2.18 Firmware Template Management Method

Reason: Essentially, the firmware manages different templates differently. Early firmware used the Binning method (version) to store calibration templates, which was relatively simple to generate, store, and load. However, with the development of the business, it can no longer meet the needs of users well. Users need to calibrate templates under different conditions. If they continue to manage and load templates according to the Binning method, the calibrated images may experience misalignment. Therefore, the multi-mode approach is more in line with the business needs of these users.

No180	HBI_FirmwareManagementTemplateType	Firmware template management template methods: Binning storage and multi mode storage. This interface can switch between two methods.
-------	------------------------------------	---

How to manage these two methods to prevent incompatibility with previous versions of Binning? Manage through parameter classification. In addition, different interface functions are used in different ways to be compatible with both old and new versions. Users choose different ways to integrate based on their different advantages, disadvantages, needs, and difficulty levels.

The newly added interface function corresponds to the Binning version interface function (see table below). Additionally, in addition to setting parameters, a new function will automatically synchronize the parameters of the current Mode item. In short, based on the integration of multiple modes, try to use 'HBI_M*' as much as possible Set parameters for the interface function.

Number	Binning version interface function	Multiple Mode Version Interface Functions	Remarks
1	HBI_UpdateTriggerMode	HBI_MUpdateTrigger	Set trigger mode
2	HBI_SetBinning	HBI_MSetBinning	Set image binding method
3	HBI_SetPGALevel	HBI_MSetPGALevel	Set the gain gear, that is, set the programmable integral capacitor gear to improve sensitivity

4	HBI_SetRoyMode	HBI_MSetRoyMode	The ROI region setting function for amorphous silicon currently only supports flat plate types including 1613-125um, 3030-140um, 4343-100um, and 2530-100um.
5	HBI_SetZoomModeRect	HBI_MSetZoomModeRect	The CMOS ZOOM Mode region setting function currently only supports tablet types including CMOS-1412-50um, CMOS-0606-50um, and 1917-120um.
6	HBI_SetSelfDumpingTime	HBI_MSetSelfDumpingTime	Set collection time interval (dynamic tablet)
7	HBI_SetLiveAcquisitionTime	HBI_MSetLiveAcquisitionTime	Set collection time interval (frame rate can be set)
8	HBI_SetFpsOfTime	HBI_MSetFpsOfTime	Set frame rate control time
9	HBI_PgaAcqTime	HBI_MPgaAcqTime	Set PGA gear and frame rate (time interval for image acquisition)
10	HBI_PgaBinningAcqTime	HBI_MPgaBinningAcqTime	Set PGA gear, binning mode, and frame rate (image acquisition interval)
11	HBI_TriggerAndCorrectDisplay	HBI_MTriggerAndCorrectDisplay	Set trigger mode and enable image calibration
12	HBI_TriggerBinningAcqTime	HBI_MTriggerBinningAcqTime	Set trigger mode, binning method, and frame rate (time interval for image acquisition)
13	HBI_TriggerPgaBinningAcqTime	HBI_MTriggerPgaBinningAcqTime	Set trigger mode, PGA gear, binning mode, and frame rate (image acquisition time interval)
14	HBI_TriggerPgaBinningAcqTimeCorrect	HBI_MTriggerPgaBinningAcqTimeCorrect	Set trigger mode, PGA gear, binning mode, and frame rate (image acquisition time interval), Image correction enable.

3.2.19 Multiple Mode Other Interface

The following interfaces are only applicable to editing multiple Mode lists, switching Mode items, saving Mode list information, refining Mode list item data, updating templates for specified Mode items, and template status information for Mode items.

No102	HBI_SetValidModeItemNumber	Set the number of valid Mode items
No103	HBI_IsMutilMode	Check if the current firmware supports multiple modes
No.087	HBI_GetCurModeList	Get Mode List
No088	HBI_UpdateModeList	Update Mode List
No089	HBI_GetCurModeData	Get the current Mode Data
No090	HBI_UpdateCurModeData	Update current Mode Data
No091	HBI_UpdateModeData	Update Mode Item Data through Mode ID
No119	HBI_GetModeData	Obtain Mode Data through Mode ID
No120	HBI_GetModeListSize	Get the number of items in Mode
No092	HBI_GetCurModeID	Get the current Mode ID
No093	HBI_SwitchUserMode	Switch Mode option
No121	HBI_ModeExists	Determine whether the mode item exists
No122	HBI_CheckCurtMode	Determine if the mode is normal
No094	HBI_ModeCorrectTempalteEnable	The calibration template for the current mode is effective or not effective
No095	HBI_AllModeCorrectTempalteEnable	All calibration templates for Mode items are effective or ineffective
No096	HBI_SaveModeList2File	Save the current user mode list
No097	HBI_DownloadModeTemplate	Download template based on Mode parameter
No098	HBI_DownloadModeTemplateByType	Download correction template files according to template type (default to current Mode Id)
No099	HBI_DownloadModeTemplateByIdAndType	Download correction template files according to Mode Id and template type
No100	HBI_GetCurModeTemplateStatus	Obtain the firmware calibration template status for the

3.3 Interface Functions

Note:

```
#ifdef _DLL_EX_IM
#define HBI_FPD_DLL__ Declspec (dllexport)
#else
#define HBI_FPD_DLL__ Declspec (dllimport)
#endif
#define _DLL_EX_IM 0//0-import function, 1-export function, default to 0 (user default).
```

3.3.1 HBI_Init

```

/*****
* No: No001
* Function Name: HBI_Init
* Function Description: Initialize dynamic dll, nonvolatile parameters and thread message parameters, establish SDK object pointer.
* Parameter Description:
    int fpidid          // Flat panel detector ID, default is 0.
* Return Value: void *
    failed: NULL, Success: Non empty
* Comment:
*****/
HBI_FPD_DLL void * HBI_Init (int fpidid=0);

```

3.3.2 HBI_Destroy

```

/*****
* No: No002
* Function Name: HBI_Destroy
* Function Description: Release dynamic library resources
* Parameter Description:
    In: void * handle -unsigned handle pointer, dynamic library FPD SDK Object object pointer
    Out: None
* Return Value: void
* Comment:
*****/
HBI_FPD_DLL void HBI_Destroy (void * handle);

```

3.3.3 HBI_ConnectDetector

```

/*****
* No: No003
* Function Name: HBI_ConnectDetector
* Function Description: Establish connection
* Parameter Description:

```



DT Imaging Technology (Shanghai) Co., Ltd

In: void *handle // unsigned handle pointer

COMM_CFG commCfg // communication type, refer to <HbiType.h>

UDP_COMM_TYPE = 0, //Standard UDP communication method

UDP_JUMBO_COMM_TYPE=1, //Standard UDP communication method, Support for jumbo

PCIE_COMM_TYPE = 2, //Fiber type

WALN_COMM_TYPE = 3, //Wireless type

int doOffsetTemp // 1: don't update pre-offset template when connection is successful

// Non-1: update pre-offset template when connection is successful

Out: None

* Return Value: int

0- Successful

Non 0- Failed, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int HBI_ConnectDetector (void * handle, COMM_CFG commCfg, int doOffsetTemp=0);

Below are several new connection functions that users can choose based on their actual needs.

3.3.4 HBI_ConnectDetectorUdp

*****/

* No: No004

* Function Name: HBI_ConnectDetectorUdp

* Function Description: Standard UDP communication method, establish connection (gigabit Ethernet interface with typical UDP packet)

* Parameter Description:

In: void *handle // unsigned handle pointer

char *szDetectorIp // FPD IP address: 192.168.10.40

unsigned short usDetectorPort // FPD port: 32897(0x8081)

char *szLocalIp // PC IP address: 192.168.10.20

unsigned short usLocalPort // PC port: 32896(0x8080)

int doOffsetTemp // 1: don't update pre-offset template when connection is successful

// Non-1: update pre-offset template when connection is successful

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_ConnectDetectorUdp (void * handle, char * szDetectorIp, unsigned short usDetectorPort, char * szLocalIp, unsigned short usLocalPort, int doOffsetTemp=0);

3.3.5 HBI_ConnectDetectorJumbo

*****/

* No: No005

* Function Name: HBI_ConnectDetectorJumbo

* Function Description: Establish connection (gigabit Ethernet interface with typical UDP packet, Support for jumbo)



DT Imaging Technology (Shanghai) Co., Ltd

*** Parameter Description:**

In: void *handle // unsigned handle pointer
char *szDetectorIp // FPD IP address: 192.168.10.40
unsigned short usDetectorPort // FPD port: 32897(0x8081)
char *szLocalIp // PC IP address: 192.168.10.20
unsigned short usLocalPort // PC port: 32896(0x8080)
int doOffsetTemp // 1: don't update pre-offset template when connection is successful
// Non-1: update pre-offset template when connection is successful

Out: None

*** Return Value: int**

0- Successful
Non 0- failed

*** Comment:**

*****/

HBI_FPD_DLL int __stdcall HBI_ConnectDetectorJumbo (void * handle, char * szDetectorIp, unsigned short usDetectorPort, char * szLocalIp, unsigned short usLocalPort, int doOffsetTemp=0);

3.3.6 HBI_ConnectDetectorPcie

/*****/

* No: No006

* Function Name: HBI_ConnectDetectorPcie

* Function Description: // Establish connection (optical fiber interface)

*** Parameter Description:**

In: void *handle // unsigned handle pointer
int doOffsetTemp // 1: don't update pre-offset template when connection is successful
// Non-1: update pre-offset template when connection is successful

Out: None

*** Return Value: int**

0- Successful
Non 0- failed

*** Comment:**

*****/

HBI_FPD_DLL int __stdcall HBI_ConnectDetectorPcie (void * handle, int doOffsetTemperature=0);

3.3.7 HBI_Connect DetectorEx

/*****/

* No: No007

* Function Name: HBI_ConnectDetectorEx

* Function Description: Establish connection

*** Parameter Description:**

In: void * handle - // unsigned handle pointer

Basic communication control parameters can be found in [HBI_FPD_CONFIG.ini](#)

The file directory is the same as the DLL.

Out: None



DT Imaging Technology (Shanghai) Co., Ltd

```
* Return Value:  int
                0- Successful
                Non 0- failed

* Comment:hbi_fpd_init.ini

* FPD IP Address:      FPD_IP = 192.168.10.40
* FPD Port:           FPD_PORT = 32897
* Local IP Address:    LOCAL_IP = 192.168.10.20
* Local Port:         LOCAL_PORT = 32896
* TRIGGER_MODE:       0-Invalid Trigger;
                     1: Static: Software Trigger;
                     2: Clear;
                     3: Static: Hvg Trigger Mode;
                     4: Static: AED Trigger Mode;
                     5: Dynamic: Hvg Sync Mode;
                     6: Dynamic: Fpd Sync Mode;
                     7: Dynamic: Free Run Mode;
                     8: Static: SAEC Mode;

* OFFSET_ENABEL:      0: No Offset Correction;
                     1: Software Preoffset Correction;
                     2: Firmware Postoffset Correction;
                     3: Firmware Preoffset Correction;

* GAIN_ENABLE:        0: No Gain Correction;
                     1: Software Gain Correction;
                     2: Firmware Gain Correction;

* DEFECT_ENABLE:      0: No Defect Correction;
                     1: Software Defect Correction;
                     2: Firmware Defect Correction;

* DUMMY_ENABLE:       0: No Dummy Correction;
                     1: Software Dummy Correction;
                     2: Firmware Dummy Correction;

* LIVE_ACQ_TIME:      1000~1000ms
* SELF_DUMPING_TIME:  500~500ms
* PREPARE_TIME:       1500~1500ms
* DO_OFFSET:          0: No action
                     1: Do offset template
```

*****/

```
HBI_FPD_DLL int __stdcall HBI_ConnectDetectorEx (void * handle);
```

3.3.8 HBI_DisConnectDetector

/*****

```
* No: No08
* Function Name:  HBI_DisConnectDetector
* Function Description: Disconnect detector
* Parameter Description:
```



DT Imaging Technology (Shanghai) Co., Ltd

In: void * handle - // unsigned handle pointer

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int HBI_DisconnectDetector (void * handle);

3.3.9 HBI_RegEventCallBackFun

*****/

* No: No09

* Function Name: HBI_RegEventCallBackFun

* Function Description: Register callback function

* Parameter Description:

In: void * handle - unsigned handle pointer

USER_Callback_HANDLE_ENVENT - Register callback function

Void * pContext - Object pointer, application software transfer object pointer to SDK in order to access static and global variable

Out: None

* Return Value: int

0- Successful

Non 0- failed, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int HBI_RegEventCallBackFun (void * handle,
USER_CALLBACK_HANDLE_ENVENT callback, void * pContext);

3.3.10 HBI_ResetDetector

*****/

* No: No10

* Function Name: HBI_ResetDetector

* Function Description: Restart the detector

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: None

* Return Value: int

0- Successful

Non 0- failed, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int HBI_ResetDetector (void * handle);

3.3.11 HBI_ResetFirmware



DT Imaging Technology (Shanghai) Co., Ltd

* No: No11

* Function Name: HBI_ResetFirmware

* Function Description: Restore factory setting

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: None

* Return Value: int

0- Successful

Non 0- failed, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_ResetFirmware(void * handle);

3.3.12 HBI_GetFPDSerialNumber

* No: No12

* Function Name: HBI_GetFPDSerialNumber

* Function Description: Get the version of DLL

* Parameter Description:

In: void * handle - unsigned handle pointer

In/Out: char * szSn, length 14 bits, version information string for DLL

* Return Value: int

0- Successful

Non 0- failed, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetFPDSerialNumber(void *handle, char *szSn);

3.3.13 HBI_GetSDKVersion

* No: No13

* Function Name: HBI_GetSDKVerion

* Function Description: Get the version of DLL

* Parameter Description:

In: void * handle - unsigned handle pointer

char * szVer - Version information string

Out: None

* Return Value: int

0- Successful

Non 0- failed, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetSDKVerion(char * szVer);



3.3.14 HBI_GetFirmwareVerion

/*****

* No: No14

* Function Name: HBI_GetFirmwareVerion

* Function Description: Get the version of firmware

* Parameter Description:

In: void * handle - unsigned handle pointer

char * szVer - Version information string

Out: None

* Return Value: int

0- Successful

Non 0- failed ,refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetFirmwareVerion (void * handle, char * szFirmwareVer);

3.3.15 HBI_GetError

/*****

* No: No015

* Function Name: HBI_GetError

* Function Description: Get error information, not support yet

* Parameter Description:

Const CodeStringTable * inTable - Error message list

int count -Number of information lists

int recode - Error code

Out: None

* Return Value: const char *

Non NULL - Success, error message

NULL - Failed

* Comment:

*****/

HBI_FPD_DLL const char * __stdcall HBI_GetError (const CodeStringTable * inTable, int count, int recode);

3.3.16 HBI_GetFpdCfgInfo

/*****

* No: No016

* Function Name: HBI_GetFpdCfgInfo

* Function Description: Get the configuration of firmware

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: RegCfgInfo * pRegCfg, see HbType. h.

* Return Value: int

0- Successful



DT Imaging Technology (Shanghai) Co., Ltd

Non 0- failed ,refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetFpdCfgInfo (void * handle, RegCfgInfo * pRegCfg);

3.3.17 HBI_GetImageProperty

* No: No17

* Function Name: HBI_GetImageProperty

* Function Description: get image property

* Parameter Description:

In: void * handle - unsigned handle pointer

IMAGE_PROPERTY *img_pro // Image property, refer to<HbiError.h> IMAGE_PROPERTY

Out: None

* Return Value: int

0- Successful

Non 0- failed ,refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetImageProperty (void * handle, IMAGE_PROPERTY * img_pro);

3.3.18 HBI_SinglePrepare

* No: No18

* Function Name: HBI_SinglePrepare

* Function Description: Single image acquisition(combine with “prepare delay time” parameter)

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: None

* Return Value: int

0- Successful

Non 0- failed ,refer to <HbiError.h>

* Comment:In the application, there will be a problem that the acquisition command type cannot be synchronized, resulting in the failure of the above figure. Use HBI_Prepare replacement

*Prepare delay=0 - indicates soft trigger single frame acquisition first HBI_Prepare after HBI_SingleAcquisition completes single frame acquisition

*Prepare delay > "0 - means soft trigger single frame acquisition, only HBI_Prepare is required can complete single frame acquisition according to the delay time

*****/

HBI_FPD_DLL int HBI_SinglePrepare (void * handle);

3.3.19 HBI_SingleAcquisition

* No: No19



DT Imaging Technology (Shanghai) Co., Ltd

* Function Name: HBI_SingleAcquisition

* Function Description: Single image acquisition

* Parameter Description:

In: void *handle // unsigned handle pointer

FPD_AQC_MODE _mode // Acquisition mode and related parameters

EnumIMAGE_ACQ_MODE // Image acquisition mode

SINGLE_ACQ_DEFAULT_TYPE, // Single frame acquisition

SINGLE_ACQ_GAIN_IMAGE, // Distributed generation correction template for template validation,

creating Gain template - single frame acquisition of bright field images

SINGLE_ACQ_DEFECT_IMAGE, // Distributed generation of correction templates for template

validation and creation of Defect templates - single frame acquisition of bright field images

EnumLIVE_ACQUISITION eLivetype; // Only for ONLY_IMAGE

int ngroupano // Group number

int nframesum // Acquisition image number

int ndiscard // Discard image number

int nframeid // Frame ID number

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int HBI_SingleAcquisition (void * handle, FPD_AQC_MODE _mode);

3.3.20 HBI_LiveAcquisition

*****/

* No: No20

* Function Name: HBI_LiveAcquisition

* Function Description: Live acquisition

* Parameter Description:

In: void *handle // unsigned handle pointer

FPD_AQC_MODE _mode // Work mode and related parameter

The structure contains fields:

EnumIMAGE_ACQ_CMD eAqcCmd; // Acquisition command, Refer to the document "HbiType. h", the default drawing

method is LIVE_ACQ_DEFAULT_TYPE, pay attention to distinguish between single-frame acquisition and continuous acquisition commands

LIVE_ACQ_DEFAULT_TYPE, // Acquisition command

// Distribute and generate correction templates for template validation

LIVE_ACQ_OFFSET_IMAGE, // Create Offset template - continuously acquire dark field image

LIVE_ACQ_GAIN_IMAGE, // Create Gain template - continuous acquisition of bright field image

LIVE_ACQ_DEFECT_IMAGE, // Create Defect template - continuous acquisition of bright field image

// Quickly generate correction templates for system integration

OFFSET_TEMPLATE_TYPE, // Quickly generate template collection types, continuously collect a group of dark field

images and generate offset templates, and firmware generates templates



DT Imaging Technology (Shanghai) Co., Ltd

GAIN_TEMPLATE_TYPE, // Quickly generate template collection types, continuously collect a group of bright field images and generate gain templates

DEFECT_TEMPLATE_GROUP1, // Quickly generate template collection types and continuously collect a group of bright field images - defect group1

DEFECT_TEMPLATE_GROUP2, // Quickly generate template collection types and continuously collect a group of bright field images - defect group2

DEFECT_TEMPLATE_GROUP3, // Quickly generate template collection types and continuously collect a group of bright field images - defect group3 and generate template

SOFTWARE_OFFSET_TEMPLATE // Quickly generate template acquisition types and continuously collect a group of dark field images - SDK generates offset template

Enum LIVE_ACQUISITION eLivetype; // Live acquisition only, detail refer to LIVE_ACQ_DEFAULT_TYPE.

// 1 - generate the firmware Preoffset template and then continue to acquire image.

// 2 - image acquisition only.

// 3 - generate firmware Preoffset template only.

int ngrouppo; // Group number.

int nAcqnumber; // Reserved image number.

int ndiscard; // Discard image number.

int nframeid; // Counter.

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int HBI_LiveAcquisition (void * handle, FPD_AQC_MODE _mode);

3.3.21 HBI_StopAcquisition

*****/

* No: No21

* Function Name: StopAcquisition

* Function Description: Stop live acquisition

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: None

* Return Value: int

0- Successful

Non 0- failed, refer to <HbiError.h>

* 备 注:

*****/

HBI_FPD_DLL int HBI_StopAcquisition (void * handle);

3.3.22 HBI_SetSinglePrepareTime

*****/

* No: No22

* Function Name: HBI_SetSinglePrepareTime



DT Imaging Technology (Shanghai) Co., Ltd

* Function Description: set single frame acquisition delay

* Parameter Description:

In: void * handle - unsigned handle pointer

int * in_Itime - delay Time ,Unit: ms

0: “HBI_Prepare” and “HBI_SingleAcquisition” command are combined to realize single image acquisition

Non-0: FPD will delay “Prepare Delay Time” and automatically acquire one single image when the detector receives “HBI_Prepare” command.

* Return Value: int

0- Successful

Non 0- failed, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetSinglePrepareTime (void * handle, int in_itime);

3.3.23 HBI_GetSinglePrepareTime

*****/

* No: No23

* Function Name: HBI_GetSinglePrepareTime

* Function Description: get single frame acquisition delay

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: int *out_itime - delay Time ,Unit: ms

0: “HBI_Prepare” and “HBI_SingleAcquisition” command are combined to realize single image acquisition

Non-0: FPD will delay “Prepare Delay Time” and automatically acquire one single image when the detector receives “HBI_Prepare” command.

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetSinglePrepareTime (void * handle, int * out_itime);

3.3.24 HBI_SetLiveAcquisitionTime

*****/

* No: No24

* Function Name: HBI_SetLiveAcquisitionTime

* Function Description: Live acquisition span time setting (apply to static and wireless type FPD)

* Parameter Description:

In: void * handle - unsigned handle pointer

int time - Live acquisition span time, unit: ms

Out: None

* Return Value: int

0- Successful

Non 0- failed ,refer to <HbiError.h>



DT Imaging Technology (Shanghai) Co., Ltd

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetLiveAcquisitionTime (void * handle, int time);

3.3.25 HBI_GetLiveAcquisitionTime

*****/

* No: No25

* Function Name: HBI_GetLiveAcquisitionTime

* Function Description: Get live acquisition span time (apply to static and wireless type FPD)

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: int *out_time // Live acquisition span time, unit: ms

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetLiveAcquisitionTime (void * handle, int * out_time);

3.3.26 HBI_TriggerAndCorrectDisplay

*****/

* No: No26

* Function Name: HBI_TriggerAndCorrectApplay

* Function Description: trigger mode and image calibration setting

* Parameter Description:

In: void *handle // unsigned handle pointer

int_triggerMode

// 1: Software trigger mode

// 3: High voltage generator trigger mode

// 4: AED trigger mode

// 5-Dynamic:Hvg Sync Mode

// 6-Dynamic:Fpd Sync Mode

// 7-Dynamic:Fpd Continue

// 8-SAEC。

IMAGE_CORRECT_ENABLE* pCorrect // refer to <HbiType.h>

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment: Replace “HBI_WorkStationInitDllCfg” and “HBI_QuckInitDllCfg” API function

*****/

HBI_FPD_DLL int __stdcall HBI_TriggerAndCorrectDisplay (void * handle, int triggerMode, IMAGE_CORRECT_ENABLE * pCorrect);



DT Imaging Technology (Shanghai) Co., Ltd

3.3.27 HBI_UpdateTriggerMode

* No: No27

* Function Name: HBI_UpdateTriggerMode

* Function Description: trigger mode setting

In: void *handle // unsigned handle pointer
int _triggerMode,
// 1: Software trigger mode
// 3: High voltage generator trigger mode
// 4: AED trigger mode
// 5-Dynamic:Hvg Sync Mode
// 6-Dynamic:Fpd Sync Mode
// 7-Dynamic:Fpd Continue
// 8-SAEC。

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_UpdateTriggerMode (void * handle, int triggerMode);

3.3.28 HBI_UpdateCorrectEnable

* No: No28

* Function Name: HBI_UpdateCorrectEnable

* Function Description: update the setting of firmware calibration enable

* Parameter Description:

In: void *handle // unsigned handle pointer
IMAGE_CORRECT_ENABLE* pCorrect // refer to < HbiType.h>

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_UpdateCorrectEnable (void * handle, IMAGE_CORRECT_ENABLE * pCorrect);

3.3.29 HBI_GetCorrectEnable

* No: No29

* Function Name: HBI_GetCorrectEnable

* Function Description: // Get image calibration enable status

* Parameter Description:



DT Imaging Technology (Shanghai) Co., Ltd

In: void *handle // unsigned handle pointer

IMAGE_CORRECT_ENABLE* pCorrect // refer to <HbiType.h>

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetCorrectEnable (void * handle, IMAGE_CORRECT_ENABLE * pCorrect);

3.3.30 HBI_SetSoftwareCorrect

*****/

* No: No30

* Function Name: HBI_SetSoftwareCorrect

* Function Description: Software calibration enable setting

* Parameter Description:

In: void *handle // unsigned handle pointer

SOFTWARE_CALIBRATE_ENABLE inEnable // refer to<< HbiType.h>> SOFTWARE_CALIBRATE_ENABLE

structure

Out: None

* Return Value: int

0 successful

Non 0 failed, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetSoftwareCorrect (void * handle,

SOFTWARE_CALIBRATE_ENABLE inEnable);

3.3.31 HBI_GetSoftwareCorrect

*****/

* No: No31

* Function Name: HBI_GetSoftwareCorrect

* Function Description: // Get software calibration enable status

* Parameter Description:

In: void *handle // unsigned handle pointer

SOFTWARE_CALIBRATE_ENABLE inEnable // refer to<HbiError.h>

SOFTWARE_CALIBRATE_ENABLE struct

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:新接口

*****/

HBI_FPD_DLL int __stdcall HBI_GetSoftwareCorrect (void * handle,



3.3.32 HBI_SetAedIntegrateTm

```
/**
 * No: No32
 * Function Name: HBI_SetAedIntegrateTm
 * Function Description: AED integration time setting
 * Parameter Description:
     In:   void *handle    // unsigned handle pointer
          int time         // AED integration time, unit: ms, default = 500ms
     Out: Null
 * Return Value: int
          // 0: Success
          // Non-0: Failure
 * Comment:
 *****/
```

```
HBI_FPD_DLL int __stdcall HBI_SetAedIntegrateTm (void * handle, int time);
```

3.3.33 HBI_GetAedIntegrateTm

```
/**
 * No: No33
 * Function Name: HBI_GetAedIntegrateTm
 * Function Description: Get AED integration time
 * Parameter Description:
     In:   void *handle    // unsigned handle pointer
     Out: int *out_time    // AED integration time, unit: ms
 * Return Value: int
          0- Successful
          Non 0- failed
 * Comment:
 *****/
```

```
HBI_FPD_DLL int __stdcall HBI_GetAedIntegrateTm (void * handle, int * out_time);
```

3.3.34 HBI_SetAedThreshold

```
/**
 * No: No34
 * Function Name: HBI_SetAedThreshold
 * Function Description: AED threshold value setting
 * Parameter Description:
     In:   void *handle    // unsigned handle pointer
          int out_ivalue   // AED threshold (10,000~1,000,000)
     Out: None
 * Return Value: int
          0- Successful
```



DT Imaging Technology (Shanghai) Co., Ltd
Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetAedThreshold (void * handle, int in_ivalue);

3.3.35 HBI_GetAedThreshold

* No: No35

* Function Name: HBI_GetAedThreshold

* Function Description: Obtain AED threshold

* Parameter Description:

In: void * handle -unsigned handle pointer (unsigned pointer)

Out: int * out_ Ivalue - threshold [10000~1000000]

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetAedThreshold (void * handle, int * out_ivalue);

3.3.36 HBI_SetsaturationValue

* No: No36

* Function Name: HBI_SetSaturationValue

* Function Description: Saturation value setting

* Parameter Description:

In: void *handle // unsigned handle pointer

Out:int *out_ivalue // Saturation value (0 ~ 65,535)

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetsaturationValue (void * handle, int in_ivalue);

3.3.37 HBI_GetsaturationValue

* No: No37

* Function Name: HBI_GetSaturationValue

* Function Description: Get image saturation value

* Parameter Description:

In: void * handle -unsigned handle pointer (

Out: int * out_ivalue - saturation value [0~65535]

* Return Value: int



DT Imaging Technology (Shanghai) Co., Ltd

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetsaturationValue (void * handle, int * out_ivalue);

3.3.38 HBI_SetClippingValue

* No: No38

* Function Name: HBI_SetClippingValue

* Function Description: Clipping value setting

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: int * out_ivalue - clipping value [0~65535]

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetClippingValue (void * handle, int in_ivalue);

3.3.39 HBI_GetClippingValue

* No: No39

* Function Name: HBI_GetClippingValue

* Function Description: Get clipping value

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: int * out_ivalue - clipping value [0~65535]

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetClippingValue (void * handle, int * out_ivalue);

3.3.40 HBI_SetLPFLevel

* No: No40

* Function Name: HBI_SetLPFLevel

* Function Description: Low pass filter setting

* Parameter Description:

In: void * handle -unsigned handle pointer (unsigned pointer)

int nLPFLevel - Gear



DT Imaging Technology (Shanghai) Co., Ltd

[n] - Invalid

[0] -210kHz, default to 210kHz

[1] -105khz

[2] -52khz

[3] -26khz

[4] -13khz

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetLPFLevel (void * handle, int nLPFLevel);

3.3.41 HBI_GetLPFLevel

* No: No41

* Function Name: HBI_GetLPFLevel

* Function Description: Get pass filter

* Parameter Description:

In: void * handle -unsigned handle pointer (

Out: None

* Return Value: int

[0] -210kHz, default to 210kHz

[1] -105khz

[2] -52khz

[3] -26khz

[4] -13khz

[n] - Invalid

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetLPFLevel (void * handle);

3.3.42 HBI_SetPGAlevel

* No: No042

* Function Name: HBI_SetPGAlevel

* Function Description: PGA gain stage setting

* Parameter Description:

In: void * handle - unsigned handle pointer

int mode - Mode

[n] - Invalid

[1] -0.6pC

[2] -1.2pC



DT Imaging Technology (Shanghai) Co., Ltd

[3] -2.4pC

[4] -3.6pC

[5] -4.8pC

[6] -7.2pC, default to 7.2pC

[8] - COMS tablet, LFW

[9] - COMS tablet, HFW

[10] -0.3pC

[11] -0.15pC

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetPGALevel (void * handle, int nGainLevel);

3.3.43 HBI_GetPGALevel

* No: No43

* Function Name: HBI_GetPGALevel

* Function Description: Get PGA gain stage setting value

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: None

* Return Value: int

[n] - Invalid

[1] -0.6pC

[2] -1.2pC

[3] -2.4pC

[4] -3.6pC

[5] -4.8pC

[6] -7.2pC

[7] -9.6pC

[8] - COMS tablet, LFW

[9] - COMS tablet, HFW

[10] -0.3pC

[11] -0.15pC

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetPGALevel (void * handle);

3.3.44 HBI_SetBinning

* No: No044



DT Imaging Technology (Shanghai) Co., Ltd

* Function Name: HBI_SetBinning

* Function Description: Binning type setting

* Parameter Description:

In:

Void * handle -unsigned handle pointer

unsigned int nbins // Binning type

// 1: 1x1

// 2: 2x2

// 3: 3x3

// 4: 4x4

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetBinning(void * handle, unsigned int nbins);

3.3.45 HBI_SetBinning

*****/

* No: No45

* Function Name: HBI_GetBinning

* Function Description: Get binning type

* Parameter Description:

In: void * handle -unsigned handle pointer

unsigned int *nbins // Binning mode

// 1: 1x1

// 2: 2x2

// 3: 3x3

// 4: 4x4

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetPGALevel (void * handle);

3.3.46 HBI_SetPreviewMode

*****/

* No: No46

* Function Name: HBI_GetPreviewMode

* Function Description: Preview mode setting

* Parameter Description:



DT Imaging Technology (Shanghai) Co., Ltd

In: void *handle - // unsigned handle pointer

int inTriggMode

1-Soft trigger

3-High voltage trigger

4-FreeAED

5-Dynamic:Hvg Sync Mode

6-Dynamic:Fpd Sync Mode

7-Dynamic:Fpd Continue

8-SAEC

Other not supported。

int inPreviewMode // Image acquire and display mode

// 0: normal image,

// 1: preview image

// 2: preview + normal image

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetPreviewMode (void * handle, int inTriggMode, int inPreviewMode);

3.3.47 HBI_GetPreviewMode

*****/

* No: No047

* Function Name: HBI_GetPreviewMode

* Function Description: Get preview mode setting value

* Parameter Description:

In: void * handle -unsigned handle pointer (unsigned pointer)

Out: int *outmode // Image acquire and display mode [0~2]

// 0: normal image,

// 1: preview image

// 2: preview + normal image

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetPreviewMode (void * handle, int * outMode);

3.3.48 HBI_SAecModeApply

*****/

* No: No48

* Function Name: HBI_SAecModeApply



DT Imaging Technology (Shanghai) Co., Ltd

* Function Description:SAEC mode setting

* Parameter Description:

In: void * handle -unsigned handle pointer
 int inPreviewMode
 0-normal image
 1-preview image
 int inPerTime - fpd pre ready delay time
 int inPostTime - fpd post ready delay time

Out: None

* Return Value: int

0- Successful
 Non 0- failed

* Comment:

1> Enable: inPreviewMode=1 (preview image), inPerTime, and inPostTime (customer specified)

2> Cancel: inPreviewMode=0 (normal image), inPerTime and inPostTime can default to 0

*****/

HBI_FPD_DLL int __stdcall HBI_SAecModeApply (void * handle, int inPreviewMode, int inPerTime=0, int inPostTime=0);

3.3.49 HBI_SAecAcq

*****/

* No: No49

* Function Name: HBI_SAecAcq

* Function Description: SAEC image acquisition

* Parameter Description:

In: void * handle -unsigned handle pointer
 Out: None

* Return Value: int

0- Successful
 Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SAecAcq (void * handle);

3.3.50 HBI_RegProgressCallBack

*****/

* No: No50

* Function Name: HBI_RegProgressCallBack

* Function Description: Register callback function, feedback template or firmware downloading progress status

* Parameter Description:

In: void * handle -unsigned handle pointer
 USER_CALLBACK_HANDLE_PROCESS handleStatusfun-Register callback function

Void * pObject - Object pointer. Generally, the upper computer software passes the object pointer into the SDK, and the callback function provides feedback to the upper computer for static or global functions to access other objects. It can also be a null pointer.



DT Imaging Technology (Shanghai) Co., Ltd

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

```
HBI_FPD_DLL int HBI_RegEventCallBackFun (void * handle,  
USER_CALLBACK_HANDLE_ENVENT callback, void * pContext);
```

3.3.51 HBI_GenerateTemplate

*****/

* No: No51

* Function Name: HBI_GenerateTemplate

* Function Description: // Quick generate offset, gain, defect template

* Parameter Description:

In: void *handle // unsigned handle pointer
EnumIMAGE_ACQ_MODE _mode // Type of template
OFFSET_TEMPLATE_MODE // Offset template
GAIN_TEMPLATE_MODE // Gain template
DEFECT_ACQ_GROUP1, // Defect template with group1 image
DEFECT_ACQ_GROUP2, // Defect template with group2 image
DEFECT_ACQ_AND_TEMPLATE // Defect template with group3 image and generate template

int bpreview // Whether to generate preview template

// 1: yes

// 0: no (default)

Out: Null

* Return Value:

int // 0: Success

// Non-0: Failure

* Comment:

* 1.OFFSET_TEMPLATE: Generate a dark field template, note that the ball tube is closed, and the firmware collects a set of dark field images and generates an Offset template

* 2.GAIN_TEMPLATE: To generate a gain correction template, it is necessary to open the ball tube, adjust the dose switch, and start the plate to collect bright field images and generate a gain template

* 3.DEFECT_TEMPLATE_GROUP1: To generate a bad spot correction template, it is necessary to open the ball tube, adjust the dose, and start collecting the first group of bright field images on the flat panel

* 4.DEFECT_TEMPLATE_GROUP2: To generate a bad spot correction template, it is necessary to open the ball tube, adjust the dose, and start collecting the second group of bright field images on the flat panel

* 5.DEFECT_TEMPLATE_GROUP3: To generate a bad spot correction template, it is necessary to open the ball tube, adjust the dose, and start the plate to collect the third group of bright field images and generate a bad template

* 6.SOFTWARE_OFFSET_TEMPLATE: Generate a dark field template. Note that the ball tube is closed. The SDK collects a set of dark field images and generates an Offset template

*****/

```
HBI_FPD_DLL int __stdcall HBI_GenerateTemplate (void * handle, EnumIMAGE-ACQ-MODE _mode, int bprev=0);
```



3.3.52 HBI_DownloadTemplate

* No: No52

* Function Name: HBI_DownloadTemplate

* Function Description: Download calibration template

* Parameter Description:

In: void * handle -unsigned handle pointer

DOWNLOAD_FILE * downloadfile - Download template file object pointer

int nModeID:Mode ID,when managing based on multiple Mode templates, it is necessary to specify the Mode ID.

emUPLOAD_FILE_TYPE emfiletype-Template type(OFFSET_TMP-offset template.GAIN_TMP-gain template,DEFECT_TMP-defect template)

int nBinningtype-template binning type (1-binning1*1,2-binning2*2,3-binning3*3,4-binning4*4)

char filepath-Template absolute path

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:Suggest replacing with **HBI_DownloadTemplateByType**

*****/

HBI_FPD_DLL int __stdcall HBI_DownloadTemplate (void * handle, DOWNLOAD_FILE * downloadfile);

3.3.53 HBI_StopDownloadTemplate

* No: No53

* Function Name: HBI_StopDownloadTemplate

* Function Description: Stop download calibration template

* Parameter Description:

In: void * handle -unsigned handle pointer

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_StopDownloadTemplate (void * handle);

3.3.54 HBI_InitDual

* No: No54

* Function Name: HBI_InitDual

* Function Description: Initialize dual the flat detector

* Parameter Description: The return parameter contains two handles



DT Imaging Technology (Shanghai) Co., Ltd

In/Out: void * * handle1- unsigned object pointer for instance of panel A

In/Out: void * * handle1- unsigned object pointer for instance of panel B

* Return Value:

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_InitDual (void * * handle1, void * * handle2);

3.3.55 HBI_InitEx

*****/

* No: No55

* Function Name: HBI_InitEx

* Function Description: Initialize double or multi detector dynamic library instance

* Parameter Description: Callback parameter, FPD instance and ID numbe

In/Out: HBI_HANDLE *pHandArr // Array object, refer to <HbiError.h>

In: int nArrSize // Array number, equal to DETECTOR_MAX_NUMBER

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_InitEx (HBI_HANDLE * pHandArr, int nArrSize);

3.3.56 HBI_DestroyEx

*****/

* No: No56

* Function Name: HBI_DestroyEx

* Function Description: Destroy all dll resources

* Parameter Description: none

* Return Value: none

* Comment:

*****/

HBI_FPD_DLL void __stdcall HBI_DestroyEx();

3.3.57 HBI_GetDetectorHandle

*****/

* No: No57

* Function Name: HBI_GetDetectorHandle

* Function Description: Get handle through FPD ID number

* Parameter Description:

In: int fpdid - tablet id, 0 "panel id"

* Return Value:

Void * - unsigned object pointer



DT Imaging Technology (Shanghai) Co., Ltd

* Comment:

*****/

HBI_FPD_DLL void* __stdcall HBI_GetDetectorHandle (int fpdid=0);

3.3.58 HBI_GetDetectorId

* No: No58

* Function Name: HBI_GetDetectorId

* Function Description: Get FPD ID number through handle

* Parameter Description:

In: void * handle - handle pointer

* Return Value:

int – The ID of detector

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetDetectorId (void * handle);

3.3.59 HBI_ActivateDetector

* No: No59

* Function Name: HBI_ActivateDetector

* Function Description: Activate detector, current SDK version supports multi FPD

* Parameter Description:

In: void * handle - Tablet handle

* Return Value:

0- Successful, callback function feedback firmware parameters

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_ActivateDetector (void * handle);

3.3.60 HBI_GetPreviewImageProperty

* No: No60

* Function Name: HBI_GetPreviewImageProperty

* Function Description: Get preview image property

* Parameter Description:

In: void * handle -unsigned handle pointer

IMAGE_PROPERTY *img_pro//Image property, refer to<HbiError.h> IMAGE_PROPERTY

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/



DT Imaging Technology (Shanghai) Co., Ltd

HBI_FPD_DLL int __stdcall HBI_GetPreviewImageProperty (void * handle, IMAGE_PROPERTY * img_pro);

3.3.61 HBI_PgaBinningAcqTime

/******

* No: No61

* Function Name: HBI_PgaBinningAcqTime

* Function Description: PGA gain stage, binning, frame speed setting

* Parameter Description:

In: void * handle -unsigned handle pointer

int nPgaLevel - PGA gear, [n] - Invalid, [1] -0.6pC, [2] -1.2pC, [3] -2.4pC, [4] -3.6pC, [5] -4.8pC, [6] -7.2pC, default 7.2pC, [7] -9.6pC, [10] -0.3pC, [11] -0.15pC

unsigned char binning -1:1x1,2:2x2,3:3x3,4:4x4, not supported for others

int acqtime - interval time, in milliseconds, greater than 0

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_PDF_DLL int __stdcall HBI_PgaBinningAcqTime (void * handle, int nPgaLevel, unsigned char binning, int acqtime, int nfeedback=0);

3.3.62 HBI_TriggerPgaBinningAcqTime

/******

* No: No62

* Function Name: HBI_TriggerPgaBinningAcqTime

* Function Description: // trigger mode, PGA gain stage, binning type, frame speed setting

* Parameter Description:

In: void *handle // unsigned handle pointer

int triggerMode // trigger mode

// Static:

// 01: Software trigger mode;

// 02: Clear;

// 03: High voltage generator(hvg) trigger mode;

// 04: Auto exposure detection mode.

// Dynamic:

// 05: Hvg pulse sync in trigger mode;

// 06: Fpd pulse sync out trigger Mode;

// 07: Free run mode.

int nPgaLevel // PGA gain stage,[n]-Invalid,[1]-0.6pC,[2]-1.2pC,[3]-2.4pC,[4]-3.6pC,[5]-4.8pC,[6]-7.2pC,[7]-9.6pC,[8]-LFW (COMS),[9]-HFW (COMS),[10]-0.3pC,[11]-0.15pC,Default: 7.2 pC.

unsigned char binning

// 1: 1x1,



DT Imaging Technology (Shanghai) Co., Ltd

// 2: 2x2,

// 3: 3x3,

// 4: 4x4,

int acqtime // Live acquisition span time, unit: ms

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

* Return Value:

int // 0: Success

// Non-0: Failure

* Comment:

*****/

HBI_PDF_DLL int __stdcall HBI_TriggerPgaBinningAcqTime (void * handle, int triggerMode, int nPgaLevel, unsigned char binning, int acqtime, int nfeedback=0);

3.3.63 HBI_TriggerBinningAcqTime

*****/

* No: No63

* Function Name: HBI_TriggerBinningAcqTime

* Function Description: // trigger mode, binning, frame speed setting

* Parameter Description:

In: void *handle // unsigned handle pointer

int triggerMode // trigger mode

// Static:

// 01: Software trigger mode;

// 02: Clear;

// 03: High voltage generator(hvg) trigger mode;

// 04: Auto exposure detection mode.

// Dynamic:

// 05: Hvg pulse sync in trigger mode;

// 06: Fpd pulse sync out trigger Mode;

// 07: Free run mode.

unsigned char binning

// 1: 1x1,

// 2: 2x2,

// 3: 3x3,

// 4: 4x4,

int time // Live acquisition span time, unit: ms

// For example: 1000ms (1fps), 500ms (2fps), 250ms (4fps), 125ms (8fps)

Out: Null

* Return Value:

int // 0: Success

// Non-0: Failure

* Comment:

*****/



DT Imaging Technology (Shanghai) Co., Ltd

HBI_PDF_DLL int __stdcall HBI_TriggerBinningAcqTime (void * handle, int triggerMode, unsigned char binning, int time);

3.3.64 HBI_PgaAcqTime

/******

* No: No64

* Function Name: HBI_PgaAcqTime

* Function Description: PGA gain stage, frame speed setting

* Parameter Description:

In: void *handle // unsigned handle pointer

int nPgaLevel //PGA gain stage,[n]-Invalid,[1]-0.6pC,[2]-1.2pC,[3]-2.4pC,[4]-3.6pC,[5]-4.8pC,[6]-7.2pC,[7]-9.6pC,[8]-

LFW (COMS),[9]-HFW (COMS),[10]-0.3pC,[11]-0.15pC,Default: 7.2 pC.

int acqtime // Live acquisition span time, unit: ms

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_PDF_DLL int __stdcall HBI_PgaAcqTime (void * handle, int nPgaLevel, int acqtime, int nfeedback=0);

3.3.65 HBI_SetSelfDumpingTime

/******

* No: No65

* Function Name: HBI_SetSelfDumpingTime

* Function Description: Self dumping span time setting (set fps) (Dynamic flat panel detector)

* Parameter Description:

In: void * handle -unsigned handle pointer

int time - interval time, in milliseconds,>=1000ms

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetSelfDumpingTime (void * handle, int time);

3.3.66 HBI_GetSelfDumpingTime

/******

* No: No66

* Function Name: HBI_GetSelfDumpingTime

* Function Description: Get self dumping span time(Dynamic flat panel detector)

* Parameter Description:

In: void * handle -unsigned handle pointer



DT Imaging Technology (Shanghai) Co., Ltd
Time - Time, in milliseconds and milliseconds

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetSelfDumpingTime (void * handle, int * out_time);

3.3.67 HBI_TriggerPgaBinningAcqTimeCorrect

*****/

* No: No067

* Function Name: HBI_TriggerPgaBinningAcqTimeCorrect

* Function description: trigger mode, PGA gain stage, binning type, frame speed, correction enabling setting

* Parameter Description:

In: void *handle - Handle (unsigned pointer)

int triggerMode // Tigger mode

// Static:

// 01: Software trigger mode;

// 02: Clear;

// 03: High voltage generator(hvg) trigger mode;

// 04: Auto exposure detection mode.

// Dynamic:

// 05: Hvg pulse sync in trigger mode;

// 06: Fpd pulse sync out trigger Mode;

// 07: Free run mode.

int nPgaLevel // PGA gain stage,[n]-Invalid,[1]-0.6pC,[2]-1.2pC,[3]-2.4pC,[4]-3.6pC,[5]-4.8pC,[6]-7.2pC,[7]-9.6pC,[8]-

LFW (COMS),[9]-HFW (COMS),[10]-0.3pC,[11]-0.15pC,Default: 7.2 pC.

unsigned char binning

// 1: 1x1,

// 2: 2x2,

// 3: 3x3,

// 4: 4x4,

int acqtime // Live acquisition span time, unit: ms

IMAGE_CORRECT_ENABLE* pCorrect, refer to <HbiError.h>

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

* Return value: int

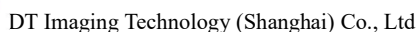
0 - Success

Non 0 - Failed

* Remarks:

*****/

HBI_FPD_DLL int __stdcall HBI_TriggerPgaBinningAcqTimeCorrect(void* handle, int triggerMode, int nPgaLevel, unsigned char binning, int acqtime, IMAGE_CORRECT_ENABLE* pCorrect);



/*****

```
* Comment:
*****/
HBI_FPD_DLL int stdcall HBI_OpenTemplateWizard (void * handle);
```

/*****

Copyright © 2021 DT Imaging Technology (Shanghai) Co., Ltd. All Rights Reserve



DT Imaging Technology (Shanghai) Co., Ltd

Non-0: Failure

* Comment:only 1613-125um.3030-140um.4343-100um.2530-100um are supported.

*****/

HBI_FPD_DLL int __stdcall HBI_SetRoiMode (void * handle, int nMode, int nfeedback=1);

3.3.71 HBI_SetZoomModeRect

* No: No071

* Function Name: HBI_SetZoomModeRect

* Function Description: The ZOOM Mode region setting function for CMOS tablets

* currently supports only CMOS - 1412-50um and CMOS - 0606-50um ,1917-120umtablet types.

* Parameter Description:

In: void *handle - unsigned handle pointer

CMOS_ZOOM_RECT stZoomRect - Zoom region

The region is represented by left, top, right, and bottom.

for example, when left, top, right and bottom are illegal values, the image will display the full image by default,

1>When left=top=right=bottom=0, the whole picture is displayed;

2>When left<0 or left>width, the whole picture will be displayed, and others are the same.

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

Out: Null

* Return Value: int

0: Success

Non-0: Failure

* Comment: currently supports only CMOS - 1412-50um and CMOS - 0606-50um 1917-120um tablet types.

*****/

HBI_FPD_DLL int __stdcall HBI_SetZoomModeRect (void * handle, CMOS_ZOOM_RECT stZoomRect, int nfeedback=1);

3.3.72 HBI_SetCorrectFileDirectory

* No: No072

* Function Name: HBI_SetCorrectFileDirectory

* Function Description:User set correction file home directory

* If this interface is not called, the calibration file defaults to the DLL directory

* Setting failed. The default is the current path*Setting failed, defaults to current path

* Parameter Description:

In: void *handle - unsigned handle pointer

char * pUserDir - User defined calibration file directory, for example: E:******\\

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/



DT Imaging Technology (Shanghai) Co., Ltd

HBI_FPD_DLL int __stdcall HBI_SetCorrectFileDirectory (void * handle, char * pUserDir);

3.3.73 HBI_GetCorrectFileDirectory

/******

* No: No073

* Function Name: HBI_GetCorrectFileDirectory

* Function Description: Obtain the home directory of user correction files

* Parameter Description:

In: void *handle - unsigned handle pointer

Out: Null

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL char * __stdcall HBI_GetCorrectFileDirectory (void * handle);

3.3.74 HBI_UpdateSaveOption

/******

* No: No074

* Function Name: HBI_UpdateSaveOption

* Function Description: Update Save Image Options

* Parameter Description:

In: void *handle - unsigned handle pointer

SAVE_IMAGE saveimge - Structure parameters

unsigned char m_bySaveType; // 0-show image only,1-save image only,2-show and save image;

unsigned char m_bySaveFileType; // 0-image,1-video;

unsigned short m_bySaveNumber; // Number of images included in the video

Out: Null

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_UpdateSaveOption(void* handle, SAVE_IMAGE saveimge);

3.3.75 HBI_SetPacketIntervalTime

/******

* No: No075

* Function Name: HBI_SetPacketIntervalTime

* Function Description: Set Package Interval

* Parameter Description:

In: void *handle - unsigned handle pointer

int *in_itime - TIME, UNIT:um



DT Imaging Technology (Shanghai) Co., Ltd

Out: Null

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetPacketIntervalTime (void * handle, int in_itime);

3.3.76 HBI_AutoWindowWidthWindowLevel

*****/

* No: No076

* Function Name: HBI_AutoWindowWidthWindowLevel (新增)

* Function Description: The automatic window width and window level algorithm converts 16bits data into 8-bit unsigned data through window width and window level processing, and users can directly display 8-bit data.

* Parameter Description:

In: void *handle - unsigned handle pointer

In/Out: unsigned char *imgbuff, Source data (16 bit unsigned small end data), destination data (8-bit unsigned data)

In/Out: int *nbufflen, Buffer data length (halve the data size after conversion)

In: int nwidth - Image data width

In: int nheight - Image data high

Out: Null

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_AutoWindowWidthWindowLevel (void * handle, unsigned char * imgbuff, int * nbufflen, int nwidth, int height);

3.3.77 HBI_SetImageRotateAndFlip

*****/

* No: No077

* Function Name: HBI_SetImageRotateAndFlip

* Function Description: Rotate 90 ° or enable mirroring (SDK processing, applicable to low frame rate or small resolution tablets)

* Parameter Description:

In: void *handle - unsigned handle pointer

In: int nRotate - 0-Do nothing;1-Left Rotate 90°;02-"Right Rotate 90°;

In: int nFlip - 0-Do nothing;1-Horizontal mirror;2-Mirror vertically;

Out: Null

* Return Value: int

0: Success

Non-0: Failure

* Comment: image processing, By default, rotate before mirro

*****/



DT Imaging Technology (Shanghai) Co., Ltd

HBI_FPD_DLL int __stdcall HBI_SetImageRotateAndFlip (void * handle, int nRotate, int nFlip);

3.3.78 HBI_Rotate90AndFlip

/*****

* No: No078

* Function Name: HBI_Rotate90AndFlip (newly increased)

* Function Description: Image rotation and image processing (only applicable to 16-bit unsigned image data, which is rotated first and then mirrored by default)

* Parameter Description:

In: void *handle - unsigned handle pointer

In/Out: unsigned char *imgbuff - Pointer type input and output parameters, source data (16-bit unsigned small-end data), destination data (16-bit unsigned small-end data)

In: int nbufflen - Buffer data length

In/Out: int *pnwidth - Pointer type input and output parameters, pointer address storage image data width

In/Out: int *pnheight - Pointer type input and output parameters, pointer address storage image data high

In: int nClockwise - Rotate 90 °, 1 - right rotation, 2 - left rotation, other - no processing In: int nVertical - Mirror processing, 1-vertical mirror, 2-horizontal mirror, other - no processing

* Return Value: int

0: Success

Non-0: Failure

* Comment:By default, rotate before mirror

*****/

HBI_FPD_DLL int __stdcall HBI_Rotate90AndFlip (void * handle, unsigned char * imgbuff, int nbufflen, int * pnwidth, int * pnheight, int nClockwise, int nVertical);

3.3.79 HBI_ResetCorrectTemplate

/*****

* No: No079

* Function Name: HBI_ResetCorrectTemplate (newly increased)

* Function Description: Reset the calibration template file - rotate, mirror, reverse process and save, generally complete the process before downloading to the firmware

* 1.Find the generated calibration template file according to the current rotation and mirroring parameters (the template file has been rotated and mirrored);

* 2.Reverse process the calibration template file (mirror first and then reverse rotation);

* 3.Automatically store the rotated calibration template file according to multi-template management rules;

* Parameter Description:

In: void *handle - unsigned handle pointer

In: int nTemplateType - 0-offset template(16-bits);1-gain template(32-bits);2-defect template(16-bits);

* Return Value: int

0: Success

Non-0: Failure

* Comment:The purpose is to solve the problem that the firmware calibration calibration file is not synchronized. The firmware uses the generated calibration template file that has not been rotated and mirrored by the dynamic library

*****/



DT Imaging Technology (Shanghai) Co., Ltd

HBI_FPD_DLL int __stdcall HBI_ResetCorrectTemplate (void * handle, int nTemplateType);

3.3.80 HBI_GetCurZoomRect

/******

* No: No080

* Function Name: HBI_GetCurZoomRect (newly increased)

* Function Description: Get the current ZOOM area size data

* Parameter Description:

In: void *handle - unsigned handle pointer

Out: unsigned int *pZoomLeft - Zoom Rect Left

unsigned int *pZoomTop - Zoom Rect Top

unsigned int *pZoomWidth - Zoom Rect Width

unsigned int *pZoomHeight - Zoom Rect Height

* Return Value: int

0: Success

Non-0: Failure

* Comment: Rect data, where all are 0, indicating the full picture.

HBI_FPD_DLL int __stdcall HBI_GetCurZoomRect (void * handle, unsigned int * pZoomLeft, unsigned int * pZoomTop, unsigned int

* pZoomWidth, unsigned int * pZoomHeight);

3.3.81 HBI_IsExistsOfCurCorrectTemplate

/******

* No: No081

* Function Name: HBI_IsExistsOfCurCorrectTemplate (newly increased)

* Function Description: Judge whether the current template file exists according to the template type

* Parameter Description:

In: void *handle - unsigned handle pointer

In: int tempType - Template type (default is 0), 0-offset template; 1-gain template; 2-defect template; 3-defect map template; 4-gainb template;

* Return Value: int

0: Success

Non-0: Failure

* Comment:

HBI_FPD_DLL int __stdcall HBI_IsExistsOfCurCorrectTemplate (void * handle, int tempType=0);

3.3.82 HBI_GetCurrentCorrectFileName

/******

* No: No082

* Function Name: HBI_GetCurrentCorrectFileName (newly increased)

* Function Description: Get the current template file name

* Parameter Description:

In: void *handle - unsigned handle pointer



DT Imaging Technology (Shanghai) Co., Ltd

In: int tempType -Template type (default is 0), 0-offset template;1-gain template;2-defect template;3-defect map template;4-gainb template;

* Return Value: char*

NULL - Failure

char* - Return file name

* Comment:

*****/

HBI_FPD_DLL char * __stdcall HBI_GetCurrentCorrectFileName (void * handle, int tempType=0);

3.3.83 HBI_GetCurTempFileFullDirectory

*****/

* No: No083

* Function Name: HBI_GetCurTempFileFullDirectory

* Function Description: Get the full directory of the current template file according to the template type

* Parameter Description:

In: void *handle - unsigned handle pointer

In: int tempType -Template type (default is 0), 0-offset template;1-gain template;2-defect template;

Out: Null

* Return Value: char*

NULL - Failure

char* - Return directory path

* Comment:

*****/

HBI_FPD_DLL char * __stdcall HBI_GetCurTempFileFullDirectory (void * handle, int tempType=0);

3.3.84 HBI_GetCurrentCorrectFilePath

*****/

* No: No084

* Function Name: HBI_GetCurrentCorrectFilePath

* Function Description: Obtain the absolute path of the current template file according to the template type

* Parameter Description:

In: void *handle - unsigned handle pointer

In: int tempType -Template type (default is 0), 0-offset template;1-gain template;2-defect template;3-defect map template;4-gainb template;

Out: Null

* Return Value: char*

NULL - Failure

char* - Return absolute path

* Comment:

*****/

HBI_FPD_DLL char * __stdcall HBI_GetCurrentCorrectFilePath (void * handle, int tempType=0);

3.3.85 HBI_Prepare

*****/



DT Imaging Technology (Shanghai) Co., Ltd

* No: No085

* Function Name: HBI_Prepare (Add interface)

* Function Description: Single image acquisition(combine with “prepare delay time” parameter)

* Parameter Description:

In: void *handle // unsigned handle pointer

EnumIMAGE_ACQ_CMD //Current acquisition command type (image above or saved template image data)

eg:

SINGLE_ACQ_DEFAULT_TYPE // Single acquisition.

SINGLE_ACQ_GAIN_IMAGE // Generate gain template with static single acquisition.

SINGLE_ACQ_DEFECT_IMAGE // Generate defect template with static single acquisition.

SINGLE_GENERATE_OFFSET, // Quickly generate template collection types, collect a group of dark field images in a single frame - SDK generates offset template.

SINGLE_GENERATE_GAIN, // Quickly generate template collection types, and collect N groups of single frame (at least 3 groups by default) bright field images - SDK generates gain templates SINGLE_GENERATE_DEFECT, // Quickly generate template collection types, and collect N groups of single frame (at least 3 groups by default) bright field images - SDK generates defect templates

SINGLE_GENERATE_GAIN_DEFECT // Quickly generate template collection types, and collect N groups of single frame (at least 3 groups by default) bright field images - SDK generates gain and defect templates

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*Prepare delay=0 - indicates soft trigger single frame acquisition first HBI_Prepare after HBI_SingleAcquisition completes single frame acquisition

*Prepare delay > "0 - means soft trigger single frame acquisition, only HBI_Prepare is required can complete single frame acquisition according to the delay time

*****/

HBI_FPD_DLL int __stdcall HBI_Prepare (void * handle, EnumIMAGE_ACQ_CMD acqcmd=EnumIMAGE_ACQ_CMD::SINGLE_ACQ_DEFAULT_TYPE);

3.3.86 HBI_DownloadTemplateByType

*****/

* No: No086

* Function Name: HBI_DownloadTemplateByType

* Function Description: Download calibration template by template type

* Parameter Description:

In: void * handle -unsigned handle pointer (unsigned pointer)

int infiletype - Download file types 0-gain template, 1-defect template, 2-offset template, others - Not supported

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:



DT Imaging Technology (Shanghai) Co., Ltd

*****/

HBI_FPD_DLL int __stdcall HBI_DownloadTemplateByType (void * handle, int infiletype);

3.3.87 HBI_GetCurModeList

*****/

* No: No087

* Function Name: HBI_GetCurModeList

* Function Description: Get Mode List

* Parameter Description:

In: void *handle - unsigned handle pointer

* Return Value: FPD_MODE_LIST*

NULL - Failure

Non-NULL - Success, The param of User Acquisition Mode Parameter Configuration

* Comment:

*****/

HBI_FPD_DLL FPD_MODE_LIST * __stdcall HBI_GetCurModeList (void * handle);

3.3.88 HBI_UpdateModeList

*****/

* No: No088

* Function Name: HBI_UpdateModeList

* Function Description: Update Mode List

* Parameter Description:

In: void *handle - unsigned handle pointer

FPD_MODE_LIST* modelist - User Acquisition Mode Parameter Configuration

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_UpdateModeList (void * handle, FPD_MODE_LIST * modelist);

3.3.89 HBI_GetCurModeData

*****/

* No: No089

* Function Name: HBI_GetCurModeData

* Function Description: Obtain the current Mode Data

* Parameter Description:

In: void *handle - unsigned handle pointer

* Return Value: FPD_MODE_DATA*

NULL - Failure

Non-NULL - Success, The param of fpd mode data information

* Comment:

*****/



DT Imaging Technology (Shanghai) Co., Ltd

HBI_FPD_DLL FPD_MODE_DATA * __stdcall HBI_GetCurModeData (void * handle);

3.3.90 HBI_UpdateCurModeData

/******

* No: No090

* Function Name: HBI_UpdateCurModeData

* Function Description: Update current Mode Data

* Parameter Description:

In: void *handle - unsigned handle pointer

FPD_MODE_DATA * mode -fpd mode data information

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_UpdateCurModeData (void * handle, FPD_MODE_DATA * mode);

3.3.91 HBI_UpdateModeData

/******

* No: No091

* Function Name: HBI_UpdateModeData

* Function Description: Update Mode Item Data through Mode ID

* Parameter Description:

In: void *handle - unsigned handle pointer

FPD_MODE_DATA * mode -fpd mode data information

unsigned int id - mode id

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_UpdateModeData (void * handle, FPD_MODE_DATA * mode, unsigned int id=0);

3.3.92 HBI_GetCurModeID

/******

* No: No092

* Function Name: HBI_GetCurModeID

* Function Description: Obtain the current Mode ID

* Parameter Description:

In: void *handle - unsigned handle pointer

* Return Value: int

0: Success

Non-0: Failure

* Comment:



DT Imaging Technology (Shanghai) Co., Ltd

*****/

HBI_FPD_DLL int __stdcall HBI_GetCurModeID (void * handle);

3.3.93 HBI_SwitchUserMode

* No: No093

* Function Name: HBI_SwitchUserMode

* Function Description: Switch Mode item

* Parameter Description:

In: void *handle - unsigned handle pointer

unsigned int modeid - mode id

FPD_MODE_DATA *PMODEDATA - fpd mode data information

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

FPD_DLL int __stdcall HBI_SwitchUserMode (void * handle, unsigned int modeid=0, FPD_MODE_DATA * PMODEDATA=NULL);

3.3.94 HBI_ModeCorrectTempalteEnable

* No: No094

* Function Name: HBI_ModeCorrectTempalteEnable

* Function Description: The calibration template of Mode is effective or not effective

* Parameter Description:

In: void *handle - unsigned handle pointer

unsigned int id - mode id

bool bEnable - 1 - valid, 0-invalid

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_ModeCorrectTempalteEnable (void * handle, unsigned int id, bool bEnable=true);

3.3.95 HBI_AllModeCorrectTempalteEnable

* No: No095

* Function Name: HBI_AllModeCorrectTempalteEnable

* Function Description: All calibration templates for Mode items are valid or not valid

* Parameter Description:

In: void *handle - unsigned handle pointer

bool bEnable - 1 - valid, 0-invalid

* Return Value: int



DT Imaging Technology (Shanghai) Co., Ltd

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_AllModeCorrectTempalteEnable (void * handle, bool bEnable=true);

3.3.96 HBI_SaveModeList2File

*****/

* No: No096

* Function Description: Save the current user mode list

* Parameter Description:

In: void *handle - unsigned handle pointer

char* szPath - file path

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SaveModeList2File (void * handle, char * szPath=NULL);

3.3.97 HBI_DownloadModeTemplate

*****/

* No:No097

* Function Name: HBI_DownloadModeTemplate (new)

* Function description: Download template based on Mode parameter

* Parameter Description:

In: void * handle - Handle (unsigned pointer)

Int intemptype - Download file types 0-gain template, 1-defect template, 2-offset template, others - Not supported

Out: None

* Return value: int

0- Success

Non 0- Failed

* Note: Download template add by mhyang 20230606

*****/

HBI_FPD_DLL int __stdcall HBI_DownloadModeTemplate (void * handle, DOWNLOAD-MODE-FILE downloadfile);

3.3.98 HBI_DownloadModeTemplateByType

*****/

* No:No098

* Function name: HBI_DownloadModeTemplateByType (new)

* Function description: Download correction template files by template type (default to current Mode Id)

* Parameter Description:

In: void * handle - Handle (unsigned pointer)

Int intemptype - Download file types 0-gain template, 1-defect template, 2-offset template, others - Not supported



DT Imaging Technology (Shanghai) Co., Ltd

Out: None

* Return value: int

0- Success

Non 0- Failed

* Note: Download template add by mhyang 20230606

*****/

HBI_FPD_DLL int __stdcall HBI_DownloadModeTemplateByType (void * handle, int intemptype);

3.3.99 HBI_DownloadModeTemplateByIdAndType

*****/

* No:No099

* Function name: HBI_DownloadModeTemplateByIdAndType (new)

* Function description: Download correction template files according to Mode Id and template type

* Parameter Description:

In: void * handle - Handle (unsigned pointer)

Unsigned int modeid - Mode Id [0,9]

Int intemptype - Download file types 0-gain template, 1-defect template, 2-offset template, others - Not supported

Out: None

* Return value: int

0- Success

Non 0- Failed

* Note: Download template add by mhyang 20230605

*****/

HBI_FPD_DLL int __stdcall HBI_DownloadModeTemplateByIdAndType (void * handle, unsigned int modeid, int intemptype);

3.3.100 HBI_GetCurModeTemplateStatus

*****/

* No:No100

* Function Name: HBI_GetCurModeTemplateStatus (new)

* Function Description: Obtain the firmware calibration template status for current Mode item

* Parameter Description:

In: void *handle - unsigned handle pointer

int * nTempStatus - Template status, 0- Dark field bright field invalid, 1- Dark field valid bright field invalid, 2- Dark field invalid bright field valid, 3- Dark field bright field valid, other - Dark field bright field invalid

Out: None

*Return Value: int

0: Successful

Non 0: Failed

*Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetCurModeTemplateStatus (void * handle, int * nTempStatus);

3.3.101 HBI_FirmwareManagementTempalteType

*****/



DT Imaging Technology (Shanghai) Co., Ltd

*No: No101

* Function name: HBI_FirmwareManagementTempalteType (new)

* Function description: Firmware template management template method

* Parameter Description:

In: void * handle - Handle (unsigned pointer)

unsigned int uType - Firmware template management template method

0-by binning type,

1-by Muti-Mode type

others-by binning type

Out: None

* Return value: int

0 - Success

Non 0 - Failed

* Remarks:

*****/

HBI_FPD_DLL int __stdcall HBI_FirmwareManagementTempalteType (void * handle, unsigned int uType);

3.3.102 HBI_SetValidModelItemNumber

*****/

*No: No102

* Function Name: HBI_SetValidModelItemNumber (new)

* Function Description: Set the number of valid Mode items

* Parameter Description:

In: void * handle -unsigned handle pointer

unsigned int uNumber - Number of Mode items, uNumber: [0,10]

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetValidModelItemNumber (void * handle, unsigned int uNumber);

3.3.103 HBI_IsMutilMode

*****/

* No: No103

* Function name: HBI_IsMutilMode (new)

* Function description: Check if the current firmware supports multiple modes

* Parameter Description:

In: void *handle - Handle (unsigned pointer)

* Return value: bool

true - support

false - Not Supported

* Remarks:The essence of Mode mode is that the firmware manages templates differently



DT Imaging Technology (Shanghai) Co., Ltd

*****/

HBI_FPD_DLL bool __stdcall HBI_IsMutilMode (void * handle);

3.3.104 HBI_MUpdateTriggerMode

*****/

* No: No104

* Function name: HBI_MUpdateTriggerMode (new)

* Function description: trigger mode setting

* Parameter Description:

In: void *handle - Handle (unsigned pointer)

int _triggerMode- trigger mode

// 1: Software trigger mode

// 3: High voltage generator trigger mode

// 4: AED trigger mode

// 5-Dynamic:Hvg Sync Mode

// 6-Dynamic:Fpd Sync Mode

// 7-Dynamic:Fpd Continue

// 8-SAEC。

// others-invaild

Out:None

* Return value: int

0 - Success

Non 0 - Failed

* Remarks:

*****/

HBI_FPD_DLL int __stdcall HBI_MUpdateTriggerMode (void * handle, int _triggerMode);

3.3.105 HBI_MSetBinning

*****/

* No: No105

* Function name: HBI_MSetBinning (new)

* Function description: Binning mode setting

* Parameter Description:

In:

Void * handle -unsigned handle pointer

unsigned int ubining -1:1x1,2:2x2,3:3x3,4:4x4

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MSetBinning (void * handle, unsigned int ubining);



DT Imaging Technology (Shanghai) Co., Ltd

3.3.106 HBI_MSetPGAlevel

* No: No106

* Function Name: HBI_MSetPGAlevel (new)

* Function Description: PGA gain stage setting

* Parameter Description:

In: void * handle -unsigned handle pointer (unsigned pointer)

int mode - Mode

[n] - Invalid

[1] -0.6pC

[2] -1.2pC

[3] -2.4pC

[4] -3.6pC

[5] -4.8pC

[6] -7.2pC, default to 7.2pC

[7] -9.6pC

[8] - LFW (COMS)

[9] - HFW (COMS)

[10] -0.3pC

[11] -0.15pC

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MSetPGAlevel (void * handle, int nGainLevel);

3.3.107 HBI_MSetRoiMode

* No: No107

* Function name: HBI_MSetRoiMode (new)

* Function Description: A-silicon ROI region setting function.

* Currently only 1613-125um.3030-140um.4343-100um.2530-100um are supported.

* Parameter Description:

In: void *handle - unsigned handle pointer

int nMode - ROI mode

for example, 1613-125um: 0-Normal Image, w*h(1248*1024), Rect(0,0,1247,1023);

1-w*h(32*1024), Rect(608,0,639,1023);

2-w*h(1248*32), Rect(0,496,1247,527);

3-w*h(1248*64), Rect(0,480,1247,543);

4-w*h(1248*128), Rect(0,448,1247,575);

Others-Normal Image.

3030-140um: 0-Normal Image, w*h(2048*2048), Rect(0,0,2047,2047);



DT Imaging Technology (Shanghai) Co., Ltd

1-w*h(1024*2048),Rect(512,0,1535,2047);

Others-Normal Image。

4343-100um: 0-Normal Image, w*h(4288*4288), Rect(0,0,4287,4287);

1-w*h(4288*800),Rect(0,1743,4287,2544);

Others-Normal Image。

2530-100um: 0-Normal Image, w*h(2432*3072), Rect(0,0,2431,3071);

1-w*h(1808*1800),Rect(316,636,2115,2435);

2-w*h(1312*1300),Rect(566,886,1865,2185);

Others-Normal Image。

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

Out: Null

* Return Value: int

0: Success

Non-0: Failure

* Comment:only 1613-125um.3030-140um.4343-100um.2530-100um are supported。

*****/

HBI_FPD_DLL int __stdcall HBI_MSetRoiMode (void * handle, int nMode, int nfeedback=1);

3.3.108 HBI_MSetZoomModeRect

*****/

* No: No108

* Function name: HBI_MSetZoomModeRect (new)

* Function Description: The ZOOM Mode region setting function for CMOS tablets

* currently supports only CMOS - 1412-50um and CMOS - 0606-50um ,1917-120umtablet types.

* Parameter Description:

In: void *handle - unsigned handle pointer

CMOS_ZOOM_RECT stZoomRect - Zoom region

The region is represented by left, top, right, and bottom.

for example, when left, top, right and bottom are illegal values, the image will display the full image by default,

1>When left=top=right=bottom=0, the whole picture is displayed;

2>When left<0 or left>width, the whole picture will be displayed, and others are the same.

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

Out: Null

* Return Value: int

0: Success

Non-0: Failure

* Comment: currently supports only CMOS - 1412-50um and CMOS - 0606-50um 1917-120um tablet types.

*****/

HBI_FPD_DLL int __stdcall HBI_MSetZoomModeRect (void * handle, CMOS_ZOOM_RECT stZoomRect, int nfeedback=1);

3.3.109 HBI_MSetSelfDumpingTime

*****/



DT Imaging Technology (Shanghai) Co., Ltd

* No: No109

* Function name: HBI_MSetSelfDumpingTime (new)

* Function Description: // Self dumping span time setting

* Parameter Description:

In: void *handle // unsigned handle pointer
int time // Self dumping span time, unit: ms,>0ms
Out: Null

* Return Value: int

// 0: Success
// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MSetSelfDumpingTime (void * handle, int time);

3.3.110 HBI_MSetLiveAcquisitionTime

*****/

* No: No110

* Function name: HBI_MSetLiveAcquisitionTime (new)

* Function Description: Live acquisition span time setting (apply to static and wireless type FPD)

* Parameter Description:

In: void *handle // unsigned handle pointer
int time // Live acquisition span time, unit: ms,>0ms
Out: Null

* Return Value: int

// 0: Success
// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MSetLiveAcquisitionTime (void * handle, int time);

3.3.111 HBI_MSetSinglePrepareTime

*****/

* No: No111

* Function name: HBI_MSetSinglePrepareTime (new)

* Function Description: // Prepare delay time setting, which is also called exposure window

* Parameter Description:

In: void *handle // unsigned handle pointer
Out: int *out_itime // Delay Time, unit: ms
// 0: “HBI_Prepare” and “HBI_SingleAcquisition” command are combined to realize single image acquisition
// Non-0: FPD will delay “Prepare Delay Time” and automatically acquire one single image when the detector receives “HBI_Prepare” command.

* Return Value: int

// 0: Success



DT Imaging Technology (Shanghai) Co., Ltd

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MSetSinglePrepareTime (void * handle, int in_itime);

3.3.112 HBI_MSetFpsOfTime

*****/

* No: No112

* Function name: HBI_MSetFpsOfTime (new)

* Function description: Set frame rate control time, support subtle

* Parameter Description:

In: void *handle - Handle (unsigned pointer)

unsigned int untimems - time interval, Unit is milliseconds, > 0ms

unsigned short ustimeus - time interval, The unit is microseconds, >= 0ms.[0,999]

Out: None

* Return value: int

0 - Success

Non 0 - Failed

* Remarks: eg, fps=16, ntimems=62ms, ntimeus=8um

*****/

HBI_FPD_DLL int __stdcall HBI_MSetFpsOfTime (void * handle, unsigned int untimes, unsigned short ustimeus=0);

3.3.113 HBI_MPgaAcqTime

*****/

* No: No191

* Function Description: // PGA gain stage, frame speed setting

* Parameter Description:

In: void *handle // unsigned handle pointer

int nPgaLevel // PGA gain stage, [n]-Invalid, [1]-0.6pC, [2]-1.2pC, [3]-2.4pC, [4]-3.6pC, [5]-4.8pC, [6]-7.2pC, [7]-9.6pC, [8]-

LFW (COMS), [9]-HFW (COMS), [10]-0.3pC, [11]-0.15pC, Default: 7.2 pC.

int acqtime // Live acquisition span time, unit: ms

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

* Return Value: int

// 0: Success

// Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MPgaAcqTime (void * handle, int nPgaLevel, int acqtime, int nfeedback=0);

3.3.114 HBI_PgaBinningAcqTime

*****/

* No: No114

* Function name: HBI_MPgaBinningAcqTime (new)



DT Imaging Technology (Shanghai) Co., Ltd

* Function Description: // PGA gain stage, binning, frame speed setting

* Parameter Description:

In: void *handle // unsigned handle pointer

int nPgaLevel // PGA gain stage,[n]-Invalid,[1]-0.6pC,[2]-1.2pC,[3]-2.4pC,[4]-3.6pC,[5]-4.8pC,[6]-7.2pC,[7]-9.6pC,[8]-

LFW (COMS),[9]-HFW (COMS),[10]-0.3pC,[11]-0.15pC,Default: 7.2 pC.

unsigned char binning

// 1: 1x1,

// 2: 2x2,

// 3: 3x3,

// 4: 4x4,

int acqtime // Live acquisition span time, unit: ms

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

* Return Value:

int // 0: Success

// Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MPgaBinningAcqTime (void * handle, int nPgaLevel, unsigned char binning, int acqtime, int nfeedback=0);

3.3.115 HBI_MTriggerAndCorrectDisplay

*****/

* No: No115

* Function name: HBI_MTriggerAndCorrectApplay (new)

* Function Description: // Tigger mode and image calibration setting

* Parameter Description:

In: void *handle // unsigned handle pointer

int _triggerMode

// 1: Software trigger mode

// 3: High voltage generator trigger mode

// 4: AED trigger mode

// 5-Dynamic:Hvg Sync Mode

// 6-Dynamic:Fpd Sync Mode

// 7-Dynamic:Fpd Continue

// 8-SAEC。

IMAGE_CORRECT_ENABLE* pCorrect // refer to<< HbiType.h>>

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure,refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MTriggerAndCorrectApplay(void* handle, int _triggerMode, IMAGE_CORRECT_ENABLE*



pCorrect);

3.3.116 HBI_MTriggerBinningAcqTime

/*****

* No: No116

* Function name: HBI_MTriggerBinningAcqTime (new)

* Function Description: // Tigger mode, binning, frame speed setting

* Parameter Description:

In: void *handle // unsigned handle pointer

int triggerMode // Tigger mode

// Static:

// 01: Software trigger mode;

// 02: Clear;

// 03: High voltage generator(hvg) trigger mode;

// 04: Auto exposure detection mode.

// Dynamic:

// 05: Hvg pulse sync in trigger mode;

// 06: Fpd pulse sync out trigger Mode;

// 07: Free run mode.

unsigned char binning

// 1: 1x1,

// 2: 2x2,

// 3: 3x3,

// 4: 4x4,

int time // Live acquisition span time, unit: ms

// For example: 1000ms (1fps), 500ms (2fps), 250ms (4fps), 125ms (8fps)

Out: Null

* Return Value:

int // 0: Success

// Non-0: Failure

* Comment:

*****/

```
HBI_FPD_DLL int __stdcall HBI_MTriggerBinningAcqTime (void * handle, int triggerMode, unsigned char binning, int acqtime, int
nfeedback=0);
```

3.3.117 HBI_MTriggerPgaBinningAcqTime

/*****

* No: No117

* Function name: HBI_MTriggerPgaBinningAcqTime (new)

* Function description: trigger mode, PGA gain stage, binning type, frame speed setting

* Parameter Description:

In: void *handle - Handle (unsigned pointer)

int triggerMode // Tigger mode

// Static:



DT Imaging Technology (Shanghai) Co., Ltd

```

// 01: Software trigger mode;
// 02: Clear;
// 03: High voltage generator(hvg) trigger mode;
// 04: Auto exposure detection mode.
// Dynamic:
// 05: Hvg pulse sync in trigger mode;
// 06: Fpd pulse sync out trigger Mode;
// 07: Free run mode.

int nPgaLevel    // PGA gain stage,[n]-Invalid,[1]-0.6pC,[2]-1.2pC,[3]-2.4pC,[4]-3.6pC,[5]-4.8pC,[6]-7.2pC,[7]-9.6pC,[8]-
LFW (COMS),[9]-HFW (COMS),[10]-0.3pC,[11]-0.15pC,Default: 7.2 pC.

unsigned char binning
// 1: 1x1,
// 2: 2x2,
// 3: 3x3,
// 4: 4x4,

int acqtime      // Live acquisition span time, unit: ms
int nfeedback    // 1: Automatic feedback config parameters if setting is successful
// Non-1: Feedback setting success status

* Return value: int
0 - Success
Non 0 - Failed

* Comment:
*****/

HBI_FPD_DLL int __stdcall HBI_MTriggerPgaBinningAcqTime (void * handle, int triggerMode, int nPgaLevel, unsigned char binning,
int acqtime, int nfeedback=0);

```

3.3.118 HBI_MTriggerPgaBinningAcqTimeCorrect

/*****/

* No: No118

* Function name: HBI_MTriggerPgaBinningAcqTimeCorrect (new)

* Function description: trigger mode, PGA gain stage, binning type, frame speed, correction enabling setting

* Parameter Description:

```

In: void *handle - Handle (unsigned pointer)

int triggerMode // Tigger mode
// Static:
// 01: Software trigger mode;
// 02: Clear;
// 03: High voltage generator(hvg) trigger mode;
// 04: Auto exposure detection mode.
// Dynamic:
// 05: Hvg pulse sync in trigger mode;
// 06: Fpd pulse sync out trigger Mode;
// 07: Free run mode.

int nPgaLevel    // PGA gain stage

```



DT Imaging Technology (Shanghai) Co., Ltd

//[n]-Invalid,[1]-0.6pC,[2]-1.2pC,[3]-2.4pC,[4]-3.6pC,[5]-4.8pC,[6]-7.2pC,[7]-

9.6pC

//[8]-LFW (COMS),[9]-HFW (COMS),[10]-0.3pC,[11]-0.15pC,Default: 7.2 pC.

unsigned char binning

// 1: 1x1,

// 2: 2x2,

// 3: 3x3,

// 4: 4x4,

int acqtime // Live acquisition span time, unit: ms

IMAGE_CORRECT_ENABLE* pCorrect, refer to <HbiError.h>

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

* Return value: int

0 - Success

Non 0 - Failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MTriggerPgaBinningAcqTimeCorrect (void * handle, int triggerMode, int nPgaLevel, unsigned char binning, int acqtime, IMAGE_CORRECT_ENABLE * pCorrect);

3.3.119 HBI_GetModeData

*****/

* No: No119

* Function Name: HBI_GetModeData

* Function Name: HBI_GetModeData

* Function Description: Obtain Mode Data through Mode ID

* Parameter Description:

In: void *handle - unsigned handle pointer

unsigned int id - mode id

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL FPD_MODE_DATA * __stdcall HBI_GetModeData (void * handle, unsigned int id=0);

3.3.120 HBI_GetModeListSize

*****/

* No: No120

* Function Name: HBI_GetModeListSize

* Function Description: Obtain the number of items for Mode

* Parameter Description:

In: void *handle - unsigned handle pointer

* Return Value: int



DT Imaging Technology (Shanghai) Co., Ltd

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL unsigned int __stdcall HBI_GetModeListSize (void * handle);

3.3.121 HBI_ModeExists

*****/

* No: No121

* Function Name: HBI_ModeExists

* Function Description: Determine whether the mode exists

* Parameter Description:

In: void *handle - unsigned handle pointer

FPD_MODE_DATA mode - fpd mode data information

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_ModeExists (void * handle, FPD_MODE_DATA mode);

3.3.122 HBI_CheckCurtMode

*****/

* No: No122

* Function Name: HBI_CheckCurtMode

* Function Description: Judge whether the mode is normal

* Parameter Description:

In: void *handle - unsigned handle pointer

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL bool __stdcall HBI_CheckCurtMode (void * handle);

3.3.123 HBI_InitDetectorParams

*****/

* No: No123

* Function name: HBI_InitDetectorParams (new)

* Function description: initialize the basic parameters of the flat panel detector

* Parameter Description:

In: void *handle - Handle (unsigned pointer)

DETECTOR_INIT_PARAMS param -refer to 《HbiType.h》

param._ucmodeid; //mode id [0,9];



DT Imaging Technology (Shanghai) Co., Ltd

param._uctriggerType; //1-soft trigger, 3-hvg trigger,4-FreeAED,5-Dynamic:Hvg Sync Mode,6-Dynamic:Fpd Sync Mode,7-Dynamic:Fpd Continue,8-SAEC;

//Only valid in Binning mode,invalid in multi-mode mode,purpose compatible with different version.

param._ucOffsetCorrection;//0-No Offset Correction;1-Software PreOffset Correction;2-Firmware PostOffset Correction;3-Firmware PreOffset Correction;

param._ucGainCorrection; //0-No Gain Correction;1-Software Gain Correction;2-Firmware Gain Correction;

param._ucDefectCorrection;//0-No Defect Correction;1-Software Defect Correction;2-Firmware Defect Correction;

param._ucDummyCorrection; //0-No Dummy Correction;1-Software Dummy Correction;2-Firmware Dummy Correction;

Out: None

* Return value: int

0 - Success

Non 0 - Failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_InitDetectorParams (void * handle, DETECTOR_INIT_PARAMS param);

3.3.124 HBI_SetWindowMode

*****/

* No: No209

* Function Name: HBI_SetWindowMode

* Function Description:

* Parameter Description:

In: void *handle - unsigned handle pointer

bool isWindowMode - true:enable,false-disable

Out:None

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetWindowMode(void* handle, bool isWindowMode);

3.3.125 HBI_GetWindowMode

*****/

* No: No210

* Function Name: HBI_GetWindowMode

* Function Description:

* Parameter Description:

In: void *handle - unsigned handle pointer

In/Out bool *isWindowMode - true:enable,false-disable

Out:None

* Return Value: int

0: Success

Non-0: Failure



DT Imaging Technology (Shanghai) Co., Ltd

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetWindowMode(void* handle, bool* isWindowMode);

3.3.126 HBI_SetBinningDivisor

* No: No211

* Function Name: HBI_SetBinningDivisor

* Function Description:

* Parameter Description:

In: void *handle - unsigned handle pointer

unsigned char binDiv,[0,1,2,4,8,16]

Out:None

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetBinningDivisor(void* handle, unsigned char binDiv);

3.3.127 HBI_GetBinningDivisor

* No: No212

* Function Name: HBI_GetBinningDivisor

* Function Description:

* Parameter Description:

In: void *handle - unsigned handle pointer

In/Out unsigned char* binDiv,[0,1,2,4,8,16]

Out:None

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetBinningDivisor(void* handle, unsigned char* binDiv);

3.4 Callback Event

At present, the interaction between the dynamic library and firmware data is asynchronous. The tablet status, parameters, image data, and monitoring information are obtained through callback functions defined in the upper computer. The prerequisite is that the upper computer must register a main callback function and pass the address of the callback function to the dynamic library through the interface.

Including the main callback function and the progress callback function.

◆ Main callback function



DT Imaging Technology (Shanghai) Co., Ltd

The status of the tablet, tablet parameters, image data, stacking number, main thread status, etc. are fed back to the upper computer through the main callback function. According to the integration process, the main callback function must be registered.

Callback function type definition:

//Notice: call back function

//@ USER_CALLBACK_HANDLE_ENVENT

//@ byteEventid: enum eEventCallbackCommType

//@ ufpdId: Tablet device ID

//@ PVEventParam1: fpd config or image data buffer addr

//@ nEventParam2: Parameter 2, such as data size or status

//@ nEventParam3: Parameter 3, such as frame rate or state, etc

//@ nEventParam4: Parameter 4, such as PCIe event ID or reserved extension

Typedef int (* USER_CALLBACK_HANDLE_ENVENT) (void * pContext, int ufpdId, unsigned char byteEventId, void * PVEventParam1, int nEventParam2, int nEventParam3, int nEventParam4);

Function Description: Callback function definition, handling configuration, images, and tablet return status.

Parameter Description:

PContext: Parameter 1, upper computer object pointer, can be null

UfpdId: Parameter 2, such as tablet ID

ByteEventid: Parameter 3, Event ID, refer to enum eCallbackEventCommType in HbiType. h

PvParam1: Parameter 4, configure structure pointer or image structure pointer

NParam2: Parameter 5, such as data size (image data is the length of the image buffer) or state

Param3: Parameter 6, such as frame number, frame ID

Param4: Parameter 7, such as frame rate, frame rate, or state, etc

◆ Download progress and result callback function

Callback function type definition:

//Notice: call back function

//@ USER_CALLBACK_HANDLE_PROCESS

//@ cmd: enum eAnswerCallbackCommType

//@ buff

//@ len

Typedef int (* USER-CALLBACK_HANDLE-PROCESS) (unsigned char cmd, int retcode, void * inContext);

Function description: Callback function definition, handling firmware update message feedback.

Parameter Description:

cmd message, command message returned from SDK.

retcode return code

inContext Object pointer, usually a class object pointer

◆ Callback event type

Below are the main types of callback events and their information:

Event ID	value	Remarks
ECALLBACK_TYPE_FPD_STATUS	0X09	Tablet status
ECALLBACK_TYPE_ROM_UPLOAD	0X10	Plate ROM parameters
ECALLBACK_TYPE_SET_CFG_OK	0XAB	Setting parameters successfully (usually returns ROM parameters) can also return a success message.
ECALLBACK_TYPE_SINGLE_IMAGE	0X51	Single frame image acquisition

ECALLBACK_TYPE_MULTIPLE_IMAGE	0X52	Continuous image acquisition
ECALLBACK_TYPE_OFFSET_TEMPLATE	0X5C	Firmware completed pre offset upload offset template data
ECALLBACK_TYPE_PREVIEW_IMAGE	0X53	Upload preview images in Preview mode
ECALLBACK_OVERLAY_NUMBER	0X5D	Image overlay, feedback overlay process, that is, overlay to which frame
ECALLBACK_OVERLAY_16BIT_IMAGE	0X5E	The overlay result is 16 bit image data
ECALLBACK_OVERLAY_32BIT_IMAGE	0X5F	The overlay result is 32-bit image data
ECALLBACK_TYPE_PACKET_MISS	0X5B	Packet loss/frame loss feedback information
ECALLBACK_TYPE_LIVE_ACQ_OK	0XA0	Feedback information on saving image files: including directory, average grayscale value
ECALLBACK_TYPE_THREAD_EVENT	0XA5	Monitoring thread or application thread monitoring information
ECALLBACK_TYPE_PACKET_MISS_MSG	0XA4	Discovering packet loss in image collection and providing feedback on packet loss information
ECALLBACK_TYPE_ACQ_DISCARD	0XA6	Continuous frame throwing during collection, SDK feedback frame throwing record
ECALLBACK_TYPE_OFFSET_ERR_MSG	0XA7	Generate offset template results for distribution generation
ECALLBACK_TYPE_GAIN_ERR_MSG	0XA8	Distribution generated gain template results
ECALLBACK_TYPE_DEFECT_ERR_MSG	0XA9	Generate defect template results based on distribution
ECALLBACK_TYPE_UPDATE_FIRMWARE	0X06	Firmware updates: progress and results
ECALLBACK_TYPE_ERASE_FIRMWARE	0X07	Firmware update: erasing firmware
ECALLBACK_TYPE_WORK_STATUS	0XAF	Working mode: Feedback on switching between production mode and debugging mode
ECALLBACK_TYPE_SAVE_SUCCESS	0XAC	Saving image feedback during common template generation processes
ECALLBACK_TYPE_GENERATE_TEMPLATE	0XAD	Quickly generate template information feedback, pay attention to the difference between step-by-step generation and step-by-step generation
ECALLBACK_TYPE_FILE_NOTEXIST	0XAE	No feedback found when searching for files

Example: The status of flat panel detector (ECALLBACK_TYPE_FPD_STATUS)

state	value	Remarks
FPD_DISCONN_STATUS	0	Callback received -11 "nEventParam2" and 0 are all considered communication exceptions. Please refer to the SDK Demo project for details
FPD_PREPARE_STATUS	1	Single frame acquisition, clearing
FPD_READY_STATUS	2	Static tablet ready state, tablet idle state, can be sampled
FPD_DOOFFSET_TEMPLATE	3	In the firmware generation pre offset template
FPD_EXPOSE_STATUS	4	In the sampling process
FPD_CONTINUE_READY	5	Dynamic tablet ready status, tablet idle status, can be sampled
FPD_DWONLOAD_GAIN	6	Download Gain template
FPD_DWONLOAD_DEFECT	7	Download the Defect template
FPD_DWONLOAD_OFFSET	8	Download the Offset template
FPD_UPDATE_FIRMARE	9	Updating firmware
FPD_RETRANS_MISS	10	Retransmitting data
FPD_STATUS_AED	11	In AED mapping
FPD_STATUS_SLEEP	12	Low power consumption of wireless tablets: sleep mode
FPD_STATUS_WAKEUP	13	Low power consumption of wireless tablet: wake-up
FPD_DOWNLOAD_NO_IMAGE	14	Wireless tablet downloading images: No images available for download
FPD_DOWNLOAD_TAIL_IMAGE	15	Wireless tablet download image: Download the last frame image



DT Imaging Technology (Shanghai) Co., Ltd

FPD_EMMC_MAX_NUMBER	16	Emmc storage capacity alarm
FPD_ENDTIME_WARNNING	17	Firmware authorization ended
FPD_CONN_SUCCESS	100	Connection successful

The callback function returns:

case ECALLBACK_TYPE_ROM_UPLOAD://Update configuration

.....

case ECALLBACK_TYPE_SINGLE_IMAGE://Single frame capture image

.....

case ECALLBACK_TYPE_MULTIPLE_IMAGE://Continuous collection of images

.....

case ECALLBACK_TYPE_FPD_STATUS://Tablet status: connected/disconnected/ready/busy

printf("ECALLBACK TYPE FPD STATUS, recode=%d \n", nlength);

if (TheDemo!=NULL)

{

CString strMsg=_T ();

if (nlength<=0&& nlength>=-11)

{

If (nlength==0)

printf("ECALLBACK_TYPE_FPD_STATUS, Err: Network not connected! \n");

else if (nlength=-1)

printf("ECALLBACK_TYPE_FPD_STATUS, Err: parameter exception! \n");

else if (nlength=-2)

printf("ECALLBACK TYPE FPD_STATUS, Err: Failed to return number of prepared descriptors! \n");

else if (nlength=-3)

printf("ECALLBACK TYPE FPD STATUS, Err: Receive timeout! \n");

else if (nlength=-4)

printf("ECALLBACK TYPE FPD STATUS, Err: Receive failed! \n");

else if (nlength=-5)

printf("ECALLBACK TYPE FPD STATUS, Err: Port unreadable! \n");

else if (nlength=-6)

printf("ECALLBACK TYPE FPD STATUS, network card unusual! \n");

else if (nlength=-7)

printf("ECALLBACK TYPE FPD STATUS, network card ok! \n");

else if (nlength=-8)

printf("ECALLBACK TYPE FPD STATUS: update Firmware end! \n");

else if (nlength=-9)

printf("ECALLBACK TYPE FPD STATUS: Fiber optic disconnected!! \n");

else if (nlength=-10)

printf("ECALLBACK TYPE FPD STATUS: read DDR failed, try restarting the PCIe driver!");

else/* if (nlength=-11)*/

printf("ECALLBACK TYPE FPD STATUS: ECALLBACK TYPE FPD STATUS: is not jump! \n");

Status=(int) FPD_DISCONN_STATUS;

}

else if (nlength==FPD_CONN_SUCCESS) { //connect

```

DT Imaging Technology (Shanghai) Co., Ltd
printf("ECALLBACK TYPE FPD STATUS, start listening! \n");
Status=(int) FPD_CONN_SUCCESS;
}
else if (nlength==FPD_REPARE_STATUS) { //ready
printf("ECALLBACK TYPE FPD STATUS, ready! \n");
Status=(int) FPD_PREPARE_STATUS;
}
else if (nlength==FPD_READY-STATUS) { //busy
printf("ECALLBACK TYPE FPD STATUS, busy! \n");
Status=(int) FPD_READY_STATUS;
}
else if (nlength==FPP_DOFFSET-TEMPLATE) { //prepare
printf("ECALLBACK TYPE FPD STATUS, prepare! \n");
Status=(int) FPD_DDOFFSET_TEMPLATE;
}
else if (nlength==FPD_EXPOSE_STATUS) { //busy expose
printf("ECALLBACK TYPE FPD STATUS: Exposing! \n");
Status=FPD_EXPOSE_STATUS;
}
else if (nlength==FPD_CONTINUE_READY) { //continue ready
printf("ECALLBACK TYPE FPD STATUS: Continue ready! \n");
Status=FPD_CONTINUE_READY;
}
else if (nlength==FPD_DWONLOAD_GAIN) { //download gain template
printf("ECALLBACK TYPE FPD STATUS: Download gain template ack! \n");
Status=FPD_DWONLOAD_GAIN;
}
else if (nlength==FPD_DWONLOAD-DEFECT) { //download defect template
printf("ECALLBACK TYPE FPD STATUS: Download defect template ack! \n");
Status=FPD_DWONLOAD_DEFECT;
}
else if (nlength==FPD_DWONLOAD_OFFSET) { //download offset template
printf("ECALLBACK TYPE FPD STATUS: Download offset template ack! \n");
Status=FPD_DWONLOAD_OFFSET;
}
else if (nlength==FPD_UPDATE_FIRMWARE) { //update firmware
printf("ECALLBACK TYPE FPD STATUS: Update firmware! \n");
Status=FPD_UPDATE_FIRMWARE;
}
else if (nlength==FPD_RETRANS_MISS) { //update firmware
printf("ECALLBACK TYPE FPD STATUS: Retransmission! \n");
Status=FPD_RETRANS_MISS;
}
else

```



DT Imaging Technology (Shanghai) Co., Ltd

```
printf("ECALLBACK TYPE FPD STATUS, Err: Other error=%d \n", nlength);
```

```
//ADD BY MH.YANG 2019/11/12
```

```
if (status!=-1)
```

```
{
```

```
    // update icon
```

```
    TheDemo ->PostMessage (WM_DETECTORA_CONNECT_STATUS, (WPARAM) status, (LPARAM) 0);
```

```
    // disconnect
```

```
    if (nlength<=0&& nlength>=-10)
```

```
    {
```

```
        CFPD_BASE_CFG * pBaseCfg=TheDemo ->get_Base_Cfg_Ptr (0);
```

```
        if (pBaseCfg!=NULL)
```

```
        {
```

```
            if (pBaseCfg ->m_pFpdHand!=NULL)
```

```
            {
```

```
                HBI_DisconnectDetector (TheDemo ->m_pFPD);
```

```
                TheDemo ->UpdateUI();
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
}
```

```
break;
```

// 注意回调函数返回值

//If there is no return value or if the return value is 0 or defaults to 0, the SDK will consider feedback failed. Therefore, if the upper computer receives a callback message from the SDK, it will return 1

Return 1;

Number	Callback Events	constant	information
1	ECALLBACK_TYPE_FPD_STATUS	0x09	Tablet state (nParam2), including normal state (>0) and abnormal state (<=0). 0: Network port disconnected (such as disconnection command); -1: Received abnormal data, resulting in disconnection; -2: Failed to return the number of ready descriptors -3: Receive wait timeout -4: Abnormal reception -5: Port unreadable -6: Network card abnormality -7: Not in use temporarily; -8: Not in use temporarily; -9: Tablet disconnect -10: Failed to read DDR address (optical communication); -11: Non giant frame (abnormal communication method, mismatch with firmware); 1: Prepare status, single frame acquisition, issue prepare command, and the tablet provides feedback on this status; 2: Ready state, single frame acquisition completed or tablet idle state;

			3: In the firmware generation pre offset template; 4: Busy state, the tablet is currently being sampled; 5: Continue ready status, continuous collection completed; 6: Upload the gain template; 7: Upload the defect template; 8: Upload the offset template; 9: Updating firmware; 10: Indicates that the dynamic library automatically initiates packet loss retransmission commands; 11: AED triggered incorrectly, invalid data, temporarily not enabled; 12: Wireless tablet energy management: The tablet receives sleep command feedback; 13: Wireless tablet energy management: The tablet receives wake-up command feedback; 14: Wireless tablet download image: no image feedback; 15: Wireless tablet download image: last frame image feedback; 16: Wireless tablet downloading images: feedback on the maximum number of stored images; 17: RTC, firmware authorization deadline feedback; 18: Wireless tablet batteries and feedback on wireless mode and bandwidth information; 19: Feedback on successful reconnection at the internet interface; 20: Confirm that the image data of this frame is normal (generate a template in a single step); 21: Wrong tablet model, please check 100: Connecting achievements
2	ECALLBACK_TYPE_ROM_UPLOAD	0x10	Firmware feedback tablet ROM parameters (basic control parameters of the tablet)
3	ECALLBACK_TYPE_FACTORY_UPLOAD	0x12	Firmware deployment configuration parameters
4	ECALLBACK_TYPE_SET_CFG_OK	0xab	Parameter setting successful (default feedback firmware parameters, interface function set feedback parameter to 0, then only receive confirmation of successful setting event)
5	ECALLBACK_TYPE_SINGLE_IMAGE	0x51	Single frame image data
6	ECALLBACK_TYPE_MULTIPLE_IMAGE	0x52	Continuous image data
7	ECALLBACK_TYPE_PREVIEW_IMAGE	0x53	Thumbnail data
8	ECALLBACK_OVERLAY_NUMBER	0x5d	Overlay Number
9	ECALLBACK_OVERLAY_16BIT_IMAGE	0x5e	Overlay result graph (16 bit graph)
10	ECALLBACK_OVERLAY_32BIT_IMAGE	0x5f	Overlay result chart (32bit chart)
11	ECALLBACK_TYPE_PACKET_MISS	0x5b	Packet loss retransmission data
12	ECALLBACK_TYPE_THREAD_EVENT	0xa5	Thread state
13	ECALLBACK_TYPE_ACQUISITION_DISCARDED	0xa6	Continuous image acquisition and frame throwing data information
14	ECALLBACK_TYPE_UPDATE_FIRMWARE	0x06	Update Firmware
15	ECALLBACK_TYPE_ERASE_FIRMWARE	0x07	Erase firmware
16	ECALLBACK_TYPE_FILE_NOTEXIST	0xae	file does not exist
17	ECALLBACK_TYPE_WLAN_BATTERY	0x13	Wireless tablet: battery, wireless mode, wireless bandwidth, and other information

3.5 Return Code

Please refer to the attachment "HbiError. h" for details on returning the code table.

Number	Macro Definition	constant	information
1	HBI_ERR_INVALID_PARAMS	8000	initialization failed
2	HBI_ERR_OPEN_DETECTOR_FAILED	8001	Failed to open detector
3	HBI_ERR_INVALID_PARAMS	8002	Invalid parameter
4	HBI_ERR_CONNECT_FAILED	8003	Connection failed
5	HBI_ERR_MALLOC_FAILED	8004	Memory allocation failed
6	HBI_ERR_RELIMGMEM_FAILED	8005	Release image resources
7	HBI_ERR_RETIMGMEM_FAILED	8006	Failed to return image buffer address
8	HBI_ERR_NODEVICE	8007	Device not found
9	HBI_ERR_DISCONNECT	8008	Unconnected
10	HBI_ERR_DEVICE_BUSY	8009	In the detector acquisition map
11	HBI_ERR_SENDDATA_FAILED	8010	Data sending failed
12	HBI_ERR_RECEIVE_DATA_FAILED	8011	Receiving data failed
13	HBI_ERR_COMMAND_MISMATCH	8012	Command mismatch
14	HBI_ERR_NO_IMAGE_RAW	8013	No image data
15	HBI_ERR_PTHREAD_ACTIVE_FAILED	8014	Failed to start listening thread
16	HBI_ERR_STOP_ACQUISITION	8015	Stop collection failed
17	HBI_ERR_INSERT_FAILED	8016	Insert data failed
18	HBI_ERR_GET_CFG_FAILED	8017	Failed to obtain tablet parameters
19	HBI_NOT_SUPPORT	8018	Not currently supported
20	HBI_REGISTER_CALLBACK_FAILED	8019	Registration callback function failed
21	HBI_SEND_MESSAGE_FAILED	8021	Sending message failed
22	HBI_ERR_WORKMODE	8022	Switching test mode failed
23	HBI_FAILED	8023	fail
24	HBI_FILE_NOT_EXISTS	8024	file does not exist
25	HBI_COMM_TYPE_ERR	8025	Command type error
26	HBI_TYPE_IS_NOT_EXISTS	8026	This type does not exist
27	HBI_SAVE_FILE_FAILED	8027	Save image failed
28	HBI_INIT_PARAM_FAILED	8028	Failed to initialize parameters
29	HBI_INIT_FILE_NOT_EXIST	8029	The initialization file does not exist
30	HBI_INVALID_FLAT_PANEL	8030	Invalid tablet type
31	HBI_INVALID_DLL_HANDLE	8031	Invalid handle object
32	HBI_FPD_IS_DISCONNECT	8032	Tablet not connected
33	HBI_ERR_DETECTOR_NUMBER	8033	The ID of the detector is incorrect
34	HBI_ERR_DATA_CHECK_FAILED	8034	Data verification failed
35	HBI_ERR_CFG_IEMPTY	8035	The configuration parameter object is empty
36	HBI_ERR_OPEN_WIZARD_FAILED	8036	Failed to open template wizard
37	HBI_ERR_WIZARD_ALREADY_EXIST	8037	Template Wizard already exists
38	HBI_DIRECTORY_NOT_EXISTS	8038	The directory does not exist
39	HBI_UNREGISTER_CALLBACK	8039	Unregistered callback function
40	HBI_ERR_IMAGE_IEMPTY	8040	EMMC no image data (wireless)
41	HBI_ERR_ABNORMMAL_IMAGE	8041	Abnormal image data
42	HBI_PREPARATION_COMPLETED	8042	Preparation completed
43	HBI_ERR_FILE_EXCEPTION	8043	File data exception, such as size mismatch or empty
44	HBI_ERR_COMM_TYPE	8044	Wrong communication type
45	HBI_ERR_INVALID_MODE	8045	Invalid Mode entry

46	HBI_ERR_FIRM_CONFIG_DATA	8046	The firmware parameters contain incorrect data
47	HBI_ERR_IS_NULL	8047	Pointer is empty
48	HBI_ERR_ROM_PARAMETER	8048	Incorrect ROM parameters
49	HBI_ERR_FILE_TYPE	8049	File type error
50	HBI_END	8050	Main monitoring thread exits

Error Message List:

Struct CodeStringTable

```
{
    int num;
    int HBIRETCODE;
    Const char * errString;
};
```

Static const CodeStringTable CrErrStrList{

```
{0, HBI_SUCCSS, "Success"},
{1, HBI-ERR-ININTFAILED, "Init DLL failed"},
{2, HBI-ERR-OPEN-DETECTOR-FAILED, "Open device driver failed"},
{3, HBI-ERR-INVALID6ARAMS, "Parameter error"},
{4, HBI-ERR-CONNECT-FAILED, "Connect failed"},
{5, HBI-ERR-MALLOC-FAILED, "Malloc memory failed"},
{6, HBI-ERR-RELIMGMEMFAILED, "Releaseimagemem fail"},
{7, HBI-ERR-RETIMGMEMFAILED, "ReturnImageMemo fail"},
{8, HBI-ERR-NODEVICE, "No Init DLL Instance"},
{9, HBI-ERR-DISCONNECT, "Is Disconnect"},
{10, HBI-ERR-DEVICE-BUSY, "FPD is busy"},
{11, HBI-ERR-SENDDATA-FAILED, "SendData failed"},
{12, HBI-ERR-RECEIVE-DATA-FAILED, "Receive Data failed"},
{13, HBI-ERR-COMMAND-DISMATCH, "Command mismatch"},
{14, HBI-ERR-NO-IMAGE-RAW, "No Image raw"},
{15, HBI-ERR-PTHREAD-ACTIVE-FAILED, "Pthread active failed"},
{16, HBI-ERR-STOP-ACQUISITION, "Pthread stop data acquisition failed"},
{17, HBI-ERR-INSERT-FAILED, "insert calibrate mode failed"},
{18, HBI-ERR_Get-CFG-FAILED, "get Fcd config failed"},
{19, HBI-NOT-SUPPORT, "not sharp yet"},
{20, HBI-REGISTER-CALLBACK-FAILED, "failed to register callback function"},
{21, HBI_SEND-MESSAGE-FAILED, "send message failed"},
{22, HBI-ERR-WORKMODE, "switch work mode failed"},
{23, HBI-FAILED, "operation failed"},
{24, HBI-FILE-NOT-EXISTS, "file does not exist"},
{25, HBI-COMM-TYPE.ERR, "communication is not existing"},
{26, HBI_TYPE_IS_NOT_EXISTS, "this type is not exists"},
{27, HBI-SAVE-FILE-FAILED, "save file failed"},
{28, HBI-INIT-PARAM-FAILED, "Init dll param failed"},
{29, HBI-INIT-FILE-NOT-EXIST, "Init dll config file is not existing"},
{30, HBI-INVALID-FLAT-PANEL, "Invalid flat panel detector"},
{31, HBI-INVALID-DLL_HANDLE, "Invalid DLL instance handle"},
```

```
{32, HBI-FPD-IS-DISCONNECT, "FPD is disconnected"},
{33, HBI-ERR-DETECTOR-NUMBER, "Detector number error"},
{34, HBI-ERR-DATA-CHECK-FAILED, "Data Check failed"},
{ 35, HBI_ERR_CFG_IEMPTY, "The configuration variable is empty"},
{36, HBI-ERR-OPEN-WIZARD-FAILED, "Open template wizard failed"},
{37, HBI-ERR-WIZARD-ALREADY-EXIST, "template wizard already exists"},
{38, HBI-DIRECTORY-NOT-EXISTS, "File directory is not existing"},
{39, HBI_UNREGISTER-CALLBACK, "Unregistered callback function"},
{40, HBI-ERR-IMAGE-IEMPTY, "Image is empty"},
{41, HBI-ERR-ABNORMMAL-IMAGE, "Abnormal image"},
{42, HBI-PREPARATION-COMPLETE, "Preparation complete"},
{43, HBI-ERR-FILE-EXCEPTION, "File exception"},
{44, HBI-ERR-COMM-TYPE, "Wrong communication type"},
{ 45, HBI_ERR_INVALID_MODE, "This mode contains incorrect parameters"},
{46, HBI-ERR-FIRM-CONFIG-DATA, "Firmware parameter data is abnormal"},
{47, HBI-ERR-IS-NULL, "The pointer is NULL"},
{48, HBI-ERR-ROM-PARAMETER, "ROM includes correct parameters"},
{49, HBI-ERR-FILE-TYPE, "File type exception"},
{50, HBI-END, "Exit monitoring"};};
```

Please refer to the attachment "HbiError. h" for details.

/*****

* Description: Get the error information by return code

* Function: GetHbiErrorString

* Parameter Descriptions: CodeStringTable* inTable // Error code table

* int count // Return error code number

* int recode // Return error code

* Return Value: const char * // Return string

* Remarks: BY MH.YANG 2019/12/09

*****/

const char * GetErrString (const CodeStringTable * inTable, int count, int recode)

```
{
    const char * retString="Unknown";
    for (int i=0; i<count; i++){
        if (inTable [i]. HBIRETCODE==recode){
            retString=inTable [i]. errString;
            break;
        }
    }
    return retString;
}
```

int main()

```
{
    static const CodeStringTable CrErrStrList[] = {
```

```
{ 0, HBI_SUCCSS, "Success" },
{ 1, HBI_ERR_ININT_FAILED, "Init dll failed" },
{ 2, HBI_ERR_OPEN_DETECTOR_FAILED, "Open device driver failed" },
{ 3, HBI_ERR_INVALID_PARAMS, "Parameter error" },
{ 4, HBI_ERR_CONNECT_FAILED, "Connect failed" },
{ 5, HBI_ERR_MALLOC_FAILED, "Malloc memory failed" },
{ 6, HBI_ERR_RELIMGMEM_FAILED, "Release image memory failed" },
{ 7, HBI_ERR_RETIMGMEM_FAILED, "Return Image memory failed" },
{ 8, HBI_ERR_NODEVICE, "No Init DLL Instance" },
{ 9, HBI_ERR_DISCONNECT, "Is Disconnect" },
{ 10, HBI_ERR_DEVICE_BUSY, "Fpd is busy" },
{ 11, HBI_ERR_SENDDATA_FAILED, "SendData failed" },
{ 12, HBI_ERR_RECEIVE_DATA_FAILED, "Receive Data failed" },
{ 13, HBI_ERR_COMMAND_DISMATCH, "Command mismatch" },
{ 14, HBI_ERR_NO_IMAGE_RAW, "No Image raw" },
{ 15, HBI_ERR_PTHREAD_ACTIVE_FAILED, "Pthread active failed" },
{ 16, HBI_ERR_STOP_ACQUISITION, "Pthread stop data acquisition failed" },
{ 17, HBI_ERR_INSERT_FAILED, "insert calibrate mode failed" },
{ 18, HBI_ERR_GET_CFG_FAILED, "get Fpd config failed" },
{ 19, HBI_NOT_SUPPORT, "not surport yet" },
{ 20, HBI_REGISTER_CALLBACK_FAILED, "failed to register callback function" },
{ 21, HBI_SEND_MESSAGE_FAILED, "send message failed" },
{ 22, HBI_ERR_WORKMODE, "switch work mode failed" },
{ 23, HBI_FAILED, "operation failed" },
{ 24, HBI_FILE_NOT_EXISTS, "file does not exist" },
{ 25, HBI_COMM_TYPE_ERR, "communication is not exist"},
{ 26, HBI_TYPE_IS_NOT_EXISTS, "this type is not exists"},
{ 27, HBI_SAVE_FILE_FAILED, "save file failed"},
{ 28, HBI_INIT_PARAM_FAILED, "Init dll param failed"},
{ 29, HBI_INIT_FILE_NOT_EXIST, "Init dll config file is not exist"},
{ 30, HBI_INVALID_FLAT_PANEL, "Invalid flat panel detector"},
{ 31, HBI_INVALID_DLL_HANDLE, "Invalid dll instance handle"},
{ 32, HBI_FPD_IS_DISCONNECT, "Fpd is disconnect"},
{ 33, HBI_ERR_DETECTOR_NUMBER, "Detector number error"},
{ 34, HBI_ERR_DATA_CHECK_FAILED, "Data Check failed"},
{ 35, HBI_ERR_CFG_IEMPTY, "The configuration variable is empty"},
{ 36, HBI_ERR_OPEN_WIZARD_FAILED, "Open template wizard failed"},
{ 37, HBI_ERR_WIZARD_ALREADY_EXIST, "template wizard already exist"},
{ 38, HBI_DIRECTORY_NOT_EXISTS, "File directory is not exists"},
{ 39, HBI_UNREGISTER_CALLBACK, "Unregistered callbacke function"},
{ 40, HBI_ERR_IMAGE_IEMPTY, "Image is empty"},
{ 41, HBI_ERR_ABNORMMAL_IMAGE, "Abnormal image"},
{ 42, HBI_PREPARATION_COMPLETE, "Preparation complete"},
{ 43, HBI_ERR_FILE_EXCEPTION, "File exception"},
```

```
{ 44, HBI_ERR_COMM_TYPE, "Wrong communication type"},
{ 45, HBI_ERR_INVALID_MODE, "This mode contains incorrect parameters"},
{ 46, HBI_ERR_FIRM_CONFIG_DATA, "Firmware parameter data is abnormal"},
{ 47, HBI_ERR_IS_NULL, "The pointer is NULL"},
{ 48, HBI_ERR_ROM_PARAMETER, "ROM includes incorrect parameters"},
{ 49, HBI_ERR_FILE_TYPE, "File type exception"},
{ 50, HBI_END, "Exit monitoring"}}};
int nCode = 0;
int count = sizeof(CrErrStrList) / sizeof(CodeStringTable);
const char *tmpString = GetErrString(CodeStringTable, count, nCode);
printf("nCode=%d,tmpString=%s\n", nCode, tmpString);
return 0;
}
```

4. Detector calibration

4.1 About Calibration

Calibration type, currently supports four calibration methods: offset calibration, gain calibration, and defect calibration. Among them, Dummy calibration is not currently supported and belongs to image preprocessing. The following content defaults to offset calibration, gain calibration, and defect calibration.

Calibration type	Remarks
Offset Calibration - Offset	Eliminating physical differences between image sensors and hardware
Gain calibration - Main	Calibrate the conical beam of the spherical tube and convert the imaging features of the conical beam into a flat image
Bad point calibration - Defect	Calibrate physical defects on image sensors

Calibration template, currently image calibration is achieved by saving calibration coefficients through the calibration template and reading them into memory. The calibration algorithm calibrates the image based on the calibration coefficients.

Calibration type	Template (hbi_id0 template, where id0 represents board A)
Offset Calibration - Offset	OffsetS02B1F00250.raw (composed of tablet model, binning type, frame rate parameters, ZOOM and other parameters)
Gain calibration - Main	Gaina.raw and gainb.raw (composed of tablet model, binning type, PGA gear, dosage, ZOOM and other parameters in the directory)
Bad point calibration - Defect	Defect.raw (composed of tablet model, binning type, PGA gear, dosage, ZOOM and other parameters in the directory)

Soft calibration and hard calibration. "Soft calibration" refers to the automatic completion of image calibration (completed by the computer) by the SDK after setting the software calibration enable. It is suitable for low frame rate or single frame acquisition. "Hard calibration" refers to setting up hardware calibration enable, scanning and reading sensor data through firmware, and performing image algorithm calibration. The SDK will receive the calibrated image (completed by the detector end), which is highly efficient and suitable for high frame rate acquisition.

Template generation, the Offset template is a template generated from a set of dark field images, the Gain template requires at least one set of bright field image generated templates, and the Defect template requires at least three sets of bright field image generated



DT Imaging Technology (Shanghai) Co., Ltd

templates with different doses. The bright field image must be an empty field and cannot be generated by occlusions, etc. Users can use the DT tool software to use template wizards for distributed generation or integrate template generation interface functions with high-pressure generation. For detailed information, please refer to "7. One click template generation". In the "hard calibration" section, the Offset calibration template can be used to generate firmware through commands, as it is a dark field image. The speed is relatively fast and the effect is good.

Template download is aimed at dynamic high frame rate tablets. In order to improve the frame rate of the tablet and reduce time loss caused by SDK calibration of images, "hard correction" or firmware calibration is used here. The prerequisite is to first download the appropriate template to the firmware, and then use firmware calibration to enable it. The tablet is uploaded to the SDK and upper computer software to obtain the calibrated image. This method is highly efficient. Therefore, the necessary step here is to download the template. In the "hard calibration" section, the offset calibration template does not need to be downloaded because it is generated by the firmware and already exists in the firmware. However, it is best to update the offset calibration template through the command during the first startup or power outage.

Calibration has a sequential order, with the default being to first perform offset calibration, then perform Gain calibration, and finally perform Defect calibration. Offset calibration cannot be skipped for Gain and Defect calibration. Similarly, completing offset and Gain calibration before performing Defect calibration is necessary, but only offset calibration or both can be performed.

The following is the calibration enable status:

Calibration type	Calibration enable status
Offset calibration - Offset	0- Do not perform offset calibration; 1-software pre offset calibration; 2-firmware post offset calibration; 3-firmware pre offset calibration;
Gain calibration	0- Do not perform gain calibration; 1-software gain calibration; 2-firmware gain calibration;
Bad Point Correction - Defect	0- Do not perform defect calibration; 1-software defect calibration; 2-firmware defect calibration;

There are three calibration methods: Offset, Gain, and Defect. The implementation of the calibration function can be done through software or firmware. Users can choose whether to calibrate through software or firmware. The corresponding functions can be configured through configuration options. The list of supported modes for software and firmware calibration is as follows:

	Pre Offset	Post Offset	Gain	Defect
Software	√	×	√	√
Firmware	√	√	√	√

Bias calibration must be performed with X-ray turned off. The bias calibration process obtains a series of dark field images and calculates the average image of these dark field images. The obtained average image is used to eliminate the inherent background bias of each pixel in the detector, known as offset bias calibration.

The gain calibration process obtains a series of exposed images (**empty fields**) without objects, which is used to eliminate the problem of image non-uniformity caused by inconsistent pixel response of the detector. Remove the exposed object, obtain several exposure images under different exposure doses, calculate and store the average exposure images under different exposure doses, and obtain the calibration template through the gain calibration template generation algorithm.

The bad point calibration process obtains a series of exposed images (**empty fields**) without objects, which is used to eliminate the problem of image non-uniformity caused by inconsistent pixel response of the detector. Remove the exposed object, obtain several exposure images under different exposure doses, calculate and store the average exposure images under different exposure doses, and obtain the calibration template through the bad point calibration template generation algorithm.

1》 Offset calibration



DT Imaging Technology (Shanghai) Co., Ltd

Offset calibration can be divided into software Pre Offset calibration, firmware Post Offset calibration, and firmware Pre Offset calibration.

(1) Software Pre Offset Calibration

The software Pre Offset calibration method requires the software to create an offset template in advance. When configuring the software offset calibration, the offset image frame rate n can be configured in the offset template generation process interface. After activating the image acquisition function, the software collects and stores n dark field images according to the configured capture frame rate n , generating an offset template.

(2) Firmware Post Offset Calibration

The firmware Post Offset calibration method does not require the software to create an offset template. Commonly seen in single frame acquisition, when configured for firmware Post Offset calibration, the firmware immediately collects one frame of image and then another frame of dark field image. The original image and dark field image automatically complete Post Offset calibration.

(3) Firmware Pre Offset Calibration

The firmware Pre offset calibration method does not require the software to create an offset template. Instead, an automatic command to generate a Pre offset is sent to the firmware. The firmware will generate a Pre offset and store it in the designated location of the firmware emmc, or upload local Pre offset calibration to the firmware. When configuring the firmware Pre offset calibration, the firmware will automatically perform Pre offset calibration when capturing images.

2》 Gain calibration

The Gain calibration method currently only supports software Gain calibration. The generation of a Gain template requires collecting multiple sets of bright field images with different exposure doses. When generating a Gain template, multiple sets of bright field images with different exposure doses are used for data fitting to generate a Gain calibration template.

3》 Defect calibration

The Defect calibration method currently only supports software Defect calibration. Defect calibration is the process of performing numerical statistics on the bright field images obtained during the Defect calibration process, identifying the defective pixels in the entire image, calibrating the specific physical coordinates of the defective pixels, and generating the corresponding Defect Map coordinate file.

Bad pixel types can be divided into:

- (1) Bad pixels: Pixels do not produce light response.
- (2) Nonlinear response pixels: The light response generated by pixels deviates significantly from other pixels.

Calibration sequence, original image ->Offset calibration ->Gain calibration ->Defect calibration ->Upper computer, cannot skip offset calibration and directly perform gain or bad point calibration.

Note that the following firmware manages template storage and loading in a Binning type manner. In the "5.3 Multi Mode Management Integration" chapter, another template storage and management method for firmware will be introduced.

4.2. Quick template generation

"Quick template generation", as its name suggests, aims to generate template files quickly. Therefore, it strives to generate templates that comply with standards while ensuring rapid generation. This template generation method is suitable for user integrated development. Users only need to call the interface to quickly generate template files, and do not need to know the template generation process. The basic parameters required for generation are default.

Calibration is currently divided into offset calibration, gain calibration, and defect calibration.

- 1》 This tablet supports firmware for offset, gain, and defect calibration;
- 2》 The firmware can be used as an offset template, but after the gain and defect template software is successfully installed, it needs to be downloaded to the firmware;
- 3》 After completing the offset, gain, and defect templates (where gain and defect need to be downloaded), enabling firmware calibration will automatically perform calibration after image acquisition. If calibration is required for the original image, it can be cancelled.

This interface is specially developed to quickly integrate and generate templates, simplify the difficulty of integrated development, and meet the needs of different development users. Below, we will provide a brief explanation of the generation process of offset, gain, and defect.

Attention: When collecting bright field images, there should be no material obstruction, such as "empty field". Secondly, ensure that the light source is fully covered, otherwise there will be "burrs" phenomenon. Because the frame rate collected by each group in the bright field is fixed, ensuring continuous and stable exposure under high pressure is crucial.

Please refer to the relevant chapters of "5. Secondary Development" for detailed code.

4.2.1 Offset Template

The offset template includes software Pre-Offset template, firmware Post-Offset template, and firmware Pre-Offset template. This template depends on parameters such as tablet type, binning type, PGA gear, frame rate, and Zoom area. Changing any of these parameters will affect the offset template.

Offset template	Functions and parameters	Remarks
Software Pre Offset Template	Interface function: HBI_GenerateTemplate Parameter: SOFTWARE_OFFSET_TEMPLAT E	Collect a set of unprocessed dark field images and generate a Pre-Offset template locally using the Offset template algorithm. After enabling the software Pre-Offset, the image will be automatically calibrated by the software Pre-Offset before being uploaded to the user. If the firmware is downloaded, enable the firmware Pre-Offset, and complete the Pre-Offset calibration before uploading the firmware to the dynamic library;
Firmware Post Offset Template	Enabling firmware Post Offset is sufficient	Common single frame acquisition , when the user enables the firmware Post Offset, after the exposure acquisition of one frame of bright field image is completed, the high-pressure ball tube is closed, and the firmware immediately collects one frame of dark field image (Post Offset template). The bright field image is calibrated with the dark field image through Post Offset, and then the image is uploaded to the upper computer;
Firmware Pre Offset Template	Interface function: HBI_GenerateTemplate Parameter: OFFSET_TEMPLATE_TYPE	Additionally, pre offset calibration can also be completed by uploading software generated pre offset templates to the firmware;

4.2.2 Gain Template

Gain templates are usually generated by the software side. If firmware Gain calibration is required, it is necessary to first generate a Gain template and download the firmware. This template depends on parameters such as tablet type, binning type, PGA gear, high-pressure dose, and Zoom area. Changing any of these parameters will affect the Gain template. The generation of Gain templates relies on the Offset template, so update the Offset template first.

Gain template	Functions and parameters	Remarks
Gain template	Interface function:	Adjust the high-voltage dose (high-voltage,

	HBI_GenerateTemplate Parameter: GAIN_TEMPLATE_TYPE	milliampere second and other parameters, because only one group of bright field images are collected, and the dose is taken normally), open the bulb tube, call the generation template interface after the bulb tube ray is stabilized, collect a group of bright field images and generate the Gain template, pay attention to the "empty field", and there should be no objects between the bulb tube and the flat plate.
--	--	--

4.2.3 Defect Template

Defect templates are usually generated by the software side. If firmware Defect calibration is required, it is necessary to first generate a Defect template and download the firmware. This template depends on parameters such as tablet type, binding type, PGA level, high-pressure dose, and Zoom area. Changing any of these parameters will affect the Defect template. The generation of the Defect template relies on the Offset template, so the Offset template is updated first. The generation process of the Defect template is similar to that of the Gain template. The Defect template needs to collect at least three sets of bright field images, and the high-pressure dose of the other three sets of bright field images is different, for example.

Defect template	Functions and parameters	Remarks
Defect template	Interface function: HBI_GenerateTemplate Parameter: DEFECT_TEMPLATE_GROUP1	Normal high voltage, milliampere second adjustment normal 10%
	Interface function: HBI_GenerateTemplate Parameter: DEFECT_TEMPLATE_GROUP2	Normal high voltage, milliampere second adjustment normal 50%
	Interface function: HBI_GenerateTemplate Parameter: DEFECT_TEMPLATE_GROUP3	Normal high voltage, milliampere second adjustment normal

4.2.4 Template Download

Only when firmware calibration is required

Note that the following firmware manages template storage and loading in a Binning type manner. In the "5.4 Multi Mode Management Integration" chapter, another template storage and management method for firmware will be introduced.

Steps:

1》 Register progress status callback function, download real-time status and results will be fed back to the upper computer through the callback function;

2》 Call the download interface function and pay attention to input parameters, such as the download template file type, binning type, and Mode ID. Some template download interface functions also require passing in the absolute path of the template file;

Number	Interface functions	Remarks
1	HBI_RegProgressCallBack	Progress status callback function, pay attention to distinguishing the main callback function, refer to SDK Demo for details.
2	HBI_DownloadTemplate HBI_DownloadTemplateByType HBI_DownloadModeTemplate HBI_DownloadModeTemplateByType HBI_DownloadModeTemplateByIdAndType	Template download, input parameters are also different. Generally, input parameters include template type, absolute path of template file, Mode Id, and Binning type. Example: 1》 Binning method, key parameter is Binning type, Mode Id can be ignored; 2》 Multiple Mode mode, key parameter is Mode Id, Binning type can be ignored; 3》 Individual functions require the absolute path of the template file;

4.2.5 Calibration Enable

After completing the template, it is necessary to reset the calibration enable. Only when the calibration enable is enabled will the image be calibrated based on these parameters, either through software calibration or firmware calibration.

Interface functions	Quasi enabling parameters
HBI_UpdateCorrectEnable	0x00- No calibration done 0x01- Software offset 0x02- Firmware post offset 0x03- Firmware pre offset, but a pre offset template needs to be created first
	0x00- No calibration done 0x01- Software gain 0x02- Firmware gain, but the gain template needs to be downloaded to the tablet first
	0x00- No calibration done 0x01- Software defect 0x02- Firmware defect, but the defect template needs to be downloaded to the tablet first

In addition, there are other interface functions that can also update calibration enablement, such as

HBI_TriggerAndCorrectDisplay

HBI_TriggerPgaBinningAcqTimeCorrect

HBI_MTriggerAndCorrectDisplay

HBI_MTriggerPgaBinningAcqTimeCorrect

4.3 Template Wizard Resources

For some customers, template wizard resources have been added to DLL. Users can directly call the interface to open the wizard resource window, generate templates or set some parameters, and close them after completion. The bright field diagram requires the cooperation of the user's high-voltage synchronization module.

4.3.1 Open Template Wizard

Code:

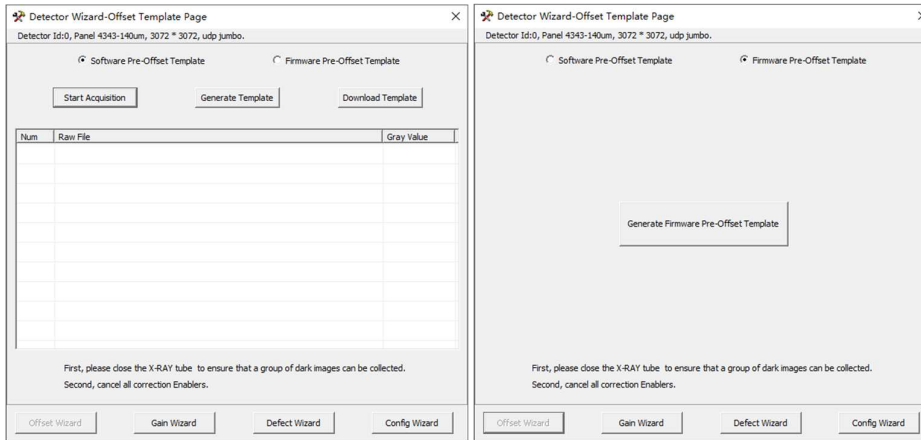
```
int ret=HBI_OpenTemplateWizard (m_pFPD)// Open Wizard Interface Function
If (ret!=HBI_SUCCSS)
{
    If (HBI_ERR_NODEVICE==ret)
    {
        : AfxMessageBox (_T("warning: Dll not initialized!"));
    }
    else if (HBI_ERR_NODEVICE_TRY_CONNECT==ret)
    {
        : AfxMessageBox (_T("warning: disconnect!"));
    }
    else if (HBI_ERR_INVALID_PARAMS==ret)
    {
        : AfxMessageBox (_T("err: pRomCfg is NULL!"));
    }
    else if (HBI_ERR_OPEN_WIZARD_FAILED==ret)
    {
        : AfxMessageBox (_T("err: open template wizard failed!"));
    }
    else if (HBI_ERR_WIZARD_ALREADY_EXIST==ret)
    {
        : AfxMessageBox (_T("warning: template wizard already exists!"));
    }
    else
    {
        : AfxMessageBox (_T("err: other error!"));
    }
}
```

This interface is relatively simple. Users only need to call the interface to pop up the template wizard dialog box, **generate and download** offset, gain, and defect calibration templates according to the interface prompts, and support setting some key parameters to reduce the difficulty of user integration. For specific usage, please refer to the XDiscovery tool user manual, which is similar to the template wizard module in XDiscovery.

The wizard interface is as follows:

4.3.2 Offset Template Generation Page

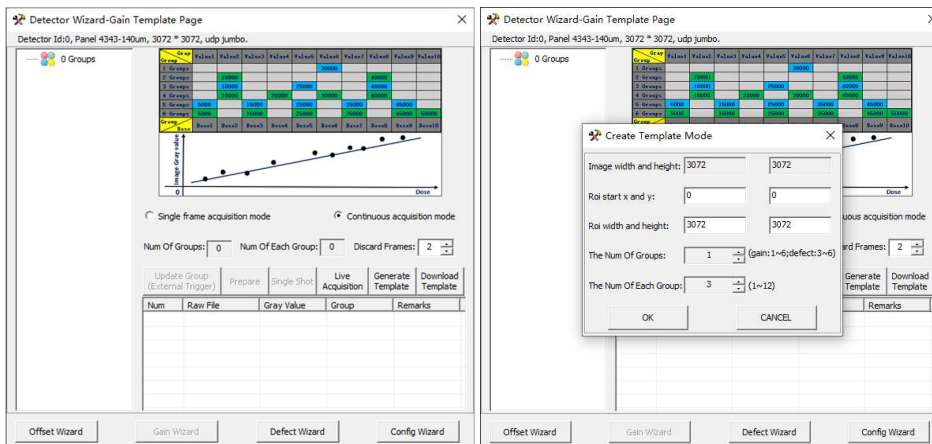
This includes generating software pre offset templates and firmware pre offset templates. The firmware pre offset template only supports dynamic tablets, while the software pre offset template supports low frame rate static tablets.



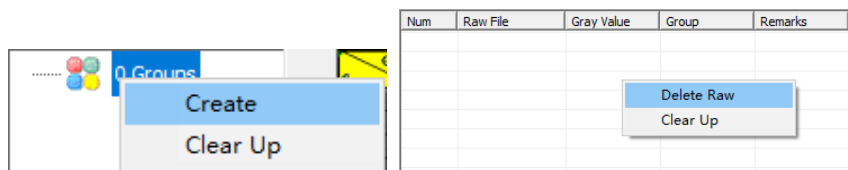
Start Acquisition ->Generate Template ->Download Offset Template ->End. The firmware calibration requires downloading a template, otherwise there is no need to download a template.

4.3.3 Gain Template Generation Page

Support generating Gain calibration templates under different triggering modes, and dynamic tablets support template downloading. The process of generating Gain templates requires high-pressure coordination, and at least one set of bright field images can be collected under normal dosage to generate Gain templates.



The tree control on the left and the list control in the small right corner use the **right mouse button** to open submenus for **creating, clearing, deleting and clearing** sub items.

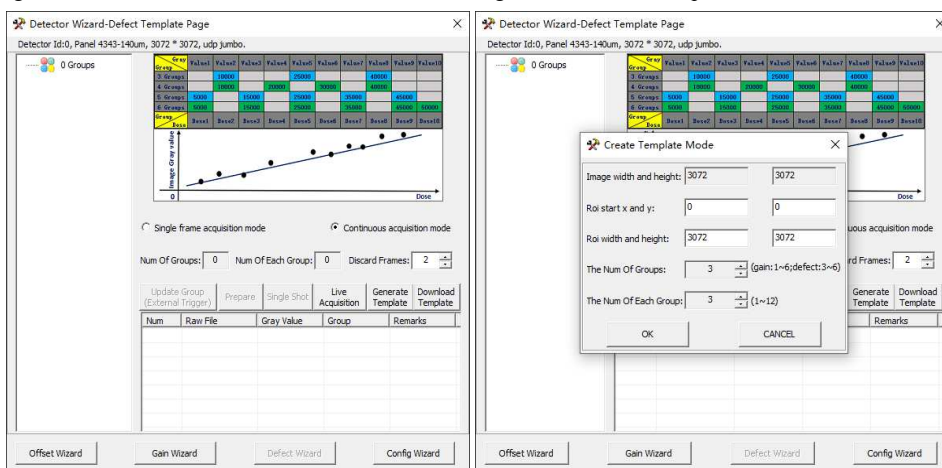


Right click on the tree control "0 Groups" in the left column ->Pop up menu "create" ->Confirm group parameters "ok" ->Adjust

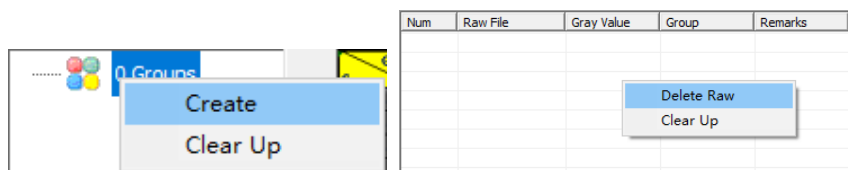
dosage, open the high-pressure arena, stabilize ->Click "Live Acquisition" to collect images ->"Generate Template"  to generate templates ->Click the download template button ->End. The firmware calibration requires downloading a template, otherwise there is no need to download a template.

4.3.4 Defect template generation page

Support generating Defect calibration templates under different triggering modes, and dynamic tablet templates support downloading. The process of generating a Defect template requires high-pressure coordination, and at least three sets of bright field images at different doses need to be collected in order to generate a Defect template.



The tree control on the left and the list control in the small right corner use the **right mouse button** to open submenus for **creating, clearing, deleting and clearing** sub items.



Right click on the tree control "0 Groups" in the left column ->Pop up menu "create" ->Confirm group parameters "ok" ->Adjust dosage, open the high-pressure arena, stabilize ->Click "Live Acquisition" to collect the first group of images ->Adjust dosage, open the high-pressure arena, stabilize ->Click "Live Acquisition" to collect the second group of images ->Adjust dosage, open the high-pressure arena, After stabilization ->Click on "Live Acquisition" to collect the third set of images ->Click on the "Generate Template"

button to generate a template ->Click on the download template button  ->End. The firmware calibration requires downloading a template, otherwise there is no need to download a template.

4.3.5 Configuration Page

Mainly including functions:



DT Imaging Technology (Shanghai) Co., Ltd

View basic information of the detector (tablet ID/type/resolution/communication method/product serial number/software firmware version number/current image size)

Set thumbnail mode (supported by some tablets)

Set Binning Type

Set PGA gear

Set Prepare delay (control single frame acquisition mode)

Set Live Time (Static Tablet Continuous Collection Time Interval)

Set Self Dumping Time (dynamic tablet continuous acquisition time interval, i.e. frame rate)

Set trigger mode and enable calibration status

Restore factory settings, export tablet parameters, and open SDK log directory

4.4 Template Management

If software calibration is enabled, the tablet will automatically search and load the template file on the local disk based on the parameters (provided that the template already exists) and achieve software side calibration. What are the prerequisites for firmware calibration? Firstly, it is necessary to download the corresponding template file to the EMMC of the tablet detector. The firmware stores the calibration file according to different versions. When the firmware calibration is enabled, the tablet will automatically load the template file and calibrate the image based on the parameters. The calibrated image will be transferred to the dynamic library, which can improve the frame rate. Below, we will mainly learn how to manage local and firmware side templates.

4.4.1 Local Template Management

Mainly introduce the template path and template file name. The **local** template path and template file name naming rules are shown in the table below. For detailed operations, please refer to the 'XDiscovery User Manual V4. *. *. * _XXX'.

➤ Template root directory

Detector A (id: 0)	Detector B (id: 1)	Detector C (id: 2)	...	Detector N (id: n)
hbi id0 template	hbi id1 template	hbi id2 template	...	hbi idn template

➤ Template directory and template file name



DT Imaging Technology (Shanghai) Co., Ltd

Taking Panel-1613 as an example							
Key letters							
S	B	P	V	Z	F	r	m
Panel size	Binning Type	PGA Level	High Voltage (Dose parameters)	ZOOM Rect	FPS (Affects offset templates)	Rotate	Mirror (1-Vertically/Mirror 2- Horizontal Mirror)
Reference tablet model	1-binning1x1	1-11 represents different PGA level, refer to PGA level table	Currently, due to the inability to obtain specific parameters, it defaults to 0	Rect(left.top,width,height)	Dynamic 1000ms/dumping time	1- Right Rotate 90° 2- Left Rotate 90°	1-Vertically Mirror
information table	2-binning1x1				Static 1000ms/living time	0 or other default	2- Horizontal Mirror
	3-binning1x1				Currently, for character alignment, it has been expanded by 1000 times	0 or other default images do nothing	3- 5 indicates that the firmware has completed different image processing, independent of template generation. 0 or other default images are not
	4-binning1x1						
Offset template (16-bit raw file)		Template file		Notes			
Example 1	offsetS03B1P6F20000			Eg. time interval 50ms, 20 frames represent 1000000/50=F20000. If the time interval is 2000ms, 0.5 frames represent F00500, the time interval is 100000 and 0.01 frames represent F000010			
Example 2	offsetS03B1P6F20000Z0000049612480032			Z0000049612480032 indicates the ZOOM rect parameter, left=0, top=496, width=1248, height=32			
Example 3	offsetS03B1P6F20000Z0000049612480032_r1m2			r1m2 indicates that the image has been rotated right and mirrored horizontally			
Gain template (32-bit raw file)		Folder		Notes			
Example 1		S03B1P6V000		Firmware only updates gain raw			
Example 2		S03B1P6V000Z0000049612480032		gaina.raw, gainb.raw	Z0000049612480032 indicates the ZOOM rect parameter, left=0, top=496, width=1248, height=32		
Example 3		S03B1P6V000		gaina.raw, gainb.raw	Indicates that the image has been rotated right and horizontally mirrored and generate gain template		
		S03B1P6V000		gaina_r1m2.raw, gainb_r1m2.raw	Indicates that the image has been rotated right and horizontally mirrored and generate gain template		
Defect template (16-bit raw file)		Folder		Notes			
Example 1		S03B1P6V000		defect.raw	The grayscale value of 65535 represents bad points, and 0 represents normal points		
Example 2		S03B1P6V000Z0000049612480032		defect.raw	Z0000049612480032 indicates the ZOOM rect parameter, left=0, top=496, width=1248, height=32		
Example 3		S03B1P6V000		defect_r1m2.raw	Indicates that the image has been rotated right and horizontally mirrored and generate defect template		
		S03B1P6V000Z0000049612480032		defect_r1m2.raw	Indicates that the image has been rotated right and horizontally mirrored and generate defect template		
Notes							
1. The ZOOM parameter only supports binning1x1.							
2. The offset template is related to the panel size, binning type, PGA gear, frame rate, Zoom parameters, and software side rotation mirror parameters.							
3. The gain and defect templates are related to the panel size, binning type, PGA gear, Zoom parameters, and software side rotation mirror parameters.							
4. At present, the rotation mirror processing sequence is: rotate first and then mirror.							

激活 W

4.4.2 Firmware Template Management

The firmware side cannot generate gain calibration templates (Gain) and bad point calibration templates (Defect), so the template file for firmware calibration needs to be uploaded from the software side to the firmware.

At present, firmware has different ways of managing templates, as follows:

1》Binning method, key parameter is Binning type, Mode Id can be ignored.

Binning type	Binning 1 * 1	Binning 2 * 2	Binning 3 * 3	Binning 4 * 4
EMMC address	Address 1	Address 2	Address 3	Address 4
template file	Offset template	Offset template	Offset template	Offset template
	Gain template	Gain template	Gain template	Gain template
	Defect template	Defect template	Defect template	Defect template

This method only uses the Binning type as a dependency parameter for storage and loading, so it is only suitable for simple applications that do not require template switching, but the integration is simple.

2》Multiple Mode mode, key parameter is Mode Id, Binning type can be ignored, $n \in [0,9]$;

Mode Id	Mode 0	Mode 1	Mode 2	Mode n	Mode 9
EMMC address	Address 1	Address 2	Address 3	Address n	Address 9
template file	Offset template	Offset template	Offset template	Offset template	Offset template
	Gain template	Gain template	Gain template	Gain template	Gain template
	Defect template	Defect template	Defect template	Defect template	Defect template

At present, it supports up to 9 sets of Mode options, and users can set valid Mode options. Each Mode option contains different parameters, such as tablet model, PGA gear, binning type, dosage, frame rate, Zoom area, and so on. This method is suitable for applications that switch templates in different scenarios. In the early stage, templates with the same Mode item need to be completed, and switching in the later stage is simple and fast.

Finally, users can choose different integration methods based on their actual business needs, and the dynamic library we provide is also compatible with these two methods.

5. Secondary development

The secondary development can be divided into two methods based on firmware for template management (**storage and loading**): Binning method and Multi Mode method. The integration process is detailed in reference to "5.3 Binning Integration" and "5.4 Multi Mode Integration". The Binning integration method is common in early firmware versions, where templates only rely on Binning type storage and loading, with few and single template types. Different templates do not need to switch frequently, and integration is relatively



DT Imaging Technology (Shanghai) Co., Ltd

simple. The multi mode integration method is designed to adapt to rapid template switching in various different scenarios and prevent template "misalignment". The user pre creates different Mode items and generates templates for each Mode and updates them to the firmware. The key is to store and load templates according to Mode Id. The user only needs to switch Mode items through Mode Id. The firmware immediately updates the current collection control parameters and loads the calibration template for that Mode item.

5.1 Interface Description

-hbi_fpd_ver *. * interface root directory

|---**HBI_FPD_Tool**: Tool software

|-**Interface document** for DT FPD Pro SDK API Programming Reference Ver *. *. pdf

|-**Factory.csv factory configuration** export file, used to restore factory settings

|-**Win32** includes 32-bit XDiscovery.exe, HBISDKApi.DLL, and 32-bit Opencv dependency libraries, as well as XML file bit engineering configuration files that can be created, saved, and modified.

|-**X64** includes 64 bit XDiscovery.exe and HBISDKApi.DLL, as well as 64 bit Opencv dependency library XML file bit engineering configuration files, which can be created, saved, and modified.

|---**HBI_DLL**: Interface file, including header file, dynamic library DLL, static library file, and configuration file

|-**HBI_DLL \ INCLUDES**: header file, please refer to the interface documentation for details.

|- 'HbiFpd.h': Interface function file, Export functions and instructions

|- 'HbiType.h': parameter, structure data type file

|- 'HbiError.h': Error List File

|-**HBI_DLL \ BIN \ x86**: 32bit dynamic library DLL, static library files, and configuration files

|-HBISDKApi.DLL and HBISDKApi.lib: 32-bit release dynamic and static library files

|-Opencv_*** : 32-bit dependency library

|-**HBI_DLL \ BIN \ x64**: 64bit dynamic library DLL, static library files, and configuration files

|-HBISDKApi.DLL and HBISDKApi.lib: 64 bit release dynamic and static library files

|-Opencv_*** : 64 bit dependency library

|-**HBI_DLL \ Document**: interface development document

|---**SDK_Demo_Example**: Interface development source code instance (C++ or C #), supported on VS2008 and above versions

|-SDK_Demo_SDK_DEMO2008.7z

5.2 Software deployment

Taking C/C++ language development as an example. Copy the "HBI-DLL" file directory to the user development specified directory, configure the header file directory and library file directory, and pay attention to the 32 bit and 64 bit library files.

Taking SDK Demo as an example, the steps are as follows:

1》 Copy the 'HBI_DLL' folder to the "DemoDll \ DemoDll" or "Demolib \ Demolib" or "HB_SDK_DEMO2013\HB_SDK_DEMO2008" directory, where the header file directory HBI_DLL\INCLUDES, library file directory HBI_DLL \ BIN \ Win32, HBI_DLL \ BIN \ x64;

2》 The " HB_SDK_DEMO2013" project relies on the Opencv341 library and can directly copy "Opencv341 Unzip to D Drive. 7z" to D Drive for direct decompression. The decompression directory is D: \ opencv341 \ opencv;

3》 In some debugging modes, the dependency library opencv requires other libraries, such as msvcp100d. DLL or msver100d. DLL or msvcp120d. DLL or msver120d. DLL. You can find 32-bit or 64 bit library files in the " HB_SDK_DEMO2013_*\ output" directory, and pay attention to 32-bit and 64 bit, as shown in the "Dependency Library Screenshot. png". You can contact the R&D



DT Imaging Technology (Shanghai) Co., Ltd

personnel to obtain them;

4》 The generated directories are: project directory \output\Win32 and project directory\output\x64, please refer to 'HB_SDK_DEMO2013' project for details;

5》 The 'HB_SDK_DEMO2008' project is developed based on MFC Dialogs, including display, template generation, etc., with the most comprehensive functions. DemoDll and Demolib are developed based on console console, including basic functions. DemoDll is a dynamically loaded Demo, and Demolib is a statically loaded Demo. Please refer to the project documentation for details, such as the "DT SDK Development DEMO Project Description Ver4.0. *. *. pdf";

6》 Users who need to integrate a "dual detectors" solution can contact after-sales or R&D personnel to request the development of DEMO engineering source code;

7》 In addition, if you need a C # SDK demo, please contact after-sales or R&D personnel;

5.3 Binning Integration

Please refer to the SDK Demo development program for details.

5.3.1 Steps

1》 Include header files in user engineering

```
#Define_DLL_EX_IM 0
#include "HbiFpd.h"//Dynamic library header file
#pragma comment(lib, "HBISDKApi. lib")
```

2》 User defined devices

```
// Device handles and configurations
Void * m_pFpd; // Fpd sdk object handle
RegCfgInfo * m_pLastRegCfg ;// Firmware configuration
FPD_AQC_MODE aqc_mode// Acquisition mode and parameters, single frame acquisition, multi frame acquisition, dark
field image and bright field image acquisition

// Image Resolution
int m_imgW, m_imgH;
char m_path[MAX_PATH];
fpd_base_cfg_st *fpd_base;
IMAGE_PROPERTY img_pro;
IMAGE_CORRECT_ENABLE* pCorrect;

// Some basic data types
m_pLastRegCfg = new RegCfgInfo;
pCorrect = new IMAGE_CORRECT_ENABLE;
fpd_base = new fpd_base_cfg_st;
```

The device list structure has been defined in 'HbiType. h' .

3》 Interface call process

```
// Initialize DLL resources
Void * m_pFpd =HBI_Init();
if (m_pFpd == NULL) {
    printf("HBI_Init init failed!\n");
    return false;
}
```

```
// Registering Callbacks
//MyCallbackFun callback function
int ret=HBI_RegEventCallBackFun (m_pFpd, myCallbackFun);
if (ret != 0) {
    printf("HBI_RegEventCallBackFun failed!\n");
    HBI_Destroy(m_pFpd);
    m_pFpd = NULL;
    return false;
}
// Connect the flat panel detector
COMM_CFG commCfg;
Bool doOffsetTemps=false// If the firmware is not used as an offset template by default
//1.PCIE mode
CommCfg._type= FPD_COMM_TYPE::PCIE_COMM_TYPE;
CommCfg._pcieId =7024;// The default single panel 7024,7024 is the PCIE card hardware ID
#ifdef 0
//2.UDP mode
//commCfg._type    = FPD_COMM_TYPE:: UDP_COMM_TYPE;
//strcpy(commCfg._localip, "192.168.10.20");
//strcpy(commCfg._remoteip, "192.168.10.40");
//commCfg._loacalPort  = 32896;
//commCfg._remotePort  = 32897;
//3.UDP JUMBO mode
//commCfg._type    = FPD_COMM_TYPE:: UDP_JUMBO_COMM_TYPE;
//strcpy(commCfg._localip, "192.168.10.20");
//strcpy(commCfg._remoteip, "192.168.10.40");
//commCfg._loacalPort  = 32896;
//commCfg._remotePort  = 32897;
#endif
//DoOffsetTemplate - false: The firmware does not use an offset template after a successful connection, true: The firmware
uses an offset template after a successful connection
ret=HBI_Connect Detector (m_pFpd, commCfg, doOffsetTemps);
//ret=HBI_Connect Detector (m_pFpd, commCfg)// After successful connection, the firmware does not create an offset
template
if (ret != 0) {
    printf("Connection failed! \n");
    HBI_Destroy(m_pFpd);
    m_pFpd = NULL;
    return false;
}
// Set trigger mode and enable image correction
Int _triggerMode=7// 1-Soft trigger, 3-High voltage trigger, 4-freeAED,
..., 7-DYNAMIC_FPD_Continue_MODE
IMAGE_CORRECT_ENABLE * pCorrect=new IMAGE_CORRECT_ENABLE;
```



DT Imaging Technology (Shanghai) Co., Ltd

```

if (pCorrect!=NULL){
    Pccorrect ->bFeedbackCfg=true// True      ECALLBACK_TYPE_ROM_UPLOAD      Event,      false
    ECALLBACK_TYPE_SET_CFG_OK Event
    Pccorrect ->ucOffsetCorrection=0x03// 00- "Do nothing";    01- "Prepare Offset Correction"; 02- "Post Offset
Correction";
    Pccorrect ->ucGainCorrection=0x02// 00- "Do nothing";    01- "Software Gain Correction"; 02- "Hardware Gain
Correction"
    Pccorrect ->ucDefectCorrection=0x02// 00- "Do nothing";    01- "Software Defect Correction"; 02- "Software Defect
Correction"
    Pccorrect ->ucDummyCorrection=0x00// 00- "Do nothing";    01- "Software Dummy Correction"; 02-
"Software Dummy Correction"
    ret=HBI_TriggerAndCorrectApply (m_pFpd, _triggerMode, pCorrect);
    if (ret!=0){
        printf("Connection failed! \n");
    }
}
// disconnect
HBI_DisconnectDetector (m_pFpd);
// Release resources (including disconnection and resource release)
HBI_Destroy (m_pFpd);
m_pFpd =NULL;

```

Note:

- 1》 After successful connection, the tablet will automatically provide feedback on ROM parameters.
- 2》 ‘HBI_Init’ and ‘HBI_Destroy’: initializes and releases device;
- 3》 HBI_ConnectDetector and HBI_DisconnectDetector: Connect and disconnect the flat detector;
- 4》 Connect the detector using ‘HBI_ConnectDetector’, callback event ECALLBACK_TYPE_ROM_UPLOAD provides feedback on the current firmware parameters. The basic information here has been fixed and users can use it directly;
- 5》 ‘HBI_GetSystemConfig’ and ‘HBI_GetFpdCfgInfo’: All read detector firmware parameters
‘HBI_GetSystemConfig’ is an asynchronous function that sends a request to the firmware to obtain parameters; ‘HBI_GetFpdCfgInfo’ is a synchronization function that obtains parameters after a successful connection or setting.
- 6》 ‘HBI_TriggerAndCorrectApply’, f After a successful connection, initialize the basic control parameters, such as trigger mode and calibration enable;
- 7》 HBI_Destroy releases resources, handle is NULL, if closed directly, call ‘HBI_Destroy’ is sufficient, The call to ‘HBI_DisConnectDetector’ is already included in ‘HBI_Destroy’.
- 8》 Define callback functions

#####

#Callback function and event description:

//Function Description: Callback function definition, handling configuration, images, and tablet return status.

//Parameter Description:

// @pContext: Parameter 1, upper computer object pointer, can be null

// @ufpdId: Parameter 2, such as tablet ID

// @byteEventId: Parameter 3, Event ID, refer to enum eCallbackEventCommType in ‘HbiType.h’

//@ pvParam1: Parameter 4, configure structure pointer or image structure pointer



DT Imaging Technology (Shanghai) Co., Ltd

// @nParam2: Parameter 5, such as data size (image data is the length of the image buffer) or state

// @param3: Parameter 6, such as frame number, frame ID

// @param4 Parameter 7, such as frame rate, frame rate, or state, etc

//Callback function ends. Return value description: The return values are integers 0 and 1, 1- Received message, 0- Not received message

#####

int CDemoDlg:: myCallbackFun (void * _context, int nfpdId, unsigned char command, void * pvParam1, int nParam2, int param3, int param4)

```
{
    int ret=0;
    if ((command == ECALLBACK_TYPE_SINGLE_IMAGE) ||
        (command == ECALLBACK_TYPE_MULTIPLE_IMAGE) ||
        (command == ECALLBACK_TYPE_ROM_UPLOAD))
    {
        if (buff == NULL || len == 0){
            printf("Registration callback function parameter exception! \n");
            return ret;
        }
    }
    //
    int status = -1;
    int j = 0;
    ret = 1;
    switch (command)
    {
        case ECALLBACK_TYPE_FPD_STATUS://Tablet status: connected/disconnected/ready/busy
            printf("ECALLBACK_TYPE_FPD_STATUS, recode=%d \n", len);
            break;
        case ECALLBACK_TYPE_ROM_UPLOAD:/* Update configuration*/
            printf("ECALLBACK TYPE ROM_UPLOAD: \n");
            break;
        case ECALLBACK_TYPE_SET_CFG_OK:
            printf("ECALLBACK TYPE SET CFG OK: \n");
            break;
        case ECALLBACK_TYPE_SINGLE_IMAGE://Single frame capture image
        case ECALLBACK_TYPE_MULTIPLE_IMAGE //Continuous collection of images
            break;
        case ECALLBACK_OVERLAY_NUMBER://Overlay progress number, such as stacking 8 frames, now stacking up to 5 frames
            break;
        case ECALLBACK_OVERLAY_16BIT_IMAGE://After stacking is completed, update the stacking result graph (16 bits)
            break;
        .....
        case ECALLBACK_TYPE_THREAD_EVENT:
```



DT Imaging Technology (Shanghai) Co., Ltd

default:

```
printf("command=%02x\n", command);
break;
}
return ret;
}
```

Return the image properties, length, width, number of bits, type, and size information, trigger mode, firmware calibration enable, etc. through callback functions.

Return the image data address and length through a callback function. If other parameters are required, please explain.

In addition, template generation and image calibration are also important parts of code integration. Next, we will focus on template generation, template download, and updating calibration enablement. The progress status callback function can be found in the section 5.2.5 Progress Status Callback Function.

5.3.2 Offset Template Generation and Download

Firstly, connect the tablet detector and update the tablet control parameters, such as binning type, PGA gear, frame rate, and Zoom area.

Collect 1 set of dark field images and generate a template until success is returned, otherwise failed is returned.

```
EnumIMAGE_ACQ_MODE enumTemplateType=EnumIMAGE_ACQ_MODE::OFFSET_TEMPLATE_TYPE;
```

```
int ret = HBI_GenerateTemplate(m_pFpd, enumTemplateType);
```

```
if (ret!=HBI_SUCCSS) {}//failed
```

```
else {}//Success
```

The offset template generated by the firmware generally does not need to be downloaded. In some scenarios, the high-pressure ball tube continuously emits X-rays, and the software needs to generate calibration in advance and download it to the firmware.

// To register a callback function, it is necessary to register and provide feedback on progress or result information

//Callback function: USER_CALLBACK_HANDLE_PROCESS handFun - Global function or static member function, refer to HbiType. h

//PContext - Pointer to the upper computer object, which can also be assigned as null if not needed by the user.

If (0!=HBI-RegProgressCallBack (m_pFPD, DownloadCallBackFun, (void *) this))

```
{
    printf("err: HBI-RegProgressCallBack failed! \n");
    Return;
}
```

// parameters

//if (downloadfile == NULL) downloadfile = new DOWNLOAD_FILE;

//if (downloadfile == NULL)

```
//{
```

```
//    printf("err:malloc downloadfile failed!\n");
```

```
//    return;
```

```
//}
```

//downloadfile->emfiletype = OFFSET_TMP;

// downloadfile->nBinningtype = 1; eg, the binningtype of template is binning1x1



DT Imaging Technology (Shanghai) Co., Ltd

```
//int j=sprintf(downloadfile->filepath,"%s\\hbi_id0_template\offsetS * * B * P * F ***** raw", theDemo->d_path);
//downloadfile->filepath[j]='\0';
// start download the template of Pre-offset
//int ret = HBI_DownloadTemplate(m_pFpd, downloadfile);
//if (ret != HBI_SUCCSS)
//    ::AfxMessageBox(_T("err:Download offset-template failed!"));
//else
//    ::AfxMessageBox(_T("Download offset-template success!"));
// 释放资源
//Delete pUploadFile;
//PUploadFile=NULL;
or
// start download the template of Pre-offset
int infiletype=2// Download file types 0-gain template, 1-defect template, 2-offset template, others - not supported
int ret=HBI_DownloadTemplateByType (d_pFPD, infiletype)// Suggested use
If (ret!=HBI_SUCCSS)
    : AfxMessage    Box (_T ("err: Download offset template failed!"));
else
    : AfxMessage    Box (_T ("Download offset template success!"));
// Update offset calibration enable, currently enabling firmware Pre offset calibration
IMAGE_ CORRECTION_ ENABLE * pccorrect=new IMAGE_ CORRECTION_ ENABLE;
If (pccorrect!=NULL)
{
    pccorrect->bFeedbackCfg=false;
    pccorrect->ucOffsetCorrection=0x03// 0x00- no calibration, 0x01- software offset, 0x02- firmware pose offset, 0x03- firmware
pre offset offset, dynamic tablet uses firmware pre offset, which is 0x03
    pccorrect->ucGainCorrection=0x00// 0x00- no calibration, 0x01- software gain, 0x02- firmware gain, but the gain template needs
to be downloaded to the tablet first
    pccorrect->ucDefectCorrection=0x00// 0x00- No calibration, 0x01- Software defect, 0x02- Firmware defect, but the defect
template needs to be downloaded to the tablet first
    pccorrect->ucDummyCorrection=0x00// 0x00- No calibration, 0x01- Currently not supported, 0x02- Firmware currently not
supported, currently not supported by software firmware
    ret=HBI_UpdateCorrectEnable (theDoc->m_pFPD, pccorrect);
    if (ret==0)
        // successful
    else
        // failed
//Release resources
delete correct;
pccorrect=NULL;
}
```

5.3.3 Gain template generation and download

Firstly, ensure that the tablet detector is connected and update the tablet control parameters, such as binning type,



DT Imaging Technology (Shanghai) Co., Ltd

PGA gear, high-pressure dose, and Zoom area.

EnumIMAGE_ACQ_MODE enumTemplateType=EnumIMAGE_ACQ_MODE::GAIN_TEMPLATE_TYPE;

int ret=HBI_GenerateTemplate(d_pFPD, enumTemplateType);

If (ret!=HBI_SUCCSS) {}//failed

else {}//Success

// register a callback function, it is necessary to register and provide feedback on progress or result information

//Callback function: USER_CALLBACK_HANDLE_PROCESS handFun - Global function or static member function, refer to HbiType. h

//PContext - Pointer to the upper computer object, which can also be assigned as null if not needed by the user.

If (0!=HBI_RegProgressCallBack(m_pFPD, DownloadCallBackFun, (void *) this))

{

printf("err: HBI_RegProgressCallBack failed! \n");

return;

}

// 参数

//if (downloadfile == NULL) downloadfile = new DOWNLOAD_FILE;

//if (downloadfile == NULL)

//{

// printf("err:malloce downloadfile failed!\n");

// return;

//}

//

//downloadfile->emfiletype = GAIN_TMP;

// downloadfile->nBinningtype = 1;// eg, the binningtype of template is binning1x1

//int j=sprintf(downloadfile->filepath, "%s \\ hbi_id0_template \\ S * * B * P * V * * * \\ gaina. raw", theDemo->m path);

//downloadfile->filepath[j] = '\0';

// **Method 1 :start download the template of Gain**

//int ret = HBI_DownloadTemplate(m_pFpd, downloadfile);

//if (ret != HBI_SUCCSS)

// ::AfxMessageBox(_T("err:Download gain-template failed!"));

//else

// ::AfxMessageBox(_T("Download gain-template success!"));

// free resources

//Delete pUploadFile;

//PUploadFile=NULL;

or

// **Method 2:start download the template of Gain**

int infiletype=0// Download file types 0-gain template, 1-defect template, 2-offset template, others - not supported

int ret=HBI_DownloadTemplateByType(d_pFPD, infiletype)// **Suggested use**

if (ret!=HBI_SUCCSS)

: AfxMessageBox(_T("err: Download gain template failed!"));

else

: AfxMessageBox(_T("Download gain template success!"));

//Update gain calibration enable, currently enabling firmware gain calibration



DT Imaging Technology (Shanghai) Co., Ltd

```
IMAGE_CORRECTION_ENABLE * pccorrect=new IMAGE_CORRECTION_ENABLE;
```

```
if (pccorrect!=NULL)
```

```
{
```

```
    pccorrect ->bFeedbackCfg=false;
```

```
    pccorrect ->ucOffsetCorrection=0x03// 0x00- no calibration, 0x01- software offset, 0x02- firmware pose offset, 0x03- firmware pre offset offset, dynamic tablet uses firmware pre offset, which is 0x03
```

```
    pccorrect ->ucGainCorrection=0x02// 0x00- no calibration, 0x01- software gain, 0x02- firmware gain, but the gain template needs to be downloaded to the tablet first
```

```
    pccorrect ->ucDefectCorrection=0x00// 0x00- No calibration, 0x01- Software defect, 0x02- Firmware defect, but the defect template needs to be downloaded to the tablet first
```

```
    pccorrect ->ucDummyCorrection=0x00// 0x00- No calibration, 0x01- Currently not supported, 0x02- Firmware currently not supported, currently not supported by software firmware
```

```
    Ret=HBI_UpdateCorrectEnable (theDoc ->m_pFPD, pccorrect);
```

```
    if (ret!=HBI_SUCCSS) {}//failed
```

```
    else {}//Success
```

```
    //Release resources
```

```
    delete correct;
```

```
    pccorrect=NULL;
```

```
}
```

5.3.4 Defect template generation and download

Firstly, ensure that the tablet detector is connected and update the tablet control parameters, such as binning type, PGA gear, high-pressure dose, and Zoom area.

Collect 3 sets of bright field images, which require the cooperation of a high-voltage generator. After the lamp tube radiation stabilizes, trigger the command to complete template generation.

1》 Normal high voltage, milliampere second adjustment normal 10%

```
EnumIMAGE_ACQ_MODE enumTemplateType=EnumIMAGE_ACQ_MODE: DEFECT_ACQ_GROUP1;
```

```
int ret=HBI_GenerateTemplate (m_pFPD, enumTemplateType);
```

```
if (ret!=HBI_SUCCSS) {}//failed
```

```
else {}//Success
```

Note: As it is a firmware pre offset template, there is no need to download it to the firmware again.

2》 Normal high voltage, milliampere second adjustment normal 50%

```
EnumIMAGE_ACQ_MODE enumTemplateType=EnumIMAGE_ACQ_MODE: DEFECT_ACQ_GROUP2;
```

```
int ret=HBI_GenerateTemplate (d_pFPD, enumTemplateType);
```

```
if (ret!=HBI_SUCCSS) {}//failed
```

```
else {}//Success
```

3》 Normal high voltage, milliampere second adjustment normal

```
EnumIMAGE_ACQ_MODE enumTemplateType=EnumIMAGE_ACQ_MODE: DEFECT_ACQ_GROUP3;
```

```
int ret=HBI_GenerateTemplate (m_pFPD, enumTemplateType);
```

```
if (ret!=HBI_SUCCSS) {}//failed
```

```
else {}//Success
```

```
/*
```



DT Imaging Technology (Shanghai) Co., Ltd

*** After the template is successfully generated, start downloading the template**

```
*/  
// register a callback function, it is necessary to register and provide feedback on progress or result information  
//Callback function: USER_CALLBACK_HANDLE_PROCESS handFun - Global function or static member  
function, refer to 'HbiType. h'  
//PContext - Pointer to the upper computer object, which can also be assigned as null if not needed by the user.  
If (0!=HBI_RegProgressCallBack (m_pFPD, DownloadCallBackFun, (void *) this))  
{  
    printf("err: HBI_RegProgressCallBack failed! \n");  
    Return;  
}  
// parameter  
//if (downloadfile == NULL) downloadfile = new DOWNLOAD_FILE;  
//if (downloadfile == NULL)  
//{  
//    printf("err:malloce downloadfile failed!\n");  
//    return;  
//}  
//  
//if (downloadfile->emfiletype != DEFECT_TMP)downloadfile->emfiletype = DEFECT_TMP;  
// downloadfile->nBinningtype = 1;// eg, the binningtype of template is binning1*1  
//int j = sprintf(downloadfile->filepath, "%s\\hbi_id0_template\\S**B*P*V***\\defec.raw",m_path);  
//downloadfile->filepath[j] = '\0';  
// method 1: start download the template of defect  
//int ret = HBI_DownloadTemplate(m_pFpd, downloadfile);  
//if (ret != HBI_SUCCSS) {  
//    ::AfxMessageBox(_T("err:Download defect template failed!"));  
//}  
//else  
//    ::AfxMessageBox(_T("Download defect template success!"));  
// free resource  
//Delete pUploadFile;  
//PUploadFile=NULL;  
or  
// method 2: start download the template of defect  
int infiletype=1// Download file types 0-gain template, 1-defect template, 2-offset template, others - not supported  
int ret=HBI_DownloadTemplateByType (d_pFPD, infiletype)// Suggested use  
If (ret!=HBI_SUCCSS)  
    : AfxMessage    Box (_T ("err: Download defect template failed!"));  
else  
    : AfxMessage    Box (_T ("Download defect template success!"));  
//Update defect calibration enable, currently enabling firmware defect calibration  
IMAGE_CORRECTION_ENABLE * pccorrect=new IMAGE_CORRECTION_ENABLE;  
If (pccorrect!=NULL)
```



```

{
    pccorrect ->bFeedbackCfg=false;
    pccorrect ->ucOffsetCorrection=0x03// 0x00- no calibration, 0x01- software offset, 0x02- firmware pose offset, 0x03- firmware
pre offset offset, dynamic tablet uses firmware pre offset, which is 0x03
    pccorrect ->ucGainCorrection=0x02// 0x00- no calibration, 0x01- software gain, 0x02- firmware gain, but the gain template needs
to be downloaded to the tablet first
    pccorrect ->ucDefectCorrection=0x02// 0x00- No calibration, 0x01- Software defect, 0x02- Firmware defect, but the defect
template needs to be downloaded to the tablet first
    pccorrect ->ucDummyCorrection=0x00// 0x00- No calibration, 0x01- Currently not supported, 0x02- Firmware currently not
supported, currently not supported by software firmware
    ret=HBI_UpdateCorrectEnable (theDoc ->m_pFPD, pccorrect);
    if (ret==0)
        // successful
    else
        // failed
    //Release resources
    delete correct;
    pccorrect=NULL;
}

```

5.3.5 Progress Status Callback Function

Before downloading the template, be sure to register the progress and result callback functions, which can provide basic status and results during the template generation process.

// Download template callback function: static member function or global function

//InContext: Upper computer object pointer, can be NULL.

Static int DownloadCallBackFun (unsigned char command, int code, void * inContext)

```

{
    ///CTemplateTool * ptr=(CTemplateTool *) inContext;
    ///if (ptr==NULL){
    ///printf("err: inContext is NULL! \n");
    ///return 0;
    ///}
    ///if (TheDemo==NULL){
    ///printf("err: TheDemo is NULL! \n");
    ///return 0;
    ///}
    int len=0;
    switch (command)
    {
        case ECALLBACK_UPDATE_STATUS_START://Download begins
            printf("DownloadCallBackFun: ECALLBACK UPDATE-STATUS-START: Download start \n");
            break;
        case ECALLBACK_UPDATE_STATUS_PROGRESS://Download progress. Code as percentage (0-100)
            printf("DownloadCallBackFun: ECALLBACK UPDATE-STATUS-PROGRESS:%% d \n", code);

```

```
break;
```

```
case ECALLBACK_UPDATE_STATUS_RESULT://Feedback results and errors
```

```
if (0<=code)&&(code<=6)//Success
```

```
{
```

```
if (code==0) {//Download the Offset template
```

```
printf("DownloadOffsetCBFun:%s\n", "Download Offset template! \n");
```

```
}
```

```
else if (code==1) {//Download the Gain template
```

```
printf("DownloadOffsetCBFun:%s\n", "Download Gain template! \n");
```

```
}
```

```
else if (code==2) {//Download the Defect template
```

```
printf("DownloadOffsetCBFun:%s\n", "Download Defect template! \n");
```

```
}
```

```
else if (code==3) {//Offset template download completed
```

```
printf("DownloadOffsetCBFun:%s\n", "Offset template download completed! \n");
```

```
}
```

```
else if (code==4) {//Gain template download completed
```

```
printf("DownloadOffsetCBFun:%s\n", "Download gain finish! \n");
```

```
}
```

```
else if (code==5) {//Defect template download completed
```

```
printf("DownloadOffsetCBFun:%s\n", "Defect template download completed! \n");
```

```
}
```

```
else/* if (code==6) */{//Download successful and exit thread
```

```
printf("DownloadOffsetCBFun:%s\n", "Download success and exit thread! \n");
```

```
}
```

```
}
```

```
else//Failed
```

```
{
```

```
if (code=-1){
```

```
printf("err: DownloadOffsetCBFun:%s\n", "wait event other error! \n");
```

```
}
```

```
else if (code=-2){
```

```
printf("err: DownloadOffsetCBFun:%s\n", "timeout! \n");
```

```
}
```

```
else if (code=-3){
```

```
printf("err: DownloadOffsetCBFun:%s\n", "downloaded offset failed! \n");
```

```
}
```

```
else if (code=-4){
```

```
printf("err: DownloadOffsetCBFun:%s\n", "downloaded gain failed! \n");
```

```
}
```

```
else if (code=-5){
```

```
printf("err: DownloadOffsetCBFun:%s\n", "downloaded defect failed! \n");
```

```
}
```

```
else if (code=-6){
```

```

        printf("err: DownloadOffsetCBFun:%s\n", "Download failed and exit thread!\n");
    }
    else if (code==7){
        printf("err: DownloadOffsetCBFun:%s\n", "read offset failed!\n");
    }
    else if (code==8){
        printf("err: DownloadOffsetCBFun:%s\n", "read gain failed!\n");
    }
    else if (code==9){
        printf("err: DownloadOffsetCBFun:%s\n", "read defect failed!\n");
    }
    else{
        printf("err: DownloadOffsetCBFun: unknown error,%d\n", code);
    }
}
break;
default://unusual
    printf("DownloadCallBackFun: unusual, retcode=%d\n", code);
    break;
}
return 1;
}

```

5.3.6 Update calibration enable

After completing the calibration template, in order to prevent the automatic cancellation of calibration enable during the template generation process, please update the calibration enable again after the template generation process is completed.

```
IMAGE_CORRECTION_ENABLE * pccorrect=new IMAGE_CORRECTION_ENABLE;
```

```
if (pccorrect!=NULL)
```

```
{
```

```
    pccorrect ->bFeedbackCfg=true// Feedback firmware parameters to the upper computer, false only returns confirmation information
```

```
    pccorrect ->ucOffsetCorrection=0x03// 0x00- no calibration, 0x01- software offset, 0x02- firmware pose offset, 0x03- firmware pre offset offset, dynamic tablet uses firmware pre offset, which is 0x03
```

```
    pccorrect ->ucGainCorrection=0x02// 0x00- no calibration, 0x01- software gain, 0x02- firmware gain, but the gain template needs to be downloaded to the tablet first
```

```
    pccorrect ->ucDefectCorrection=0x02// 0x00- No calibration, 0x01- Software defect, 0x02- Firmware defect, but the defect template needs to be downloaded to the tablet first
```

```
    pccorrect ->ucDummyCorrection=0x00// 0x00- No calibration, 0x01- Currently not supported, 0x02- Firmware currently not supported, currently not supported by software firmware
```

```
    ret=HBI_UpdateCorrectEnable (theDoc ->m_pFPD, pccorrect);
```

```
    If (ret==0)
```

```
        // successful
```

```
    else
```

```
        // failed
```



DT Imaging Technology (Shanghai) Co., Ltd

```
//Release resources
```

```
Delete correct;
```

```
Pccorrect=NULL;
```

```
}
```

In addition, some parameters can affect the calibration template, such as Binning type, PGA level, frame rate, high pressure dose, and ZOOM parameters, all of which can cause the template to fail. Therefore, if the above parameters are changed, it is best to update the calibration template to prevent loss of calibration.

5.4 Integration of multiple modes

Users often experience "anomalies" in the current image calibration due to switching between several (1 or more) tablet control parameters. The specific reason is that the calibration template does not match (i.e., the phenomenon of image misalignment). The most direct way to solve this problem is to regenerate the calibration template file under the current tablet control parameter conditions. However, the process of generating templates often requires certain skills and relevant professional knowledge,

In order to cooperate with the multi template management of firmware calibration (or hardware calibration), the latest DLL supports user multi mode management, making it convenient for users to quickly switch between different mode items. Mode items synchronize and control detector image acquisition in real-time based on parameters. In addition, the correction and parameter setting process is simplified. For details, please refer to the "DT SDK Image Calibration User Mode and Multi Template Calibration Management Integration Explanations. pdf".

What are the main contents of multi template management?

1》 Enable multiple Mode management status parameters, which can be distinguished and compatible with Binning storage methods;

2》 The maximum number of Mode list items is 10, which can support 10 different templates. Often, customers do not need to manage many templates. The number of Mode items can be dynamically managed, and interface functions for modification are provided;

3》 The data of the Mode item can be edited and updated. The Mode item includes parameters such as Mode ID, Binning type, PGA level, dose value, ZOOM area parameters, and template status (all valid, all invalid, dark and bright field template status);

4》 Mode item switching, template downloading, etc.

The following mainly describes the process of multi template calibration management and integration from two steps: user mode update and user mode switching. User mode update is a prerequisite for user mode switching, which includes modifying mode item parameters, switching mode items, generating mode item templates, and downloading mode item templates. In short, switching the Mode option is only effective after the user Mode update is completed. As shown in Figure 5-4-1:

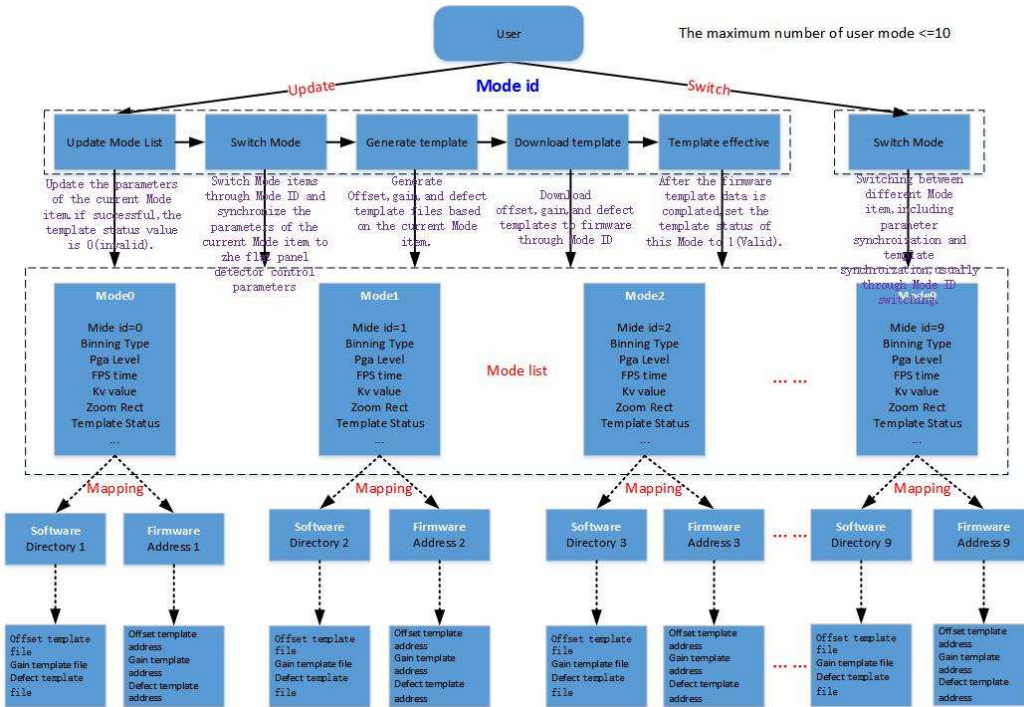


Figure 5-4-1

Tip: Multi mode management must correspond to the firmware of the tablet detector. Early firmware did not support multi mode management. Currently, common detectors that support multi mode management include 1613-125um, 1917-120um, and 1717-111um. Before integrated development, it should be confirmed with after-sales or refer to the user manual instructions.

5.4.1 Steps

- 1》 Include header files in user engineering


```
#Define_DLL_EX_IM 0
#include "HbiFpd.h"//Dynamic library header file
#pragma comment (lib, "HBISDKApi. lib")
```
- 2》 User defined devices


```
// Device handles and configurations
Void * m_pFPD// Fpd sdk object handle
RegCfgInfo * m_pLastRegCfg// Firmware configuration
FPD_AQC_MODE aqc_mode// Acquisition mode and parameters, single frame acquisition, multi frame acquisition, dark field image and bright field image acquisition
// Image Resolution
int m_imgW, m_imgH;
char m_path[MAX_PATH];
fpd_base_cfg_st *fpd_base;
IMAGE_PROPERTY img_pro;
IMAGE_CORRECT_ENABLE* pCorrect;
// Some basic data types
```



```
DT Imaging Technology (Shanghai) Co., Ltd  
m_pLastRegCfg = new RegCfgInfo;  
pCorrect = new IMAGE_CORRECT_ENABLE;  
fpd_base = new fpd_base_cfg_st;
```

The device list structure has been defined in HbiType. h

3» Interface call process

```
// Initialize DLL resources  
Void * m_pFPD=HBI_Init(0);  
If (m_pFPD==NULL){  
    printf("HBI_Init init failed! \n");  
    Return false;  
}  
// Registering Callbacks function  
//MyCallbackFun callback function  
int ret=HBI_RegEventCallBackFun (m_pFPD, myCallbackFun);  
If (ret!=0){  
    printf("HBI_RegEventCallBackFun failed! \n");  
    HBI_Destroy (m_pFPD);  
    M_PFPD=NULL;  
    Return false;  
}  
//Connect the flat panel detector  
COMM_CFG commCfg;  
Bool doOffsetTemps=false;// If the firmware is not used as an offset template by default  
//1.PCIE mode  
CommCfg._type=FPD_COMM_TYPE: PCIE_COMM_TYPE;  
CommCfg._pcieId =7024// The default single panel is 7024,7024 is the PCIE card hardware ID  
#If 0  
//2.UDP mode  
//commCfg._type = FPD_COMM_TYPE:: UDP_COMM_TYPE;  
//strcpy(commCfg._localip, "192.168.10.20");  
//strcpy(commCfg._remoteip, "192.168.10.40");  
//commCfg._localPort = 32896;  
//commCfg._remotePort = 32897;  
//3.UDP JUMBO  
//commCfg._type = FPD_COMM_TYPE:: UDP_JUMBO_COMM_TYPE;  
//strcpy(commCfg._localip, "192.168.10.20");  
//strcpy(commCfg._remoteip, "192.168.10.40");  
//commCfg._localPort = 32896;  
//commCfg._remotePort = 32897;  
#Endif
```

//DoOffsetTemplate - false: The firmware does not use an offset template after a successful connection,



DT Imaging Technology (Shanghai) Co., Ltd

true: The firmware uses an offset template after a successful connection

```
ret=HBI_ConnectDetector (m_pFPD, commCfg, doOffsetTemps);
```

//ret=HBI_Connect Detector (m_pFPD, commCfg)// After successful connection, the firmware does not create an offset template

```
if (ret!=0){  
    printf("Connection failed! \n");  
    HBI_Destroy (m_pFPD);  
    m_pFPD =NULL;  
    return false;  
}
```

// Switch Mode option, modify trigger mode, and enable image correction

```
DETECTOR_INIT_PARAMS param;
```

```
param.ucmodeid=0// Mode id [0,9];
```

para.uctriggerType=7// 1-Soft trigger, 3-High voltage trigger, 4-FreeAED, 5-Dynamic: Hvg Sync Mode, 6-Dynamic:

FpdSync Mode, 7-Dynamic: FpdContinue, 8-SAEC;

//Only valid in Binning mode, invalid in Multi Mode mode because the Mode item already contains trigger mode parameters to be compatible with different versions.

para.m_ucOffsetCorrection=0x03; //0-No Offset Correction;1-Software PreOffset Correction;2-Firmware PostOffset Correction;3-Firmware PreOffset Correction;

```
param._ucGainCorrection=0x02; //0-No Gain Correction;1-Software Gain Correction;2-Firmware Gain Correction;
```

```
param._ucDefectCorrection=0x02; //0-No Defect Correction;1-Software Defect Correction;2-Firmware Defect
```

Correction;

param._ucDummyCorrection=0x00; //0-No Dummy Correction;1-Software Dummy Correction;2-Firmware Dummy Correction;

```
int ret=HBI_InitDetectorParams (m_pFPD, inparam);
```

```
if (ret != HBI_SUCCSS)
```

```
{  
    if (ret == HBI_INVALID_DLL_HANDLE) printf("error:m_pFpd is NULL!");  
    else if (ret == HBI_ERR_DISCONNECT) printf("error:disconnect!\n");  
    else if (ret == HBI_ERR_CFG_IEMPTY) printf("error:m_pRomCfg is NULL!\n");  
    else if (ret == HBI_ERR_INVALID_PARAMS) printf("error:invalid params!\n");  
    else if (ret == HBI_ERR_NOT_SUPPORT) printf("warning:not support mutil-mode!");  
    else if (ret == HBI_ERR_ROM_PARAMETER) printf("warning:mode number or mode id!\n");  
    else if (ret == HBI_ERR_INVALID_MODE)printf("error:Mode prammeter is abnormality!\n");  
    else if (ret == HBI_ERR_MALLOC_FAILED) printf("error:malloc failed!\n");  
    else if (ret == HBI_ERR_SENDDATA_FAILED) printf("error:send failed!\n");  
    else printf("err:Apply the mode failed!\n");  
}
```

```
else
```

```
{  
    printf("HBI_InitDetectorParams success!\n");  
}
```

```
// disconnect
```

```
HBI_DisconnectDetector (m_pFPD);
```



DT Imaging Technology (Shanghai) Co., Ltd

// free resource (includedisconnect and free resource)

HBI_Destroy (m_pFPD);

m_pFPD =NULL;

Note:

- 1》 After successful connection, the tablet will automatically provide feedback on ROM parameters.
- 2》 'HBI_Init' and 'HBI_Destroy': initializes and releases device;
- 3》 HBI_ConnectDetector and HBI_DisconnectDetector: Connect and disconnect the flat detector;
- 4》 Connect the detector using 'HBI_ConnectDetector', callback event ECALLBACK_TYPE_ROM_UPLOAD provides feedback on the current firmware parameters. The basic information here has been fixed and users can use it directly;
- 5》 'HBI_GetSystemConfig' and 'HBI_GetFpdCfgInfo': All read detector firmware parameters
'HBI_GetSystemConfig' is an asynchronous function that sends a request to the firmware to obtain parameters; 'HBI_GetFpdCfgInfo' is a synchronization function that obtains parameters after a successful connection or setting.
- 6》 '**HBI_InitDetectorParams**', f After a successful connection, initialize the basic control parameters, such as trigger mode and calibration enable;
- 7》 HBI_Destroy releases resources, handle is NULL, if closed directly, call 'HBI_Destroy' is sufficient, The call to 'HBI_DisConnectDetector' is already included in 'HBI_Destroy'.
- 8》 Define callback functions

#####

#Callback function and event description:

//Function Description: Callback function definition, handling configuration, images, and tablet return status.

//Parameter Description:

// @pContext: Parameter 1, upper computer object pointer, can be null

// @ufpdId: Parameter 2, such as tablet ID

// @byteEventId: Parameter 3, Event ID, refer to enum eCallbackEventCommType in 'HbiType.h'

//@pvParam1: Parameter 4, configure structure pointer or image structure pointer

// @nParam2: Parameter 5, such as data size (image data is the length of the image buffer) or state

// @param3: Parameter 6, such as frame number, frame ID

// @param4 Parameter 7, such as frame rate, frame rate, or state, etc

//Callback function ends. Return value description: The return values are integers 0 and 1, 1- Received message, 0- Not received message

#####

```
int CDemoDlg:: myCallbackFun (void * _context, int nfpdId, unsigned char command, void * pvParam1, int nParam2,
int param3, int param4)
```

```
{
    int ret=0;
    if ((command == ECALLBACK_TYPE_SINGLE_IMAGE) ||
        (command == ECALLBACK_TYPE_MULTIPLE_IMAGE) ||
        (command == ECALLBACK_TYPE_ROM_UPLOAD))
    {
        if (pvParam1 == NULL || nParam2 == 0)
        {
            printf("Registration callback function parameter exception! \n");
            return ret;
        }
    }
}
```

```

    }
}
//
int status=-1;
int j=0;
ret=1;
switch (command)
{
case ECALLBACK_TYPE_FPD_STATUS://Tablet status: connected/disconnected/ready/busy
    printf("ECALLBACK TYPE NET-ERR-MSG, recode=%d \n", len);
    break;
case ECALLBACK_TYPE_ROM_UPLOAD:/* Update configuration*/
    printf("ECALLBACK TYPE ROM_UPLOAD: \n");
    break;
case ECALLBACK_TYPE_SET_CFG_OK:
    printf("ECALLBACK_TYPE_SET_CFG_OK:\n");
    break;
case ECALLBACK_TYPE_SINGLE_IMAGE://Single frame capture image
case ECALLBACK_TYPE_MULTIPLE_IMAGE://Continuous collection of images
    break;
case ECALLBACK_OVERLAY_NUMBER://Overlay progress number, such as stacking 8 frames, now stacking up to 5 frames
    break;
case ECALLBACK_OVERLAY_16BIT_IMAGE://After stacking is completed, update the stacking result graph (16 bits)
    break;
.....
case ECALLBACK_TYPE_THREAD_EVENT:
default:
    printf("command=%02x\n", command);
    break;
}
return ret;
}

```

Return the image properties, length, width, number of bits, type, and size information, trigger mode, firmware calibration enable, etc. through callback functions.

Return the image data address and length through a callback function. If other parameters are required, please explain.

In addition, template generation and image calibration are also important parts of code integration. Next, we will focus on template generation, template download, and updating calibration enablement. The progress status callback function can be found in the section 5.2.5 Progress Status Callback Function.

5.4.2 Multi Mode Management

It is currently known that the combination of detector parameters in the following table will have a significant impact on the image calibration template, as listed below.

Calibration Type	Detector Size	Binning Type	PGA Level	FPS Time	Dose (Kv)	Zoom Rect
Offset	√	√	√	√	×	√
Gain	√	√	√	×	√	√
Defect	√	√	√	×	√	√

Detector Size: Different detector models have different calibration templates. At present, the generated templates will be classified and stored according to **different detector** models, and downloaded by searching and matching the detector models.

The remaining parameters are for the **same detector internally**, as different parameters affect the calibration template.

Offset template (dark field correction): Binning Type, PGA Level (Pga level), frame rate (continuous image time interval), and Zoom Rect parameters affect the offset template;

Gain template (gain correction): The parameters of Binning Type, PGA Level, Dose (dose Kv Value), and Zoom Rect affect the Gain template;

Defect template (bad point correction): Binning Type, PGA Level (Pga level), Dose (Kv Value), and Zoom Rect parameters affect the Defect template;

In addition, **rotation and mirroring** also have a certain impact on the template, but by setting the panel rotation and mirroring parameters, the image will be rotated and mirrored.

5.4.2.1 Enabling Multi-Mode Management

The main two interface functions needed are 'HBI_FirmwareManagementTempalteType' and 'HBI_SetValidModeItemNumber', where 'HBI_FirmwareManagementTempalteType' enables multiple mode management and writes the enable/disable status to the firmware basic parameter table. The firmware can support up to 10 sets of calibration templates, The interface 'HBI_SetValidModeItemNumber' can dynamically set the number of valid Mode items.

❖ Enable or disable multiple modes

```

unsigned int uSelect=1// Enable
//unsigned int uSelect = 0; // 0 or other means disabled, default to using binning method
int ret=HBI_FirmwareManagementTempalteType (m_pFPD, (unsigned int) select1);
if (ret!=HBI_SUCCSS)
{
    AfxMessageBox (_T ("error: HBI_FirmwareManagementTempalteType failed!"));
}
else
{
    AfxMessageBox (_T ("HBI_FirmwareManagementTempalteType success!"));
}

```

❖ Set the number of valid Mode items

```

int nModeNumber=4// For example, setting 4 sets of Mode items must be greater than 0
int ret=HBI_SetValidModeItemNumber (m_pFPD, nModeNumber);
If (ret!=HBI_SUCCSS)
{
    AfxMessageBox (_T ("error: HBI_SetValidModeItemNumber failed!"));
}

```

```

}
else
{
    AfxMessageBox(_T("HBI_SetValidModeItemNumber success!"));
}

```

Note: The dynamic library is compatible with both the firmware Binning management template method and the multi-mode management template method. However, in order to better manage different management methods, the above two interfaces are added to synchronize firmware parameters for management. The dynamic library will distinguish between types and management templates based on firmware parameters.

5.4.2.2 Updating Mode List

In order to avoid synchronization and mismatch between software, firmware, and calibration template files, we introduce a Mode List (User Mode List), which can create n ($n \in [1, 10]$) user Mode item members according to user needs. Each Mode item member contains all template dependent parameters, such as Mode ID, binning type, PGA level, frame rate time, Kv value, Zoom Rect, template status identifier, and so on. The Mode List is stored in the basic parameter list of the tablet firmware. After the tablet is successfully connected, the basic parameter list of the tablet firmware is automatically synchronized to the DLL and upper computer software. The Mode List data of the tablet firmware basic parameter list can also be modified through the DLL API. Users can switch to different user Mode options through the current Mode ID. The current tablet control parameters will be automatically synchronized based on the attribute parameters of the current Mode, which is fast and convenient!

Note: Currently, it supports a maximum of 10 Mode items, which means it can support up to 10 sets of calibration templates. The number of valid Mode items can be modified. Please refer to section 5.3.2.1 "Enabling Multiple Mode Management" for information on enabling and setting the number of valid Mode items.

Mode Id	Binning Type	PGA Level	FPS Time (ms)	Prepare Time (ms)	Voltage (Kv)	Zoom Rect(l,t,w,h)	Trigger Mode	Template Status(Valid)
Mode - 0	1x1	7.2pC	50	1000	0	0,0,0,0	7-Dynamic:Fpd Continue	Yes
Mode - 1	2x2	1.2pC	50	1000	0	0,0,0,0	7-Dynamic:Fpd Continue	Yes
Mode - 2	1x1	2.4pC	50	1000	0	0,0,0,0	7-Dynamic:Fpd Continue	Yes
Mode - 3	1x1	7.2pC	100	1000	0	0,0,0,0	7-Dynamic:Fpd Continue	No
Mode - 4	2x2	3.6pC	33	0	0	0,0,0,0	7-Dynamic:Fpd Continue	No
Mode - 5	1x1	7.2pC	1500	1500	0	0,0,0,0	7-Dynamic:Fpd Continue	No
Mode - 6	1x1	7.2pC	1000	2000	0	0,0,0,0	7-Dynamic:Fpd Continue	No
Mode - 7	1x1	4.8pC	1000	1200	0	0,0,0,0	7-Dynamic:Fpd Continue	No
Mode - 8	1x1	9.6pC	1000	0	0	0,0,0,0	7-Dynamic:Fpd Continue	No
Mode - 9	2x2	4.8pC	500	500	0	0,0,0,0	7-Dynamic:Fpd Continue	No

In order to facilitate the upper computer to better obtain the Mode list information, the Mode list file can be exported through the XDiscovery tool in the following format:

###hbi_id0_mode_list.ini file###

[MODE-LIST] # Mode list

Panel_Size=1 # detector model

Mode_Number=4 # The default total number of mode items is 10, and the current number of valid Mode items is 4, which can be modified by the user

Mode_Curid=0 # The current Mode ID is 0

[MODE-0] # Mode ID 0 Information

Binning_Type=1 # binning type



DT Imaging Technology (Shanghai) Co., Ltd

```
Fps_Time=50 # frame rate time
PGA_Level=6 # pga gear
Do_Kv=0 # Dose High Pressure
Zoom_Left=0 # Zoom rect left
Zoom_Top=0 # Zoom rect top
Zoom_Width=0 # Zoom rect width
Zoom_Height=0 # Zoom rect height
Trigger_Mode=7 # Trigger Mode
Prepare_Time=1000 # prepare delay
Enbale_Status=1 # Is the template invalid? 0- Dark and bright fields are invalid, 1- Only dark fields are valid, 2- Only bright fields are
valid, 3- Dark and bright fields are valid, and others - Dark and bright fields are invalid
[MODE_1] # Mode ID 1 Information
Binning_Type=2
Fps_Time=50
PGA_Level=2
Do_Kv=0
Zoom_Left=0
Zoom_Top=0
Zoom_Width=0
Zoom_Height=0
Trigger_Mode=7
Prepare_Time=1000
Enbale_Status=1
```

...

Tip: Users can pre configure multiple Mode lists based on their actual business needs. After configuration is completed, they only need to switch between different Mode items, generate corresponding templates under different Mode items, and download the templates to the firmware. The firmware will store the templates in different addresses according to the download parameters (currently stored according to Mode ID).

It is recommended to update the Mode list through the XDiscovery tool's Multi Mode Management view page, and individual customers can also update the corresponding interface functions by calling them. For the list of XDiscovery configuration modes, please refer to the relevant chapters in the XDiscovery software operation manual.

Below we will introduce the use of interface functions to update the Mode list.

function	parameter	Remarks
HBI_UpdateModeList	FPD_MODE_LIST* modelist; typedef struct fpd_mode_list_st { unsigned char m_byCurModeId; FPD_MODE_DATA m_stModeArray[MODE_LIST_TOTAL]; }FPD_MODE_LIST;	Update the Mode List interface function to support up to 10 sets of Mode items. You can set the number of valid Mode items to n, and set it to update the content of the previous n sets of Mode items. Note that each Mode item parameter must be legal, otherwise the update may fail due to check failure.
HBI_UpdateCurModeData	FPD_MODE_DATA* mode; typedef struct fpd_mode_data_st { unsigned char m_byBinningType; unsigned int m_unFpsTime; unsigned char m_byPgaLevel;	Update the current Mode item data, including binning type, PGA gear, high pressure dose, ZOOM area parameters, trigger mode, prepare delay, and template status (dark field

	unsigned short m_usKvValue; unsigned short m_sZoomRectLeft; unsigned short m_sZoomRectTop; unsigned short m_sZoomRectWidth; unsigned short m_sZoomRectHeight; unsigned char m_byFpdTrigger; unsigned int m_unPrepareTime; unsigned char m_byTempStatus; } FPD_MODE_DATA;	valid, bright field valid, all valid, all invalid)
HBI_UpdateModeData	FPD_MODE_DATA* mode; unsigned int id = 0;	Update the Mode item Data through the Mode ID. This interface function is similar to HBI_UpdateCurModeData, but the Mode ID requires input and can be used to update any Mode item data.

// Update Mode List

```

FPD_MODE_LIST modelist;
modelist.m_byCurModeId = usermode.m_byCurModeId;
memcpy(modelist.m_stModeArray, usermode.m_stModeList, sizeof(FPD_MODE_DATA) * MODE_LIST_TOTAL);
//Get the number of valid Mode items
unsigned int useize = HBI_GetModeListSize(m_pFpd);
if(useize > MODE_LIST_TOTAL)
{
    ::AfxMessageBox(_T("err:HBI_GetModeListSize failed!"));
    return;
}

//Default parameter update
if (modelist.m_byCurModeId < 0 || modelist.m_byCurModeId >= useize )
{
    modelist.m_byCurModeId = 0;
}
//
BYTE bytePanelSize = theDoc->m_pRegRomCfg->m_SysBaseInfo.m_byPanelSize;
//Automatically update other parameters to default values
for (int i = 0; i < useize; i++)
{
    if (modelist.m_stModeArray[i].m_byFpdTrigger <= 0 || modelist.m_stModeArray[i].m_byFpdTrigger > 8)
        modelist.m_stModeArray[i].m_byFpdTrigger = 7;
    //
    if (modelist.m_stModeArray[i].m_byBinningType < 1 || modelist.m_stModeArray[i].m_byBinningType > 4)
        modelist.m_stModeArray[i].m_byBinningType = 1;
    //Pga Level
    if (bytePanelSize == 0x0a ||
        bytePanelSize == 0x0b ||
        bytePanelSize == 0x12 ||
        bytePanelSize == 0x13 ||

```

```
bytePanelSize == 0x14 ||
bytePanelSize == 0x15)

{
    if (modelist.m_stModeArray[i].m_byPgaLevel != 8 && modelist.m_stModeArray[i].m_byPgaLevel != 9)
        modelist.m_stModeArray[i].m_byPgaLevel = 8;
}
else
{
    if (modelist.m_stModeArray[i].m_byPgaLevel < 1 || modelist.m_stModeArray[i].m_byPgaLevel > 11)
    {
        modelist.m_stModeArray[i].m_byPgaLevel = 6;
    }
    else
    {
        if (modelist.m_stModeArray[i].m_byPgaLevel == 8 || modelist.m_stModeArray[i].m_byPgaLevel == 9)
            modelist.m_stModeArray[i].m_byPgaLevel = 8;
        }
    }
}
//
if (modelist.m_stModeArray[i].m_unFpsTime <= 0)
    modelist.m_stModeArray[i].m_unFpsTime = 1000;
//
if (modelist.m_stModeArray[i].m_sZoomRectWidth == 0)
{
    if (modelist.m_stModeArray[i].m_sZoomRectLeft != 0)
        modelist.m_stModeArray[i].m_sZoomRectLeft = 0;
}
//
if (modelist.m_stModeArray[i].m_sZoomRectHeight == 0)
{
    if (modelist.m_stModeArray[i].m_sZoomRectTop != 0)
        modelist.m_stModeArray[i].m_sZoomRectTop = 0;
}
}
//
int ret = HBI_UpdateModeList(m_pFpd, &modelist);
if (HBI_SUCCSS != ret)
{
    ::AfxMessageBox(_T("err:HBI_UpdateModeList failed!"));
}
else
{
    ::AfxMessageBox(_T("HBI_UpdateModeList success!"));
}
```



//Update current list items

```
FPD_MODE_DATA user_mode;
user_mode.m_byBinningType = 1;
user_mode.m_unFpsTime = 50;
user_mode.m_unPrepareTime = 1500;
user_mode.m_byPgaLevel = 6;
user_mode.m_sZoomRectLeft = 0;
user_mode.m_sZoomRectTop = 0;
user_mode.m_sZoomRectWidth = 0;
user_mode.m_sZoomRectHeight = 0;
user_mode.m_usKvValue = 0;
user_mode.m_byFpdTrigger = 7;
user_mode.m_byTempStatus = 0;
//
int ret = HBI_UpdateCurModeData(theDoc->m_pFpd, &user_mode);
if (ret != HBI_SUCCSS)
{
    ::AfxMessageBox(_T("err:HBI_UpdateCurModeData failed!"));
}
else
{
    ::AfxMessageBox(_T("err:HBI_UpdateCurModeData success!"));
}
```

//Update list items through Mode ID

unsigned int nModeItem=1;// Update the second Mode entry, Mode ID $\in [0,9]$, but must be less than the number of valid Mode entries

```
FPD_MODE_DATA user_mode;
user_mode.m_byBinningType = 1;
user_mode.m_unFpsTime = 50;
user_mode.m_unPrepareTime = 1500;
user_mode.m_byPgaLevel = 6;
user_mode.m_sZoomRectLeft = 0;
user_mode.m_sZoomRectTop = 0;
user_mode.m_sZoomRectWidth = 0;
user_mode.m_sZoomRectHeight = 0;
user_mode.m_usKvValue = 0;
user_mode.m_byFpdTrigger = 7;
user_mode.m_byTempStatus = 0;
//
int ret = HBI_UpdateModeData(theDoc->m_pFpd, &user_mode, nModeItem);
if (ret != HBI_SUCCSS)
{
    ::AfxMessageBox(_T("err:HBI_UpdateModeData failed!"));
}
```

```

}
else
{
    ::AfxMessageBox(_T("err:HBI_UpdateModeData success!"));
}

```

5.4.2.3 Switching Mode Items

After the user finishes editing and updating the Mode item, call the Mode item through the Mode ID, synchronize the Mode item parameters to the panel collection control parameters, and complete the Mode item switch.

Interface functions	Interface functions	Parameter Description
Switch Mode	HBI_SwitchUserMode(void* handle, unsigned int modeid = 0, FPD_MODE_DATA *PMODEDATA=NULL);	Unsigned int modeid identifies the specified Mode item, FPD_MODE-DATA * * PMODEDATA, updates the parameters of this Mode item. PMODEDATA can be null (mode item parameters are not updated, only switch). If it is not null, update the Mode item parameters and switch to the Mode item. The myEnable parameter can be defaulted, and the software will automatically update it.

Example:

```

// switch to Mode 2
unsigned int umodeid=2;
int ret=HBI_SwitchUserMode (m_pFPD, umodeid, NULL/*&modedata */);
If (ret!=HBI_SUCCSS)
{
    AfxMessageBox (_ T ("err: HBI_SwitchUserMode failed!"));
}
else
{
    AfxMessageBox (_ T ("HBI_SwitchUserMode success!"));
}

```

5.4.2.4 Calibration template generation and download

The user forms an effective strong correlation mapping relationship with multiple templates (template generation dependency parameters) through the Mode parameter. By switching through the Mode option, the user first synchronizes the current tablet control parameters, and then switches the corresponding calibration template (existing) to the application (associated generation template, download template, and switch template, etc.), achieving a perfect match and fit between the template and the tablet control scene, and ultimately achieving better calibration results. As shown in Figure 5-3-2-1.

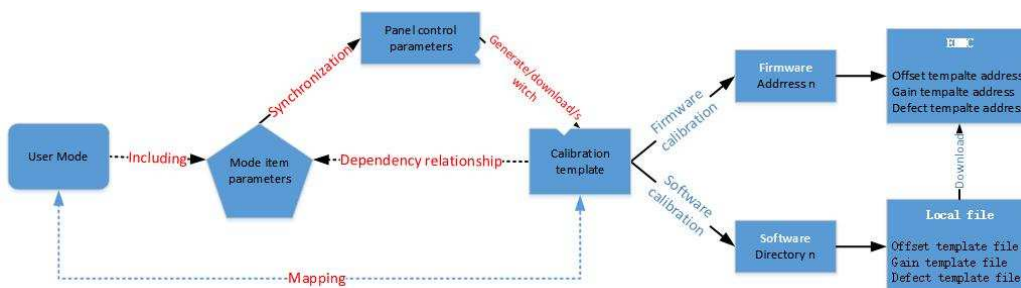


Figure 5-3-2-1

Steps:

Step 1: After the user has edited and updated the Mode item, call the Mode item through the Mode ID, synchronize the Mode item parameters to the tablet collection control parameters, and complete the Mode item switch.

Step 2: After switching the Mode option, generate different types of templates (offset, gain, defect) in the current tablet control parameter scene, and associate the calibration template with the current Mode option scene.

Interface functions	Interface functions	Parameter Description
One click quick generation of calibration templates	HBI_GenerateTemplate(void *handle, EnumIMAGE_ACQ_CMD _mode, int bpreview);	EnumIMAGE_ACQ_CMD _mode, continuous collection command, enumeration class, select the type of generated template, as follows // One-key quick generation mode OFFSET_TEMPLATE_TYPE, // Quick generation process of offset template-Firmware generation offset template GAIN_TEMPLATE_TYPE, // Quick generation process of gain template with 1 group image DEFECT_TEMPLATE_GROUP1, // Quick generation process of defect template - group1. DEFECT_TEMPLATE_GROUP2, // Quick generation process of defect template - group2. DEFECT_TEMPLATE_GROUP3, // Quick generation process of defect template - group3. SOFTWARE_OFFSET_TEMPLATE, // Quick generation process of offset template - SDK generates offset template int bpreview - Whether to generate a preview template, 1-generated, 0-not generated.

Note: After generating the template, it is best to update the image correction enable state, as the image correction enable state needs to be cancelled during the template generation process, except for the firmware offset correction enable state.

Step 3: For firmware calibration, the corresponding template file needs to be downloaded to the firmware emmc in advance. The download must rely on the current Mode parameter (mode ID), and the firmware will write the calibration template to the specified address based on the parameters.

Interface functions	Interface functions	Parameter Description
Download template based on Mode parameter	HBI_DownloadModeTemplate(void* handle, DOWNLOAD_MODE_FILE downloadfile);	DOWNLOAD_MODE_FILE downloadfile, download template matching downloadfile information download
Download correction template files according to template type (default to current Mode Id)	HBI_DownloadModeTemplateByType(void* handle, int intemptype)	int intemptype, template type
Download correction template files according to Mode Id and template type	HBI_DownloadModeTemplateByIdAndType(void* handle, unsigned int modeid, int intemptype)	int modeid, template id int intemptype, template type
Download correction template (default to current Mode Id)	HBI_DownloadTemplate(void *handle, DOWNLOAD_FILE *downloadfile);	DOWNLOAD_FILE *downloadfile, download the template file to the current mode, and the Mode ID is the current default Mode ID.
Download the specified correction template file by default according	HBI_DownloadTemplateByType(void *handle, int infiletype);	int intemptype, template type



to the type (default to the current Mode ID)		
--	--	--

Data type DOWNLOAD_MODE_FILE:

//Template file attributes

// template file attributes

typedef struct download_mode_file_st

```

{
    download_mode_file_st()
    {
        m_byModelId    = 0;
        m_byBinningType = 1;
        m_byPgaLevel    = 6;
        m_unFpsTime     = 0;
        m_usKvValue     = 0;
        m_sZoomRectLeft = 0;
        m_sZoomRectTop  = 0;
        m_sZoomRectWidth = 0;
        m_sZoomRectHeight = 0;
        emtemplatetype  = GAIN_TEMPLATE;
    };
    unsigned char m_byModelId;    // New, specify Mode ID
    unsigned char m_byBinningType;
    unsigned char m_byPgaLevel;
    unsigned int m_unFpsTime;
    unsigned short m_usKvValue;
    unsigned short m_sZoomRectLeft;
    unsigned short m_sZoomRectTop;
    unsigned short m_sZoomRectWidth;
    unsigned short m_sZoomRectHeight;
    emTEMPLATE_TYPE emtemplatetype;
}DOWNLOAD_MODE_FILE;

```

Step 4: After generating and downloading templates for all Mode items, users can complete image calibration in different modes by switching Mode items.

Please refer to sections 5.4.3 to 5.4.6 for template generation and template download code.

5.4.3 Offset Template Generation and Download

// eg, Generate a template for Mode 2 and download the template

//Firstly, connect the tablet detector and switch the Mode option. Once successful, the template can be generated in that Mode.

```

unsigned int umodeid = 2;
int ret = HBI_SwitchUserMode(m_pFpd, umodeid, NULL/*&modedata*/);
if (ret != HBI_SUCCSS)
{
    AfxMessageBox(_T("err:HBI_SwitchUserMode failed!"));
}

```



```

}
else
{
    AfxMessageBox(_T("HBI_SwitchUserMode success!"));
}

```

```

// Collect 1 set of dark field images and generate a template until success is returned, otherwise failed is returned.
EnumIMAGE_ACQ_MODE enumTemplateType=EnumIMAGE_ACQ_MODE::OFFSET_TEMPLATE_TYPE;
int ret = HBI_GenerateTemplate(m_pFpd, enumTemplateType);
if (ret != HBI_SUCCSS) {}// failed
else {} //success
/*

```

The offset template generated by the firmware generally does not need to be downloaded. In some scenarios, the high-pressure ball tube continuously emits X-rays, and the software needs to generate calibration in advance and download it to the firmware.

```

*/
// Registering Callbacks
//Callback function: USER_CALLBACK_HANDLE_PROCESS handFun - Global function or static member function, refer to HbiType.
h
//pContext - Pointer to the upper computer object, which can also be assigned as null if not needed by the user.
if (0 != HBI_RegProgressCallBack(m_pFpd, DownloadCallBackFun, (void *)this))
{
    printf("err:HBI_RegProgressCallBack failed!\n");
    return;
}
// parameters
//DOWNLOAD_MODE_FILE *downloadfile=NULL;
//if (downloadfile == NULL) downloadfile = new DOWNLOAD_MODE_FILE;
//if (downloadfile == NULL)
//{
//    printf("err:malloce downloadfile failed!\n");
//    return;
//}
//Example of Mode Item Data
//downloadfile->m_byModelId    = (BYTE)umodeid;
//downloadfile->m_byBinningType = 1;
//downloadfile->m_byPgaLevel   = 7;
//downloadfile->m_unFpsTime    = 20;
//downloadfile->m_usKvValue     = 0;
//downloadfile->m_sZoomRectLeft = 0;
//downloadfile->m_sZoomRectTop  = 0;
//downloadfile->m_sZoomRectWidth = 0;
//downloadfile->m_sZoomRectHeight = 0;
//downloadfile->emtemplatetype = OFFSET_TEMPLATE;
// method 1: start downloadthe template of Pre-offset

```



DT Imaging Technology (Shanghai) Co., Ltd

```
//int ret = HBI_DownloadModeTemplate(m_pFpd, downloadfile);
//if (ret != HBI_SUCCSS)
//    ::AfxMessageBox(_T("err:Download offset-template failed!"));
//else
//    ::AfxMessageBox(_T("Download offset-template success!"));
// free resource
//Delete pUploadFile;
//PUploadFile=NULL;
or
// method 2: start downloadthe template of Pre-offset
int inftype=2// Download file types 0-gain template, 1-defect template, 2-offset template, others - not supported
int ret=HBI_DownloadModeTemplateByType (d_pFPD, inftype)// Suggested use
If (ret!=HBI_SUCCSS)
    : AfxMessage    Box (_T ("err: Download offset template failed!"));
else
    : AfxMessage    Box (_T ("Download offset template success!"));

// Update offset calibration enable and enable firmware Pre offset calibration
IMAGE_CORRECT_ENABLE *pcorrect = new IMAGE_CORRECT_ENABLE;
if (pcorrect != NULL)
{
    pcorrect->bFeedbackCfg = false;
    pccorrect ->ucOffsetCorrection=0x03// 0x00- no calibration, 0x01- software offset, 0x02- firmware pose offset, 0x03- firmware
pre offset offset, dynamic tablet uses firmware pre offset, which is 0x03
    pccorrect ->ucGainCorrection=0x00// 0x00- no calibration, 0x01- software gain, 0x02- firmware gain, but the gain template needs
to be downloaded to the tablet first
    pccorrect ->ucDefectCorrection=0x00// 0x00- No calibration, 0x01- Software defect, 0x02- Firmware defect, but the defect
template needs to be downloaded to the tablet first
    Pccorrect ->ucDummyCorrection=0x00// 0x00- No calibration, 0x01- Currently not supported, 0x02- Firmware currently not
supported, currently not supported by software firmware
    ret=HBI_UpdateCorrectEnable (theDoc ->m_pFPD, pcorrect);
    if (ret==0)
        // success
    else
        // failed
    //Release resources
    delete correct;
    pccorrect=NULL;
}
```

5.4.4 Gain template generation and download

// eg, Generate a template for Mode 2 and download the template

Firstly, connect the tablet detector and switch the Mode option. Once successful, a template can be generated in that Mode. If it has already been switched, there is no need to repeat the switch.



DT Imaging Technology (Shanghai) Co., Ltd

```
unsigned int umodeid=2;
int ret = HBI_SwitchUserMode(m_pFpd, umodeid, NULL/*&modedata*/);
if (ret != HBI_SUCCSS)
{
    AfxMessageBox(_T("err:HBI_SwitchUserMode failed!"));
}
else
{
    AfxMessageBox(_T("HBI_SwitchUserMode success!"));
}
```

Collect 1 set of bright field images, with constant high voltage and normal milliampere seconds. Here, a high voltage generator is required to cooperate. Turn on the high voltage before calling the interface, and trigger the command to complete template generation after the light tube radiation stabilizes.

```
EnumIMAGE_ACQ_MODE enumTemplateType = EnumIMAGE_ACQ_MODE::GAIN_TEMPLATE_TYPE;
int ret = HBI_GenerateTemplate(m_pFpd, enumTemplateType);
if (ret!=HBI_SUCCSS) {}//failed
else {}//Success
```

```
// Registering Callbacks
//Callback function: USER_CALLBACK_HANDLE_PROCESS handFun - Global function or static member function, refer to HbiType.
h
// pContext- Upper computer object pointer, if not needed by the user, can also be assigned as null.
if (0 != HBI_RegProgressCallBack(m_pFpd, DownloadCallBackFun, (void *)this))
{
    printf("err:HBI_RegProgressCallBack failed!\n");
    return;
}
// parameter
//DOWNLOAD_MODE_FILE *downloadfile =NULL;
//if (downloadfile == NULL) downloadfile = new DOWNLOAD_MODE_FILE;
//{
//    printf("err:malloc downloadfile failed!\n");
//    return;
//}
//Example of Mode Item Data
//downloadfile->m_byModelId    = (BYTE)umodeid;
//downloadfile->m_byBinningType  = 1;
//downloadfile->m_byPgaLevel    = 7;
//downloadfile->m_unFpsTime      = 20;
//downloadfile->m_usKvValue      = 0;
//downloadfile->m_sZoomRectLeft  = 0;
//downloadfile->m_sZoomRectTop    = 0;
//downloadfile->m_sZoomRectWidth = 0;
```



DT Imaging Technology (Shanghai) Co., Ltd

```
//downloadfile->m_sZoomRectHeight = 0;
//downloadfile->emtemplatetype = GAIN_TEMPLATE;
// method 1 :start downloadthe template of Gain
//int ret = HBI_DownloadModeTemplate(m_pFpd, downloadfile);
//if (ret != HBI_SUCCSS)
//    ::AfxMessageBox(_T("err:Download gain-template failed!"));
//else
//    ::AfxMessageBox(_T("Download gain-template success!"));
or
// method 2 :start downloadthe template of Gain
int infiletype=0// Download file types 0-gain template, 1-defect template, 2-offset template, others - not supported
int ret=HBI_DownloadModeTemplateByType(d_pFPD, infiletype)// Suggested use
If (ret!=HBI_SUCCSS)
    : AfxMessageBox (_ T ("err: Download gain template failed!"));
else
    : AfxMessageBox (_ T ("Download gain template success!"));

//Update gain calibration enable and enable firmware gain calibration
IMAGE_CORRECT_ENABLE *pcorrect = new IMAGE_CORRECT_ENABLE;
if (pcorrect != NULL)
{
    pcorrect->bFeedbackCfg = false;
    pccorrect ->ucOffsetCorrection=0x03// 0x00- no calibration, 0x01- software offset, 0x02- firmware pose offset, 0x03- firmware
pre offset offset, dynamic tablet uses firmware pre offset, which is 0x03
    pccorrect ->ucGainCorrection=0x02// 0x00- no calibration, 0x01- software gain, 0x02- firmware gain, but the gain template needs
to be downloaded to the tablet first
    pccorrect ->ucDefectCorrection=0x00// 0x00- No calibration, 0x01- Software defect, 0x02- Firmware defect, but the defect
template needs to be downloaded to the tablet first
    pccorrect ->ucDummyCorrection=0x00//
    ret=HBI_UpdateCorrectEnable (theDoc ->m_pFPD, pcorrect);
    if (ret==0)
        // success
    else
        // faild
    //Release resources
    delete correct;
    pccorrect=NULL;
}
```

5.4.5 Defect template generation and download

// example, Generate a template for Mode 2 and download the template

Firstly, connect the tablet detector and switch the Mode option. Once successful, a template can be generated in that Mode. If it has already been switched, there is no need to repeat the switch.

```
unsigned int umodeid = 2;
```



DT Imaging Technology (Shanghai) Co., Ltd

```
int ret = HBI_SwitchUserMode(m_pFpd, umodeid, NULL/*&modedata*/);
```

```
if (ret != HBI_SUCCSS)
```

```
{
```

```
    AfxMessageBox(_T("err:HBI_SwitchUserMode failed!"));
```

```
}
```

```
else
```

```
{
```

```
    AfxMessageBox(_T("HBI_SwitchUserMode success!"));
```

```
}
```

1》 Normal high voltage, milliampere second adjustment normal 10%

```
EnumIMAGE_ACQ_MODE enumTemplateType = EnumIMAGE_ACQ_MODE::DEFECT_ACQ_GROUP1;
```

```
int ret = HBI_GenerateTemplate(m_pFpd, enumTemplateType);
```

```
if (ret!=HBI_SUCCSS) {}//failed
```

```
else {}//Success
```

Note: As it is a firmware pre offset template, there is no need to download it to the firmware again.

2》 Normal high voltage, milliampere second adjustment normal 50%

```
EnumIMAGE_ACQ_MODE enumTemplateType = EnumIMAGE_ACQ_MODE::DEFECT_ACQ_GROUP2;
```

```
int ret = HBI_GenerateTemplate(m_pFpd, enumTemplateType);
```

```
if (ret!=HBI_SUCCSS) {}//failed
```

```
else {}//Success
```

3》 Normal high voltage, milliampere second adjustment normal

```
EnumIMAGE_ACQ_MODE enumTemplateType = EnumIMAGE_ACQ_MODE::DEFECT_ACQ_GROUP3;
```

```
int ret = HBI_GenerateTemplate(m_pFpd, enumTemplateType);
```

```
if (ret!=HBI_SUCCSS) {}//failed
```

```
else {}//Success
```

```
/*
```

```
* After the template is successfully generated, start downloading the template
```

```
*/
```

```
// Registering Callbacks
```

```
//Callback function: USER_CALLBACK_HANDLE_PROCESS handFun - Global function or static member function, refer to HbiType.
```

```
h
```

```
//PContext - Pointer to the upper computer object, which can also be assigned as null if not needed by the user.
```

```
if (0 != HBI_RegProgressCallBack(m_pFpd, DownloadCallBackFun, (void *)this))
```

```
{
```

```
    printf("err:HBI_RegProgressCallBack failed!\n");
```

```
    return;
```

```
}
```

```
// 参数
```

```
//DOWNLOAD_MODE_FILE *downloadfile =NULL;
```

```
//if (downloadfile == NULL) downloadfile = new DOWNLOAD_MODE_FILE;
```

```
//if (downloadfile == NULL)
```



DT Imaging Technology (Shanghai) Co., Ltd

```
//{
//  printf("err:malloce downloadfile failed!\n");
//  return;
//}

//Example of Mode Item Data
//downloadfile->m_byModeId    = (BYTE)umodeid;
//downloadfile->m_byBinningType = 1;
//downloadfile->m_byPgaLevel   = 7;
//downloadfile->m_unFpsTime    = 20;
//downloadfile->m_usKvValue     = 0;
//downloadfile->m_sZoomRectLeft = 0;
//downloadfile->m_sZoomRectTop  = 0;
//downloadfile->m_sZoomRectWidth = 0;
//downloadfile->m_sZoomRectHeight = 0;
//downloadfile->emtemplatetype = DEFECT_TEMPLATE;

// method 1 :start download the template of defect
//int ret = HBI_DownloadModeTemplate(m_pFpd, downloadfile);
//if (ret != HBI_SUCCSS) {
//  ::AfxMessageBox(_T("err:Download defect template failed!"));
//}
//else
//  ::AfxMessageBox(_T("Download defect template success!"));
// 释放资源
//Delete pUploadFile;
//PUploadFile=NULL;

or

// method 2 :start download the template of defect
int infiletype=1// Download file types 0-gain template, 1-defect template, 2-offset template, others - not supported
int ret=HBI_DownloadModeTemplateByType (m_pFPD, infiletype)// Suggested use
if (ret != HBI_SUCCSS)
  ::AfxMessageBox(_T("err:Download defect-template failed!"));
else
  ::AfxMessageBox(_T("Download defect-template success!"));
//Update defect calibration enable and enable firmware defect calibration
IMAGE_CORRECT_ENABLE *pcorrect = new IMAGE_CORRECT_ENABLE;
if (pcorrect != NULL)
{
  pcorrect->bFeedbackCfg = false;
  pccorrect ->ucOffsetCorrection=0x03// 0x00- no calibration, 0x01- software offset, 0x02- firmware pose offset, 0x03- firmware
pre offset offset, dynamic tablet uses firmware pre offset, which is 0x03
  pccorrect ->ucGainCorrection=0x02// 0x00- no calibration, 0x01- software gain, 0x02- firmware gain, but the gain template needs
to be downloaded to the tablet first
  pccorrect ->ucDefectCorrection=0x02// 0x00- No calibration, 0x01- Software defect, 0x02- Firmware defect, but the defect
template needs to be downloaded to the tablet first
```



DT Imaging Technology (Shanghai) Co., Ltd

pccorrect ->ucDummyCorrection=0x00// 0x00- No calibration, 0x01- Currently not supported, 0x02- Firmware currently not supported, currently not supported by software firmware

```
ret=HBI_UpdateCorrectEnable (theDoc ->m_pFPD, pccorrect);
if (ret==0)
    // success
else
    // failed
//Release resources
delete correct;
pccorrect=NULL;
}
```

5.4.6 Progress Status Callback Function

//Download template callback function: static member function or global function

//InContext: Upper computer object pointer, can be NULL.

Static int DownloadCallBackFun (unsigned char command, int code, void * inContext)

```
{
    ///CTemplateTool *ptr = (CTemplateTool *)inContext;
    ///if (ptr == NULL) {
    ///    printf("err:inContext is NULL!\n");
    ///    return 0;
    ///}
    ///if (theDemo == NULL) {
    ///    printf("err:theDemo is NULL!\n");
    ///    return 0;
    ///}
    int len = 0;
    switch (command)
    {
    case ECALLBACK_UPDATE_STATUS_START://Download begins
        printf("DownloadCallBackFun: ECALLBACK_UPDATE_STATUS_START: Download start \n");
        break;
    case ECALLBACK_UPDATE_STATUS_PROGRESS://Download progress. Code as percentage (0-100)
        printf("DownloadCallBackFun: ECALLBACK_UPDATE_STATUS_PROGRESS:%% d \n", code);
        break;
    case ECALLBACK_UPDATE_STATUS_RESULT://Feedback results and errors
        if (0<=code)&&(code<=6)//Success
        {
            if (code==0) { //Download the Offset template
                printf("DownloadOffsetCBFun:% s \n", "Download Offset template! \n");
            }
            else if (code==1) { //Download the Gain template
                printf("DownloadOffsetCBFun:% s \n", "Download Gain template! \n");
            }
        }
    }
```

```

else if (code==2) { //Download the Defect template
    printf("DownloadOffsetCBFun:% s \n", "Download Defect template! \n");
}
else if (code==3) { //Offset template download completed
    printf("DownloadOffsetCBFun:% s \n", "Offset template download completed! \n");
}
else if (code==4) { //Gain template download completed
    printf("DownloadOffsetCBFun:% s \n", "Download gain finish! \n");
}
else if (code==5) { //Defect template download completed
    printf("DownloadOffsetCBFun:% s \n", "Defect template download completed! \n");
}
else/* if (code==6) */{ //Download successful and exit thread
    printf("DownloadOffsetCBFun:% s \n", "Download success and exit thread! \n");
}
}
else//Failed
{
    if (code==1){
        printf("err: DownloadOffsetCBFun:% s \n", "wait event other error! \n");
    }
    else if (code==2){
        printf("err: DownloadOffsetCBFun:% s \n", "timeout! \n");
    }
    else if (code==3){
        printf("err: DownloadOffsetCBFun:% s \n", "downloaded offset failed! \n");
    }
    else if (code==4){
        printf("err: DownloadOffsetCBFun:% s \n", "downloaded gain failed! \n");
    }
    else if (code==5){
        printf("err: DownloadOffsetCBFun:% s \n", "downloaded defect failed! \n");
    }
    else if (code==6){
        printf("err: DownloadOffsetCBFun:% s \n", "Download failed and exit thread! \n");
    }
    else if (code==7){
        printf("err: DownloadOffsetCBFun:% s \n", "read offset failed! \n");
    }
    else if (code==8){
        printf("err: DownloadOffsetCBFun:% s \n", "read gain failed! \n");
    }
    else if (code==9){
        printf("err: DownloadOffsetCBFun:% s \n", "read defect failed! \n");
    }
}

```

```

    }
    else{
        printf("err: DownloadOffsetCBFun: unknown error,%d \n", code);
    }
}
break;
default://unusual
    printf("DownloadCallBackFun: unusual, retcode=%d \n", code);
    break;
}
return 1;
}

```

5.4.7 Status of calibration template

This refers to the offset and gain/defect template states (valid or invalid) stored on the firmware in the current mode, as follows:

- ◆ When the Binning Type, PGA Level (Pga level), frame rate (continuous image time interval), and Zoom Rect change, the Dark Field (Offset) template will become invalid. The solution is to update the current Offset template, including updating the generated and downloaded status to valid;
- ◆ When the Binning Type, PGA Level, Dose Kv Value, and Zoom Rec change, the bright field (gain and defect) templates will become invalid. The solution is to update the current gain and defect templates, including updating the generated and downloaded status to valid;
- ◆ Set template association parameters or download templates, and the template status will be synchronized.

Data type `FPD_MODE_DATA`:

//Notice: fpd mode data information: 22

```

typedef struct fpd_mode_data_st
{

```

```

    ...

```

unsigned char m_byTempStatus; //01//0-Both dark and bright field templates are invalid, //1-Only dark field templates are valid, 2-Only the bright field template is valid, 3-Both dark and bright field //templates are valid, others-Both dark and bright field templates are invalid

```

}FPD_MODE_DATA;

```

Template status table:

Status value	describe
0	Both dark field (offset) and bright field (gain and defect) are invalid
1	The dark field (offset) is valid, while the bright field (gain and defect) is invalid
2	Dark field (offset) invalid, bright field (gain and defect) valid
3	Both the dark field (offset) and the bright field (gain and defect) are valid

Add template status feedback to the main callback event:

```

int handleCommandEventA (void * pvContext, int nDevId, unsigned char byteEvnetid, void * pvParam1, int nParam2, int nParam3,
int nParam4)

```

```

{

```

```

    ...

```

```

    case ECALLBACK_FIRMWARE_TEMPLATE_STATUS:

```

```

        { //nParam2 represents the current Mode ID, and nParam3 represents the template status information of the Mode, as

```

shown in the Template Status Table above.



DT Imaging Technology (Shanghai) Co., Ltd

```
printf("ECALLBACK_FIRMWARE_TEMPLATE_STATUS:CurModelId=%d,nTempStatus=%d", nParam2, nParam3);
}
break;
...
}
```

5.4.8 Update calibration enable

After completing the calibration template, in order to prevent the automatic cancellation of calibration enable during the template generation process, please update the calibration enable again after the template generation process is completed.

```
IMAGE_CORRECT_ENABLE *pcorrect = new IMAGE_CORRECT_ENABLE;
```

```
if (pcorrect != NULL)
```

```
{
```

```
    pccorrect->bFeedbackCfg=true// Feedback firmware parameters to the upper computer, false only returns confirmation information
```

```
    pccorrect->ucOffsetCorrection=0x03// 0x00- no calibration, 0x01- software offset, 0x02- firmware pose offset, 0x03- firmware pre offset offset, dynamic tablet uses firmware pre offset, which is 0x03
```

```
    pccorrect->ucGainCorrection=0x02// 0x00- no calibration, 0x01- software gain, 0x02- firmware gain, but the gain template needs to be downloaded to the tablet first
```

```
    pccorrect->ucDefectCorrection=0x02// 0x00- No calibration, 0x01- Software defect, 0x02- Firmware defect, but the defect template needs to be downloaded to the tablet first
```

```
    pccorrect->ucDummyCorrection=0x00// 0x00- No calibration, 0x01- Currently not supported, 0x02- Firmware currently not supported, currently not supported by software firmware
```

```
    ret=HBI_UpdateCorrectEnable (theDoc ->m_pFPD, pcorrect);
```

```
    if (ret==0)
```

```
        // success
```

```
    else
```

```
        // failed
```

```
    //release resources
```

```
    delete correct;
```

```
    pcorrect=NULL;
```

```
}
```

There are several situations where the calibration template for the Mode item needs to be regenerated and downloaded to the firmware.

1》 Updating the Binning Type, PGA Level (Pga level), frame rate (continuous image time interval), and Zoom Rect parameters affects the Offset template. These parameters will be automatically synchronized to the Mode item in the Mode list based on the current default mode ID, that is, modifying the parameters of the current Mode item;

2》 Updating the Binning Type, PGA Level (Pga level), Dose (Dose Kv Value), and Zoom Rect parameters affects the Gain and Defect templates. These parameters will be automatically synchronized to the Mode item in the Mode list based on the current default mode ID, which modifies the parameters of the current Mode item;

3》 During the template generation process, it may be necessary to cancel the image correction enable state during the template generation process, except for the firmware pre offset correction enable state. Therefore, it is best to update the template correction enable state parameters after completing the template;

4》 It is best for users to pre-set the Mode List parameters and generate templates based on each Mode item. They need to download and update firmware calibration templates to avoid mismatches between the current Mode item parameters and the template due to



DT Imaging Technology (Shanghai) Co., Ltd

adjusting individual template dependency parameters. Users can modify the Mode item by switching between the Mode items and controlling the parameters in real-time, which is advisable to prevent mismatches in the calibration template.

5.5 Problems and Suggestions

For example, if there is an abnormality in the PCIE method, please provide further information.

1> Multiple socket exceptions occurred with error codes 10049 and 100048

Specific reason: a. Port is occupied or IP address is incorrect. For example, if a program has already been opened, attempting to open a second program connection will result in an error code of 10049

b. Closing the socket resource without fully releasing and reconnecting, resulting in an error code of 10048.

Solution:

Avoid opening multiple programs that occupy the same port;

Confirm the disconnection failed before connecting again.

2> A socket exception occurred with error code 10040

Specific reason: Receive buffer overflow or insufficient space, the reason is that the firmware is a jumbo version and the software is using a non jumbo version, and the firmware and software versions do not match.

Solution:

Replace firmware or software versions according to customer requirements;

3> Socket exception, sending data failed with error code 10051

Specific reasons:

The local network address is incorrect;

Solution:

Check the configuration and make modifications.

4> Release and debug differ between dynamic and static library files

Recently, some users have reported that they have experienced crashes while developing release library files in debugging mode.

Based on this, they have investigated and verified the issue,

Specific reasons:

The incompatibility between the user's release library and the user's debug environment has resulted in the current release version being provided to the user;

Solution: If necessary, contact the R&D personnel to obtain the debug version library file.

5> Some users experience firmware parameter confusion in their development projects

The reasons: When setting parameters by the user, the firmware parameters are not synchronized with the upper computer parameters. When issuing parameters, abnormal parameters are issued, resulting in parameter confusion;

Solution: At present, it has been processed. For the convenience of user development, corresponding interface functions are provided, and the API functions are automatically synchronized internally. Users only need to modify some parameters.

6> Call HBI_After TriggerAndCorrectApply, wait for the tablet parameters to be updated before proceeding with HBI_GetFpdCfgInfo, otherwise the modified parameters will not be synchronized,

Because currently, we only consider the parameter settings successful after receiving normal feedback from the flat panel detector.

7> The process of log feedback is basically normal, but pay attention to controlling the process and simplifying the call process. If you need to provide feedback on firmware parameters, please contact DT R&D personnel.

8> Callback function usage

Taking VS as an example, it supports static global functions or static member functions, but there may be pointer access issues in individual user integrations

6. Image overlay

Image overlay can be divided into software overlay and firmware overlay. Overlay number greater than 0, but try to avoid stacking too many times, which may cause data overflow.

6.1 Software Overlay

Refer to the implementation of the continuous acquisition interface function, and set the parameters as follows:

For example, software overlays 8 frames.

```
aqc_mode.eAqcCmd = LIVE_ACQ_DEFAULT_TYPE;
aqc_mode.eLivetype = OVERLAY_16BIT_IMG; // overlap result is 16bit image
//aqc_mode.eLivetype = OVERLAY_32BIT_IMG; // overlap result is 32bit image
aqc_mode.ndiscard = 0;
aqc_mode.nframeid = 0;
aqc_mode.nAcqNumber = 8; // overlay number
aqc_mode.ngroupno = 0;
int ret = HBI_LiveAcquisition(m_pFpd, aqc_mode);
if (ret != HBI_SUCCSS)
{
    AfxMessageBox(_T("error:HBI_LiveAcquisition failed!"));
}
else
{
    AfxMessageBox(_T("HBI_LiveAcquisition success!"));
}
//The stacking process and results can be obtained through the main callback function.
case ECALLBACK_OVERLAY_NUMBER: //Overlay progress number, such as stacking 8 frames, now stacking up to 5 frames
...
break;
case ECALLBACK_OVERLAY_16BIT_IMAGE: //After stacking is completed, update the stacking result graph (16 bits)
...
break;
case ECALLBACK_OVERLAY_32BIT_IMAGE: //After stacking is completed, update the stacking result graph (32-bit)
...
break;
```

6.2 Firmware Overlay

By setting the firmware overlay frame number, if the frame number is greater than 0, the firmware overlay will not be performed. Please note that currently only some tablets are supported, and it is best to confirm with after-sales whether it is supported.

//eg: Firmware overlay 8 frames and return image

```

unsigned int noverlaynum = 8;
int ret = HBI_SetFirmwareOverlay(m_pFpd, noverlaynum);
if (ret != HBI_SUCCSS)
{
    AfxMessageBox(_T("error:HBI_SetFirmwareOverlay failed!"));
}
else
{
    AfxMessageBox(_T("HBI_SetFirmwareOverlay success!"));
}
//eg: Cancel image overlay
noverlaynum = 0;
ret = HBI_SetFirmwareOverlay(m_pFpd, noverlaynum);
if (ret != HBI_SUCCSS)
{
    AfxMessageBox(_T("error:HBI_SetFirmwareOverlay failed!"));
}
else
{
    AfxMessageBox(_T("HBI_SetFirmwareOverlay success!"));
}

```

7. Rotate and Mirror

Image rotation and mirroring are divided into software rotation/mirroring and firmware mirroring. At present, the firmware does not support rotation function, but supports various mirroring. If the image needs to be rotated and mirrored at the same time, it defaults to rotating first and then mirroring. Tip: It is best to use firmware to complete the mirroring operation, which is more efficient.

As shown in the table below:

Mirror	Rotate/ Software/Firmware	Left rotation 90 °	Rotate 90 ° to the right	Horizontal Mirror	mirror vertically	Mirror horizontally first and then vertically
Software (dynamic library)		√	√	√	√	×
firmware		×	×	√	√	√

Rotation and mirroring can be achieved by setting different parameters, as shown in the table below.

rotate	0- Do nothing;	Do not rotate
	1-Right Rotate 90 °;	Rotate the software (dynamic library) 90 ° to the right
	2-Left Rotate 90 °;	Software (Dynamic Library) Left Rotation 90 °
mirror	0- Do nothing;	Do not mirror
	1-Vertically mirror;	Software (dynamic library) vertical mirroring
	2-Horizontal mirror;	Horizontal mirroring of software (dynamic library)
	3-Firmware Vertically mirror;	Firmware vertical mirroring
	4-Firmware Horizontal mirror;	Firmware horizontal mirroring
	5-Firmware Vertically Horizontal mirror;	Firmware vertically and horizontally mirrored



DT Imaging Technology (Shanghai) Co., Ltd

The rotation and mirroring functions can also cause different calibration templates, especially before downloading to the firmware, it is necessary to reverse process the template files generated after rotation and mirroring, and then download the firmware. Currently, this function has been automatically recognized and processed internally in the software, so users do not need to pay attention.

7.1 Software Rotation and Mirroring

For example, the image is rotated 90 ° to the right and vertically mirrored through software.

```
int nRotate=1;
int nFlip=2;
int ret = HBI_SetImageRotateAndFlip(m_pFpd, nRotate, nFlip);
if (ret != HBI_SUCCSS)
{
    AfxMessageBox(_T("error:HBI_SetImageRotateAndFlip failed!"));
}
else
{
    AfxMessageBox(_T("HBI_SetImageRotateAndFlip success!"));
}
```

7.2 Firmware Mirroring

```
int nRotate=0;
int nFlip=3;
int ret = HBI_SetImageRotateAndFlip(m_pFpd, nRotate, nFlip);
if (ret != HBI_SUCCSS)
{
    AfxMessageBox(_T("error:HBI_SetImageRotateAndFlip failed!"));
}
else
{
    AfxMessageBox(_T("HBI_SetImageRotateAndFlip success!"));
}
```

8. Preview mode

By collecting points from the original data (Binning1*1) according to certain rules (currently it is 4*4, that is, picking one point from 16 points), a new data matrix is formed and uploaded to the host computer (for example, workstation), so that the host computer software can adjust the exposure dose through preview data analysis for software AEC mode.

The preview image matrix is 1/16 of the full resolution of the original data (width / 4, height / 4).

The preview setting functions: HBI_SetPreviewMode (void *handle, int inMode), where inMode is used to set the current image mode, 0-normal image, 1-preview image, 2-preview image + normal image (only supports single frame capture).



Application 1:

Upload preview and normal image separately. Upload preview image in 1-Preview image mode, and upload normal image in 0-normal image mode.

- 1> inMode=1, call HBI_SetPreviewMode, set it to Preview mode, and call “ECALLBACK_TYPE_ROM_UPLOAD” event feedback to update parameters after setting it successfully. “m_sImageHeight and m_sImageHeight” will be updated to the latest size, which is 1/4 of the original value.
- 2> Software command, high voltage signal trigger or exposure with AED mode will initiate one image acquisition process, the image data will be uploaded by “ECALLBACK_TYPE_PREVIEW_IMAGE”, supporting single frame and continuous capture;
- 3> After acquiring the preview image, cancel preview mode (inMode=0) and call HBI_SetPreviewMode. m_sImageHeight and m_sImageHeight will also be updated to the actual size;
- 4> Normal images will be acquired next time.
- 5> Preview image data is fed back to the computer via the “ECALLBACK_TYPE_PREVIEW_IMAGE” event (event no. 0X53) of the callback function. Normal image data is fed back to the computer through “ECALLBACK_TYPE_SINGLE_IMAGE (event no. 0X51)” or “ECALLBACK_TYPE_MULTIPLE_IMAGE” (event no. 0X52).

Application 2:

Upload preview image and normal image consecutively, that is, upload one frame of preview image and one frame of normal image at a time.

- 1> inMode=2, call HBI_SetPreviewMode, setting preview image + normal image mode, after setting success, call back ECALLBACK_TYPE_ROM_UPLOAD event feedback update parameter, m_sImageHeight and m_sImageHeight will be updated to the latest size, which is 1/4 of the original value.
- 2> Software command, high voltage signal trigger or exposure with AED mode will initiate one image acquisition process, the image data will be uploaded by “ECALLBACK_TYPE_PREVIEW_IMAGE”, supporting single frame and continuous capture;
- 3> If the preview image + normal image mode is not disabled, the single frame acquisition process will always follow the way of preview image + normal image.
- 4> Cancel preview image + normal image mode (inMode=0) and call HBI_SetPreviewMode. Setting m_sImageHeight and m_sImageHeight parameter will also update to the image to full resolution, and the next acquisition will be normal.
- 5> Preview image data is fed back to the host computer through the ECALLBACK_TYPE_PREVIEW_IMAGE event (event no. 0X53) of the callback function, and normal image data is fed back to the host computer through ECALLBACK_TYPE_SINGLE_IMAGE (event no. 0X51).

Finally, pay attention to the resolution of preview image and normal image, currently only support panel-2530-85um and panel-2530-75um and panel-3543-140um products.

The preview mode is mutually exclusive with the binning2*2, binning3*3, and binning4*4 modes. For the preview mode, the binning type must be binning1*1. Preview mode is not supported with the binning2*2, binning3*3, and binning4*4.

9. SAEC mode

In S-AEC exposure mode, exposure is divided into pre-exposure and normal exposure. The first frame is pre-exposed, and the system will adjust the exposure condition for the second exposure according to the pre-exposure image. The time of each exposure cannot exceed 30 seconds.

Note:

- 1> SAEC mode only supports panel-2530-85um detector;



DT Imaging Technology (Shanghai) Co., Ltd

2> The SAEC mode is trigger Mode =0x08;

3> Steps:

a. Call the “HBI_SAecModeApply” interface function, where preview 1 means upload preview image, otherwise 0 means upload normal image, m_nPerReadyTm, m_nPostReadyTm are respectively detector per ready delay time and detector post ready delay time;

b. Call the “HBI_SAecModeApply” interface function, where preview 0 means uploading normal images, and “m_nPerReadyTm” and “m_nPostReadyTm” non-key parameters default.

c. Call “HBI_SAecAcq” interface function to complete normal graph collection.

d. ID of the event in the callback function:

case ECALLBACK_TYPE_SINGLE_IMAGE: Normal images, for example Panel-2530-85um, The resolution defaults to width=2816, height=3584;

case ECALLBACK_TYPE_PREVIEW_IMAGE: preview image, such as Panel-2530-85um, the resolution defaults to:Width=2816/4=704, height=3584/4=896;

```
void CSAecModeTester::OnBnClickedBtnSaecAutoTest()
{
    // TODO: Add control notification handler code here
    this->UpdateData(TRUE);
    //
    if (!IsOpen) {
        AfxMessageBox(_T("err:detector is disconnect!"));
        return;
    }
    //
    if (m_pRegRomCfg == NULL) {
        AfxMessageBox(_T("err:m_pRegRomCfg!"));
        return;
    }
    //
    if (m_pRegRomCfg -> m_SysBaseInfo.m_byPanelSize != 0x05) {
        AfxMessageBox(_T("err:detector is not Pane2530-85um!"));
        return;
    }
    //
    if (m_nPerReadyTm * m_nPostReadyTm <= 0) {
        AfxMessageBox(_T("err:Ready Time is error!"));
        return;
    }
    // STEP1
    theDoc -> ReportError(_T("STEP1:SAEC Preview-Mode Start!"));
    int preview = 0x01;    // 01-Preview Image; 00-Normal Image;
    int ret = HBI_SAecModeApply(m_pFpd, preview, m_nPerReadyTm, m_nPostReadyTm);
    if (ret != HBI_SUCCSS) {
        AfxMessageBox(_T("err:Set SAEC Preview-Mode failed!"));
        return;
    }
}
```



DT Imaging Technology (Shanghai) Co., Ltd

```

}
ReportError(_T("STEP1: SAEC Preview-Mode Finished!"));

// Waiting to upload preview images
ReportError(_T("STEP1: SAEC Preview-Image!"));
// .....
ReportError(_T("STEP1: SAEC update hvg dose!"));
// Image change dose

// STEP2
ReportError(_T("STEP2:SAEC Normal-Mode Start!"));
preview = 0x00;      // 01-Preview Image; 00-Normal Image;
ret = HBI_SAecModeApply(m_pFpd, preview, m_nPerReadyTm, m_nPostReadyTm);
if (ret != HBI_SUCCSS) {
    AfxMessageBox(_T("err: Set SAEC Normal-Mode failed!"));
    return;
}
ReportError(_T("STEP2: SAEC Normal-Mode Finished!"));
// STEP3
ReportError(_T("STEP3: SAEC Acquisition Image Start!"));
ret = HBI_SAecAcq(m_pFpd);
if (ret != HBI_SUCCSS) {
    AfxMessageBox(_T("err:Send SAEC Acquisition Image!"));
    return;
}
ReportError(_T("STEP3:SAEC Acquisition Image Finished!"));
}

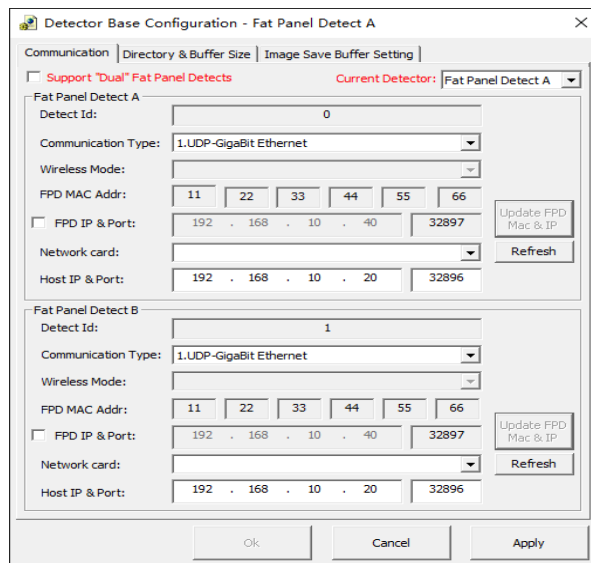
```

10. Modify Detetcor's IP

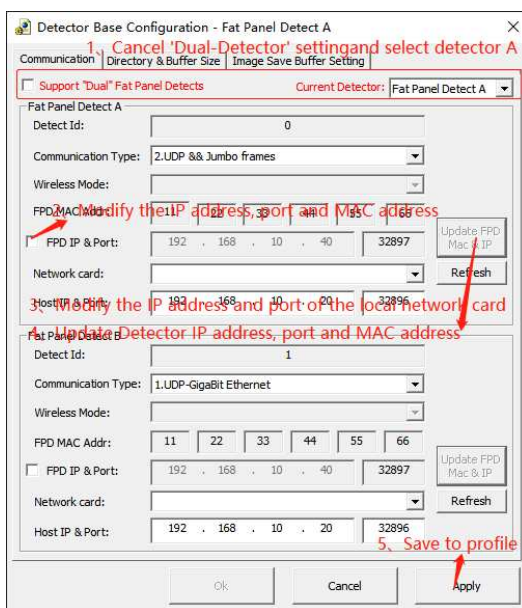
Based on network communication, tablets may have IP or MAC conflicts in actual production environments. Tablets support modifying IP and MAC addresses. The following is an introduction to how to modify tablet IP and MAC addresses.

Steps:

- 1》 Firstly, open the XDdiscover tool. The default IP address for the tablet is 192.168.10.40, and configure the local network card IP address to 192.168.10.20 to connect to the tablet;
- 2》 Open the main menu "Detector" ->select the submenu "Communication Configuration" ->pop up a dialog box "Detector Base Configuration" ->sub page "Communication";



3》 Modify the IP and MAC addresses, such as changing the tablet address to "192.168.20.40". The tablet MAC address generally does not need to be modified on a single board, but needs to be modified in a "dual board" solution to prevent IP conflicts;



4》 After setting up the tablet IP and MAC, disconnect the tablet connection, modify the local network card address, such as "192.168.20.20", connect to the tablet again, and test whether the modification is normal;

5》 Restore to factory settings, the tablet IP will be restored to the "192.168.10.40" network segment. If you need to modify the factory default IP, you can contact after-sales support for assistance;

Finally, when modifying the tablet IP and MAC address, please be careful not to modify them unless necessary. If you forget the IP or fail to connect, you will have to return to the company for processing.

11. Spot and perspective

None!

12. "Dual panels" solution

This version of the SDK supports static network port, dynamic network port, and PCIe optical port communication, and supports "dual panels" solutions composed of different communication methods, such as dual gigabit network cards or dual board solutions composed of gigabit network cards and PCIe cards, router or gigabit switch connection solutions. Contact after-sales personnel.

Compared to the single board solution, the dual panels solution requires the upper computer to distinguish and control the parameters and image data of different tablets. Other similar to "single panel", please refer to the API manual and SDK demo source code for details.

Steps:

1》 Initialize Dll: HBI_InitDual or HBI_InitEx;

After initialization, two instance handles are obtained, and the subsequent operations are all for instance operations. The instance handles automatically distinguish between tablets.

may also

HBI_Init (0) and HBI_Init (1)

2》 Register callback function: HBI_RegEventCallBackFun, register the callback functions for panel A and panel B respectively;

3》 Connecting tablet: HBI_ConnectDetector, connecting the callback functions of panel A and panel B respectively;

4》 Setting trigger mode and correction enable: HBI_TriggerAndCorrectDisplay;

5》 Other settings, obtaining parameters, etc;

6》 Disconnect tablet: HBI_DisconnectDetector;

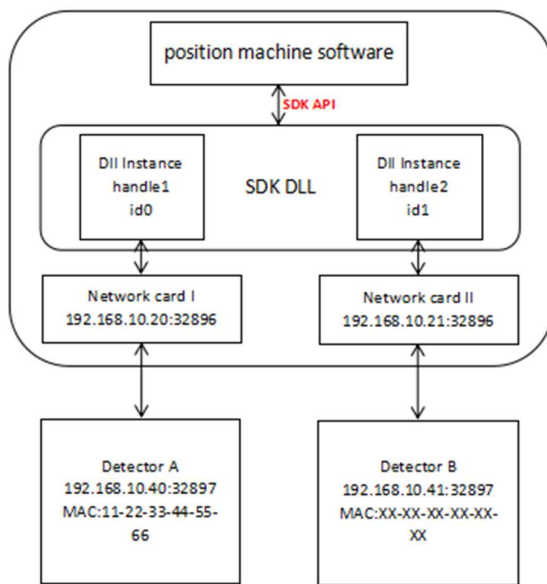
7》 Release Dll resources: HBI_DestroyEx;

Taking dual gigabit network cards as an example for "dual panels", as shown in the following figure:

1》 Panel A and B must have different MAC and IP addresses, and they must be tested one by one before integration;

2》 Network card A communicates with panel A, network card B communicates with panel B. Network card A and panel A are in the same network segment, network card B and panel B are in the same network segment, and network card A and network card B have different addresses, which can be in the same network segment or in different network segments;

3》 The default IP and MAC addresses for panel are 192.168.10.40 and 11-22-33-44-55-66. If you need to modify the IP and MAC addresses, please refer to the XDiscovery software operation manual.



If you need a router or switch to implement a dual panel network solution, you can contact after-sales or R&D personnel for further communication.

The "multi panels" solution has been well validated in some projects. If there is a need to integrate "multi panels", technical issues can be consulted with after-sales or R&D personnel. We will provide the demo source code engineering and guidance manual for the "multi panels" integration solution.

13. Develop Demo

For the convenience of users to quickly integrate and develop, we provide the SDK Demo project source code, as shown in the 'SDK_Demo_Example' directory "DT SDK Development DEMO Project Description Ver *. *. *. pdf" and "HB_SDK_DEMO2013 ". If there is a need to integrate a "dual panels" solution, please contact after-sales or R&D to request information on the "dual board" solution.

Number	SDK Demo	Remarks
1	HB_SDK_DEMO2013	Developed based on VS MFC, including interface, image display, and basic interface calls. Image display relies on Opencv341 (can be continuously sold or developed), C++engineering, including development instructions.
2	HBI_FPDCSharpdemo	Developed based on C # language, including interface, image display, and basic interface calls, C++engineering, including development instructions.
3	DemoDll	Developed based on VS console, unable to display images, including basic interface calls, using dynamic library calls, C++engineering.
4	DemoLib	Developed based on VS console, unable to display images, including basic interface calls, using static calls to dynamic libraries, C++engineering.
5	HB_SDK_DEMO2013_Two	Example of dual board solution, including interface, image display, and basic interface calls, C++engineering, including development instructions.
6	HB_SDK_DEMO2013_SIX	Example of a multi board solution, including interface, image display, and basic interface calls, capable of connecting up to 6 tablets simultaneously, C++engineering, including development instructions.
7	HBI_FPDCSharpdemo* en si	An example of a multi board solution, including interface, image

	x	display, and basic interface calls, can connect up to 6 tablets simultaneously, C # project.
8	hbi_python_demo	Python Demo project (relatively simple, currently suspended maintenance, needs to be further improved).
9	HB_SDK_QTDemo	Develop a demo based on the QT platform, including interface, image display, and basic interface calls, as well as a development manual.

Appendix I: Table of panel types and resolutions

Num	Panel size	Pixel Size	Resolution (actual width x height)	type	communication	Maximum frame rate (fps)
1	4343	140um	3072 x 3072	static state	Gigabit UDP	1. 2, 4, 8, 12, 24
2	3543	140um	2560 x 3072	static state	Gigabit UDP	Refer to the product manual
3	1613	125um	1248 x 1024	static state	Gigabit UDP	thirty
4	3030	140um	2048 x 2048	static state	Gigabit UDP	Refer to the product manual
5	2530	85um	2816 x 3584	Dynamic/Static	Gigabit Ethernet UDP/Fiber	1. 8, 16
6	3025	140um	2048 x 1792	static state	Gigabit UDP	Refer to the product manual
7	4343	100um	4288 x 4288	static state	Gigabit Ethernet UDP/Fiber	Refer to the product manual
8	2530	75um	3072 x 3840	static state	Gigabit UDP	Refer to the product manual
9	2121	200um	1024 x 1024	static state	Gigabit UDP	thirty
10	1412	50um	2784 x 2400	dynamic	Fiber optic	twenty-nine
11	0606	50um	1056 x 1200	dynamic	Fiber optic	fifty-one
12	2323	148um	1536 x 3072	dynamic	Fiber optic	40/80
13	2530	100um	2432 x 3072	Dynamic/Wireless	Gigabit Ethernet UDP/Fiber	Refer to the product manual
14	3030	148um	2048 x 2048	dynamic	Fiber optic	Refer to the product manual
15	3543	100um	3534 x 4302	dynamic	Gigabit Ethernet UDP/Fiber	Refer to the product manual
16	1616	105um	1536 x 1536	dynamic	Fiber optic	Refer to the product manual
17	43100	140um	7680 x 3072	Static/Dynamic	Gigabit Ethernet UDP/Fiber	Refer to the product manual
18	1515	100um	1070 x 1200	dynamic	Fiber optic	Refer to the product manual
19	2121	100um	2063 x 2049	dynamic	Fiber optic	Refer to the product manual
20	3131	100um	3095 x 3073	dynamic	Fiber optic	Refer to the product manual
21	2301	100um	2331 x 76	dynamic	Gigabit UDP	Refer to the product manual
22	1917	120um	1536, 1386	dynamic	Gigabit UDP	Refer to the product manual
23	3030	99um	3072, 3072	dynamic	Gigabit UDP	Refer to the product manual
24	1515	75um	2048, 2048	dynamic	Gigabit UDP	Refer to the product manual
25	2530	70um	3488, 4356	dynamic	Gigabit UDP	Refer to the product manual
26	2530	85um	3584, 3072	dynamic	Gigabit UDP	Refer to the product manual
27	4315	140um		dynamic	Gigabit UDP	Refer to the product manual
28	1313	85um	1536, 1536	dynamic	Gigabit UDP	Refer to the product manual
29	3030WO A	140um	2048, 2048	dynamic	Gigabit UDP	Refer to the product manual
30	4343WO A	140um	3072, 3072	dynamic	Gigabit UDP	Refer to the product manual
31	1818	84um	2048, 2048	dynamic	Gigabit UDP	Refer to the product manual
32	1717	111um	1536, 1536	dynamic	Gigabit UDP	Refer to the product manual
33	4343	100um	4302, 4302	dynamic	Gigabit UDP	Refer to the product manual

ROI related configurations and image resolution

Num	Panel Name	Resolution (w, h)	ROI (Zoom) mode	Interface functions
3	1613-125um	1248 * 1024	Mode 1: 32 * 1024 Mode 2: 1248 * 32 Mode 3: 1248 * 64 Mode 4: 1248 * 128	HBI_SetRoyMode
4	3030-140um	2048 * 2048	Mode 1: 1024 * 2048	HBI_SetRoyMode
7	4343-100um	4288 * 4288	Mode 1: 4288 * 800	HBI_SetRoyMode
13	2530-100um	2432 * 3072	Mode 1: 1808 * 1800 Mode 2: 1312 * 1300	HBI_SetRoyMode
10	CMOS-1412-50um	2784 * 2400	ZOOM area size can be set	HBI_SetZoomModeRect
11	CMOS-0606-50um	1056 * 1200	ZOOM area size can be set	HBI_SetZoomModeRect

PGA Level Number	PGA Level Value
1	0.6pC
2	1.2pC
3	2.4pC
4	3.6pC
5	4.8pC
6	7.2pC
7	9.6pC
8	LFW (COMS)
9	HFW (COMS)
10	0.3pC
11	0.15pC

Figure 12-1

Appendix II: Configuration Files

Connect the configuration file hbi_fpd_init.ini

[DLL_CONFIG]

COMM_ TYPE=2 # Communication type, 1-UDP jumbo, 2-PCIe optical port communication, other - UDP communication

FPD_ IP=192.168.10.40 # Tablet IP address, for Ethernet communication

FPD_ PORT=32897 # Tablet Port Number

LOCAL_ IP=192.168.10.20 # Local network card IP address

LOCAL_ PORT=32896 # Local network card port

TRIGGER_ MODE=7 # Trigger Mode

OFFSET_ ENABEL=3 # offset correction enable

GAIN_ ENABLE=2 # gain correction enable

DEFECT_ ENABLE=2 # defect correction enable

LIVE_ ACQ_ TIME=1000 # Continuous acquisition time interval (static tablet)

SELF_ DUMPING_ TIME=500 # Continuous acquisition time interval (dynamic tablet)

Prepare_ TIME=1500 # Single frame acquisition Prepare delay time interval

DO_ OFFSET=1 # Whether to create firmware pre offset template during connection

Appendix III: Callback Function Parameters and States

Callback function parameter table

//Notice: call back function

//@ USER_CALLBACK_HANDLE_EVENT

//@ pContext: Parameter 1, upper computer object pointer, can be null

//@ ufpdId: Parameter 2, such as tablet ID

//@ byteEventid: Parameter 3, Event ID, refer to enum eCallbackEventCommType in HbiType. h

//@ PVEventParam1: Parameter 4, configure structure pointer or image structure pointer

//@ nEventParam2: Parameter 5, such as data size or status



DT Imaging Technology (Shanghai) Co., Ltd

//@ nEventParam3: Parameter 6, such as frame number, frame ID

//@ nEventParam4: Parameter 7, such as frame rate or state, etc

Typedef int (* USER_CALLBACK_HANDLE_ENVENT) (void * pContext, int ufpdId, unsigned char byteEventId, void * PVEventParam1, int nEventParam2, int nEventParam3, int nEventParam4);

PContext: The object pointer passed in from the upper computer, which can be NULL.

Notes	UfpdId	ByteEventId	PVEvent Param1	NEvent Param2	NEvent Param3	NEvent Param4
Plate parameters	Instance ID	ECALLBACK_TYPE_ROM_UPLOAD	Non null pointer	Configuration parameter data length	invalid	invalid
Tablet status	Instance ID	ECALLBACK_TYPE_FPD_STATUS	Null pointer	state	invalid	invalid
Successfully set parameters	Instance ID	ECALLBACK_TYPE_SET_CFG_OK	Null pointer	invalid	invalid	invalid
Single frame image	Instance ID	ECALLBACK_TYPE_SINGLE_IMAGE	Image structure pointer	Image buffer length	Frame number	Frame rate
Continuous image	Instance ID	ECALLBACK_TYPE_MULTIPLE_IMAGE	Image structure pointer	Image buffer length	Frame number	Frame rate
Offset template data	Instance ID	ECALLBACK_TYPE_OFFSET_TMP	Image structure pointer	Image buffer length	Frame number	Frame rate
	Instance ID	ECALLBACK_TYPE_PREVIEW_IMAGE	Image structure pointer	Image buffer length	Frame number	Frame rate
Number of stacking processes	Instance ID	ECALLBACK_OVERLAY_NUMBER	Null pointer	Stacking counter	Frame number	Frame rate
16 bit stacked data	Instance ID	ECALLBACK_OVERLAY_16BIT_IMAGE	Image structure pointer	Image buffer length	Frame number	Frame rate
32-bit overlay data	Instance ID	ECALLBACK_OVERLAY_32BIT_IMAGE	Image structure pointer	Image buffer length	Frame number	Frame rate
Generate Template	Instance ID	ECALLBACK_TYPE_GENERATE_TEMPLATE	Null pointer	Progress and Results	Template type	invalid
...	...	...	...	...	...	...

Appendix IV: Switching from old version to the latest version

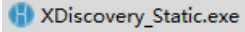
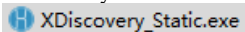
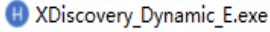
The ultimate goal of the **comprehensive version** is to standardize interface functions, parameter types, etc., simplify the integration process and solve the problem of multiple integrations, ultimately saving labor costs.

The current version supports tablet types (classified by communication method): Ethernet UDP, Ethernet UDP Jumbo Frame, PICE Optical Port, WLAN 2.4g/5g. Corresponding to different firmware versions, it can be divided into static and dynamic versions based on frame rate and transmission speed. Ethernet UDP and WLAN 2.4g/5g belong to low frame rate versions, while Ethernet UDP Jumbo packet and PICE optical ports belong to high frame rate versions.

So what are the differences in integration or usage between these versions? Below, we will provide a detailed explanation of the similarities and differences between these versions. See Appendix 4.

For some users who have previously integrated our network port static or dynamic versions, if they want to switch to the current integrated version, they often ask a question: what are the similarities and differences between the integrated version and the current integrated version? Here is a unified reply:

(1) What versions have been released so far

Name	Dynamic Library	Tool software	Remarks
Network port static version	HBI_FPD.DLL HBI_FPD.lib	XDiscovery_Static.exe 	1 frame per second, as the basic version for research and development, the longest and most challenging version, with a gigabit network card and software calibration, applied to medical or pet workstations;
Network interface static Jumbo frame version	HBI_FPD.DLL HBI_FPD.lib	XDiscovery_Static.exe 	Using Jumbo packet communication is more efficient than general communication, but the gigabit network card must be able to support Jumbo packet, be hardware calibrated, and require downloading templates to firmware. It is commonly used in point machines.
Network port dynamic version	HBI_FPD_E.DLL HBI_FPD_E.Lib	XDiscovery_Dynamic_E.Exe 	Model 4343 has a resolution of 3072 * 3072 frames per second, while Model 1613 has a resolution of 1248 * 1024 frames per second. It is mainly used for hardware calibration and is commonly used for online machine detection.
Comprehensive version	HBISDKApi.dll HBISDKApi.lib	XDiscovery.exe 	Supports dynamic/static tablets, tablets with different communication methods, compatible with previous product features, and tablets with added optical ports and wireless communication methods, resulting in less memory usage and higher efficiency.

(2) Why switch to the comprehensive version

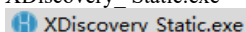
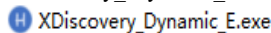
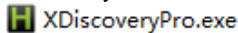
Four simple points to summarize:

- 1>This version has the advantage of being a latecomer, as the development of the first two versions started after they were basically stable. Drawing on the strengths and weaknesses of the first two versions, the functional modules were extensively validated, the framework was also restructured and optimized, and the interfaces were more standardized. Therefore, it has the characteristics of being the most complete in functionality, data, stable in operation, and high in resource utilization;
- 2>Support all currently produced or under development tablets of DT, such as those that support network port static, network port static giant frame, network port dynamic, optical port dynamic, and wireless static methods. The supported tablet types are the most comprehensive, which is beneficial for secondary development users. One development is compatible with multiple versions, saving manpower resources, improving efficiency, and eliminating the need for re integration;
- 3>Compatible with different tablets (old and new tablets), there is no need to worry about changing tablets and re integrating development;
- 4>As the company's future main product for external promotion, we will continue to maintain and support it

(3) Is the integrated comprehensive version complex?

For old users who have already been integrated, we have taken into consideration during development to achieve inheritance and compatibility as much as possible. Therefore, replacing the old version SDK is not very complicated. Simply replace the header and library files, and modify individual interface parameter types. The mainstream program and many interface functions are the same as the old version. [The following mainly introduces the similarities and differences of these versions.](#)

(4) Header files, library files, and dependency libraries

version	Header file	Library files	Dependency library	Tool software
Network port static version Ethernet static giant frame version	HbDllError.h HbDllType.h HbFpdDll.h	HBI_FPD.DLL HBI_FPD.lib	Opencv_Core249.91l Opencv_Highgui249.91l Opencv_Imgproc249.91l	XDiscovery_Static.exe 
Network port dynamic version	HbDllError.h HbDllType.h HbFpdDll.h	HBI_FPD_E.DLL HBI_FPD_E.Lib	Opencv_Core249.91l Opencv_Highgui249.91l Opencv_Imgproc249.91l	XDiscovery_Dynamic_E.Exe 
Integrated version	HbiError.h HbiFpd.h	HBISDKApi.dll HBISDKApi.lib	Opencv_World341.dll	XDiscovery.exe 

(Ethernet, optical, wireless; static, dynamic)	HbiType.h			
--	-----------	--	--	--

(5) Change in callback function

Note: Image events, note that the buffer is a pointer to the structure

version	Callback function	Remarks
Network port static version Ethernet static giant frame version Network port dynamic version	HandleCommandEvent (unsigned char command, void * buff, int len, int nid)	ByteEventid: Parameter 1, Event ID, refer to enum eCallbackEventCommType in HbDllType.h Buff: Parameter 2, configure structure pointer or image buffer pointer Len: Parameter 5, such as data size or status
Integrated version (Ethernet, optical, wireless; static, dynamic)	HandleCommandEventA (void * pvContext, int nDevId, unsigned char byteEventid, void * pvParam1, int nParam2, int nParam3, int nParam4)	PContext: Parameter 1, upper computer object pointer, can be null UfpdId: Parameter 2, such as tablet ID ByteEventid: Parameter 3, Event ID, refer to enum eCallbackEventCommType in HbiType.h PVEventParam1: Parameter 4, configure structure pointer or image structure pointer NEventParam2: Parameter 5, such as data size or status NEventParam3: Parameter 6, such as frame number, frame ID NEventParam4: Parameter 7, such as frame rate or state, etc

(6) Similarities and Differences in Parameter Types

Example:

interface	Interface and parameters	Network port static version	Network port dynamic version	Comprehensive version
Single frame/continuous acquisition	HBI_LiveAcquisition (void * handle, FPD_AQC_MODE mode); HBI_SingleAcquisition (void * handle, FPD_AQC_MODE mode); Note parameters: The number and name of parameters are the same, but the members are different.	typedef struct FPD_AQC_MODE_St { EnumLIVE_ACQUISITION eLivetype; int ngrouppno; int nframesum; int ndiscard; int nframeid; } FPD_AQC_MODE;	typedef struct FPD_AQC_MODE_St { EnumIMAGE_ACQ_MODE aqc_mode; int ngrouppno; int nframesum; int ndiscard; int nframeid; int nLiveMode; BOOL isOverLap; BOOL bSimpleGT; EGRAY_MODE nGrayBit; } FPD_AQC_MODE;	typedef struct FPD_AQC_MODE_St { EnumIMAGE_ACQ_C MD eAqccmd; EnumLIVE_ACQUISITION eLivetype; int ngrouppno; int nAcqnumber; int ndiscard; int nframeid; } FPD_AQC_MODE;
Generate Template	The interface functions and parameters are different, with significant differences	HBI_ImmediateGenerateTemplate (void * handle, EnumIMAGE_ACQ_MODE mode, int bpreview);	HBI_GenerateTemplateEx (void * handle, EnumGENERATE_TEMPLATE_TYPE templateType);	HBI_GenerateTemplate (void * handle, EnumIMAGE_ACQ_C MD mode, int bpreview);
Download template	The interface functions and parameters are different, with significant differences	HBI_DownloadTemplate (void * handle, DOWNLOAD_FILE * downloadfile); HBI_DownloadTemplateEx (void * handle,	HBI_DownloadTemplate (void * handle, HBIDOWNLOAD_FILE * downloadfile); HBI_DownloadOffsetTemplate (void * handle);	HBI_DownloadTemplate (void * handle, DOWNLOAD_FILE * downloadfile); HBI_DownloadTemplateByType (void * handle,

		DOWNLOAD_MODE * downloadmode); HBI_DownloadTemplateByType (void * handle, int filetype); HBI_DownloadTemplate (void * handle, DOWNLOAD_FILE * downloadfile);	HBI_DownloadFileToFirmware (void * handle, DOWNLOAD_TOILE&d param);	int filetype);
Single frame acquisition Prepare interface	Interface function names are different	HBI_Prepare (void * handle);	HBI_Prepare (void * handle);	HBI_SinglePrepare (void * handle);
Single frame acquisition setting Prepare time	The interface functions and parameters are different, with significant differences	HBI_SetPreAcqTm	HBI_SetSinglePrepareTime;	HBI_SetSinglePrepareTime
Continuous collection setting frame rate	The interface functions and parameters are different, with significant differences	Ethernet static Jumbo packet: HBI_SetSelfDumpingTime; Network port static/wireless: HBI_SetLiveAcquisition Time.	HBI_SetAcqSpanTm;	Static Jumbo packet at Ethernet Port/Dynamic Ethernet Port/Optical Port: HBI_SetSelfDumpingTime Network port static/wireless: HBI_SetLiveAcquisitionTime;

(7) Support for "dual panels" solutions

Yes, only the **latest version** supports the "dual panels" solution.

(8) Support for multi template management

The current template file and log file are automatically generated based on the dynamic library Dll path, therefore, they are in the same directory as the dynamic library Dll file.

version	Template path	Naming rules
Network port static version Ethernet static jumbo packet version	DLL file path\template\	Offset template: offset.raw, Gain templates: gaina.raw, gainb.raw Defect template: defect. raw
Network port dynamic version	DLL file path\template\	Divide into multiple templates, such as S01B1F01P5L0000T0000W0000H0000 S01-panel size is 1, B1-Binning1x1, P5-PGA Level is 5, L0000T0000W0000H0000-ZOOM parameters. Template file storage path: DLL file path\template\S01B1F01P5L0000T0000W0000H0000\ Offset template: offset.raw, Gain templates: gaina.raw, gainb.raw Defect template: defect. raw
Integrated version (Ethernet, optical, wireless; static, dynamic)	DLL file path\hbi_id0_template\ Among them, "Tablet A" log hbi_id0_template, on the contrary, "Plate B" means HBI_Id1_TEMPLATE. E.	Offset template path: DLL file path\hbi_id0_template, id0 represents panel A, panel ID is 0, id0 represents panel B, panel ID is 1; Offset template: offsetS02B1F00250. raw represents panel size is 2, and binning1x1,00250 is a continuous acquisition time interval used to mark the frame rate; Gain and defect template paths: DLL file path\hbi_id0_template\S02B1P5V000 \. S02B1P5V000, S02-panel size is 2, B1=binning1x1, P5-PGA Level 5, V000- not support yet.



DT Imaging Technology (Shanghai) Co., Ltd

		Gain templates: gaina.raw, gainb.raw Defect template: defect. raw
--	--	--

Example:

Taking Panel-1613 as an example							
Key letters							
S	B	P	V	Z	F	r	m
Panel size	Binning Type	PGA Level	High Voltage (Dose parameters)	ZOOM Rect	FPS (Affects offset templates)	Rotate	Mirror (1-Vertically/Mirror,2-Horizontal/Mirror)
Reference tablet model information table	1-binning1x1 2-binning1x1 3-binning1x1 4-binning1x1	1-11 represents different PGA level, refer to PGA level table	Currently, due to the inability to obtain specific parameters, it defaults to 0	Rect(left,top,width,height)	Dynamic 1000ms/dumping time Static 1000ms/living time Currently, for character alignment, it has been expanded by 1000 times	1-Right Rotate 90° 2-Left Rotate 90° 0 or other default images do nothing	1-Vertically Mirror 2-Horizontal Mirror 3-5 indicates that the firmware has completed different image processing, independent of template generation, 0 or other default images are not
Offset template (18-bit raw file)		Template file		Notes			
Example 1		offset50381P6F20000		Eg. time interval 50ms, 20 frames represent 1000000/50=F20000. If the time interval is 2000ms, 0.5 frames represent F00500, the time interval is 100000, and 0.01 frames represent F000010			
Example 2		offset50381P6F20000Z0000049612480032		Z0000049612480032 indicates the ZOOM rect parameter, left=0, top=496, width=1248, height=32			
Example 3		offset50381P6F20000Z0000049612480032_r1m2		r1m2 indicates that the image has been rotated right and mirrored horizontally			
Gain template (32-bit raw file)		Folder		Template file		Notes	
Example 1		S0381P6V000		gaina.raw, gainb.raw		Firmware only updates gaina.raw	
Example 2		S0381P6V000Z0000049612480032		gaina.raw, gainb.raw		Z0000049612480032 indicates the ZOOM rect parameter, left=0, top=496, width=1248, height=32	
Example 3		S0381P6V000		gaina_r1m2.raw, gainb_r1m2.raw		Indicates that the image has been rotated right and horizontally mirrored and generate gain template	
		S0381P6V000Z0000049612480032		gaina_r1m2.raw, gainb_r1m2.raw		Indicates that the image has been rotated right and horizontally mirrored and generate gain template	
Defect template (18-bit raw file)		Folder		Template file		Notes	
Example 1		S0381P6V000		defect.raw		The grayscale value of 65535 represents bad points, and 0 represents normal points	
Example 2		S0381P6V000Z0000049612480032		defect.raw		Z0000049612480032 indicates the ZOOM rect parameter, left=0, top=496, width=1248, height=32	
Example 3		S0381P6V000		defect_r1m2.raw		Indicates that the image has been rotated right and horizontally mirrored and generate defect template	
		S0381P6V000Z0000049612480032		defect_r1m2.raw		Indicates that the image has been rotated right and horizontally mirrored and generate defect template	
Notes							
1. The ZOOM parameter only supports binning1x1.							
2. The offset template is related to the panel size, binning type, PGA gear, frame rate, Zoom parameters, and software side rotation mirror parameters.							
3. The gain and defect templates are related to the panel size, binning type, PGA gear, Zoom parameters, and software side rotation mirror parameters.							
4. At present, the rotation mirror processing sequence is: rotate first and then mirror.							

激活 Win

测试 Wind

Appendix V: Window width and window level algorithm

double dFactor, min, max;

int nPixelVal;

min = (2*window_center - window_width)/2.0 + 0.5;

max = (2*window_center + window_width)/2.0 + 0.5;

dFactor = 255.0/(double)(max - min);

for (i = 0; i < nNumPixels; i++)

{

if (pixel_val < min)

{

disp_pixel_val = 0;

continue;

}

if (pixel_val > max)

{

disp_pixel_val = 255;

continue;

}

nPixelVal = (int)((pixel_val - min)*dFactor);

if (nPixelVal < 0)

disp_pixel_val = 0;

else if (nPixelVal > 255)

disp_pixel_val = 255;

else

disp_pixel_val = nPixelVal;

}